

□□□□□□AWS□□

□□ □□ (Tomoyuki Mano) <tomoyukimano@gmail.com>

Version 1.0, 2021-06-14

Table of Contents

1. 序	2
1.1. 本書の目的	2
1.2. 本書の構成	2
1.3. AWSの紹介	3
1.4. 環境	3
1.5. 前提	4
1.6. 開発環境	4
1.7. 開発環境の構築	4
2. 基礎	6
2.1. 基礎	6
2.2. 基礎	8
3. AWS	11
3.1. AWS	11
3.2. AWS	11
3.3. Region と Availability Zone	13
3.4. AWS	14
3.5. CloudFormation と AWS CDK	18
4. Hands-on #1: EC2の操作	22
4.1. 序	22
4.2. SSH	22
4.3. 基礎	23
4.4. 基礎	27
4.5. 序	33
5. 基礎	34
5.1. 基礎	34
5.2. GPU 基礎	34
6. Hands-on #2: AWS の操作	38
6.1. 序	38
6.2. 基礎	38
6.3. 基礎	41
6.4. 基礎	42
6.5. Jupyter Notebook 基礎	44
6.6. PyTorch基礎	46
6.7. 基礎! MNIST基礎	49
6.8. 基礎	54
7. Docker 基礎	56
7.1. 基礎	56
7.2. Docker 基礎	57

7.3. Docker	59
7.4. Elastic Container Service (ECS)	62
8. Hands-on #3: AWS	64
8.1. Fargate	64
8.2.	65
8.3. Transformer question-answering	65
8.4.	67
8.5.	71
8.6.	73
8.7.	74
8.8.	76
9. Hands-on #4: AWS Batch	77
9.1. Auto scaling groups (ASG)	77
9.2. AWS Batch	77
9.3.	79
9.4. MNIST (MNIST)	79
9.5.	82
9.6.	86
9.7. Docker image ECR	88
9.8.	91
9.9. Job	95
9.10.	99
9.11.	100
10. Web	102
10.1. — Twitter	102
10.2. REST API	103
10.3. Twitter API	104
11. Serverless architecture	106
11.1. Serverful (Serverful)	106
11.2. Serverless	107
11.3.	108
12. Hands-on #5:	113
12.1. Lambda	113
12.2. DynamoDB	118
12.3. S3	122
13. Hands-on #6: Bashoutter	126
13.1.	126
13.2.	127
13.3.	134
13.4. API	136
13.5. API	138

13.6. Bashoutter GUI	139
13.7.	140
13.8.	141
14.	142
15. Appendix:	143
15.1. AWS	143
15.2. AWS	147
15.3. AWS CLI	149
15.4. AWS CDK	150
15.5. WSL	151
15.6. Docker	154
15.7. Python venv	155
15.8. Docker image	156
16.	158
17.	159
18.	160


~~~~~

<https://github.com/tomomano/learn-aws-by-coding>

~~~~~

~~~~~  
(1000000)~~~~~20210927~~~~~  
~~~~~

~~~~~: AWS~~~~~ (~~~~~360~~~~~)

- Amazon (~~~~~ or Kindle) ⇒ <https://www.amazon.co.jp/dp/4839977607/>
- ~~~~~ (~~~~~ or PDF) ⇒ <https://book.mynavi.jp/ec/products/detail/id=124113>

CDK v2 ~~~~~

~~~~~~~CDK v1~~~~~~~~~AWS 20230601~~~~v1~~~~~~~~~CDK v2~~~~~~~ (~~~~~)

~~~~~~~@takashi-uchida~~~~~~~~~CDK v2~~~~~~~~~(PR#49)~~~~~~~  
[https://github.com/takashi-uchida/learn-aws-by-coding/tree/feature/to\\_cdkv2](https://github.com/takashi-uchida/learn-aws-by-coding/tree/feature/to_cdkv2)

~~~~~

(2026/04/18 ~~~) @takashi-uchida ~~~ CDK v2 ~~~ PR#49 ~~~~~ CDK v2
~~~~~ Issue ~~~~~

~~~~~ ~~~ ~~~~~

Chapter 1. 概要

1.1. 環境構築

本書は2021年S1/S2版の「環境構築」をテーマに、Amazon Web Services (AWS) の環境構築について解説する。

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

Table 1. 環境構築

| | 環境構築 | 環境構築 |
|------------|------|--|
| 環境 (1-4) | 環境構築 | • AWS環境構築 |
| 環境 (5-9) | 環境構築 | • AWS の Jupyter Notebook 環境構築
• 環境構築
• 環境構築 |
| 環境 (10-13) | 環境構築 | • Lambda, DynamoDB, S3 環境
• SNS "Bashoutter" 環境 |

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

本書は、AWS の環境構築について解説する。AWS の環境構築について解説する。AWS の環境構築について解説する。

1.2. 環境構築

本書は「環境構築」をテーマに、AWS の環境構築について解説する。

環境構築

本書は「環境構築」をテーマに、AWS の環境構築について解説する。

- **UNIX** ユーザー: ユーザー名を指定して SSH 接続して、UNIX ユーザー名を指定して Mac 側 Linux 環境に OS をインストールする(インストール済み)環境に Windows 環境を [Windows Subsystem for Linux \(WSL\)](#) として Linux 環境をインストールする ([Section 15.5](#) 参照)
- **Docker**: Docker をインストールして、[Section 15.6](#) を参照
- **Python**: Version 3.6 をインストールして `venv` をインストールして `venv` を [Section 15.7](#) を参照
- **Node.js**: version 12.0 以上をインストール
- **AWS CLI**: [Version 2](#) をインストールして [Section 15.3](#) を参照
- **AWS CDK**: Version 1.100 をインストールして Version 2 をインストールして [Section 15.4](#) を参照
- **AWS** 環境: AWS API をインストールして (secret key) をインストールして [Section 15.3](#) を参照

1.4.1. Docker Image

Python, Node.js, AWS CDK などのアプリケーション/インフラコード Docker image として
パッケージ化して Docker 環境で実行可能にする

1111111111111111

```
$ docker run -it tomomano/labc
```

□□ Docker image □□□□□□□□ [Section 15.8](#) □□□□□□□□

1.5. □□□□

[illegible]

- **Python** 書籍タイトル: Pythonの教科書
Pythonの教科書の目次を参照してください。 Python のインストールと環境構築
- **Linux** 書籍タイトル: Linux のコマンドライン Linux Linux
Linux のコマンドラインの目次を参照してください。 The Linux Command Line by William Shotts
Linux のインストールと環境構築

1.6. □□□□□□□□

□ □

<https://github.com/tomomano/learn-aws-by-coding>

1.7. □□□□□□□□□□□□□□□□

- `monospace letter` `monospace`
- `monospace letter` `monospace` `$` `monospace` `$` `monospace` `&` `monospace` `$` `monospace`

□□□□□□□□□□□□□□□□□□



□□□□□□□□□□



□□□□□□□□□□□□□□□□



□□□□□□□□□□□□□□



□□□□□□□□□□□□□□

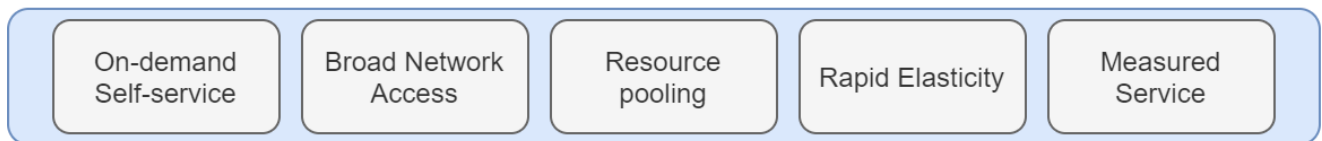
Chapter 2. □□□□□□

2.1. □□□□□□□

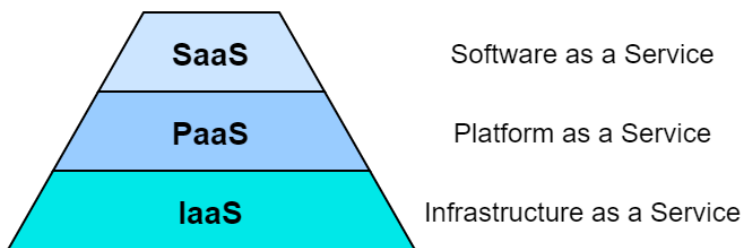
[illegible]

Figure 2: The NIST Definition of Cloud Computing

Essential characteristics



Service models



Deployment models

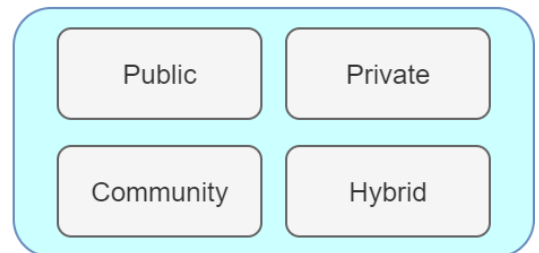


Figure 2. The NIST Definition of Cloud Computing

[illegible]

- **On-demand self-service** 用戶可以隨時隨地自助獲取資源
- **Broad network access** 用戶可以通過多種設備訪問資源
- **Resource pooling** 用戶可以共享資源池中的資源
- **Rapid elasticity** 用戶可以根據需要快速擴展或縮減資源
- **Measured service** 用戶可以根據實際使用量付費

...

CPU CPU CPU

インターネットに接続されたコンピュータは、インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ

インターネットに接続されたコンピュータは、CPUが1000個、メモリが100GB、ストレージが1TBから10TBまであり、
インターネットを通じて1000 CPU、メモリ、ストレージを共有し、インターネットを通じてGoogle
Drive、Dropbox、Amazon S3などのサービスを利用する。インターネットを通じて100%の
インターネットに接続されたコンピュータ

インターネットに接続されたコンピュータは、インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ。インターネットを通じて100%のインターネットに
接続されたコンピュータ。インターネットを通じて100%のインターネットに接続された
コンピュータ。インターネットを通じて100%のインターネットに接続されたコンピュータ

インターネットに接続されたコンピュータ

インターネットに接続されたコンピュータは、インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ

図 2 The NIST Definition of Cloud Computing インターネットを通じて他のコンピュータと通信し、LAN
(Figure 2)

- **Software as a Service (SaaS):** インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ。Google Drive、Slack
インターネットを通じて100%のインターネットに接続されたコンピュータ
- **Platform as a Service (PaaS):** インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ。API、データベース、PaaS
インターネットを通じて100%のインターネットに接続されたコンピュータ。Google App
Engine、Heroku、PaaS
- **Infrastructure as a Service (IaaS):** インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ。IaaS、AWS EC2、PaaS

インターネットに接続されたコンピュータは、インターネットを通じて他のコンピュータと通信し、LAN
ネットワークに接続されたコンピュータ。インターネットを通じて100%のインターネットに
接続されたコンピュータ。インターネットを通じて100%のインターネットに接続された
コンピュータ。インターネットを通じて100%のインターネットに接続されたコンピュータ
インターネットを通じて100%のインターネットに接続されたコンピュータ。PaaS
(インターネットを通じて100%のインターネットに接続されたコンピュータ) PaaS
インターネットを通じて100%のインターネットに接続されたコンピュータ。PaaS
インターネットを通じて100%のインターネットに接続されたコンピュータ。PaaS

SaaS インターネットを通じて100%のインターネットに接続されたコンピュータ インターネットを通じて100%のインターネットに
接続されたコンピュータ。SNS (Chapter 13) インターネットを通じて100%のインターネットに
接続されたコンピュータ

インターネットを通じて100%のインターネットに接続されたコンピュータ。Function as a Service (FaaS) インターネットを通じて100%のインターネットに
接続されたコンピュータ。Chapter 12 インターネットを通じて100%のインターネットに
接続されたコンピュータ。インターネットを通じて100%のインターネットに接続された
コンピュータ

図 2 The NIST Definition of Cloud Computing インターネットを通じて他のコンピュータと通信し、LAN
(Figure 2)


```
Macintosh HD — top — 80x24
Processes: 210 total, 2 running, 9 stuck, 199 sleeping, 901 threads 23:30:03
Load Avg: 1.40, 1.75, 1.00 CPU usage: 4.15% user, 4.40% sys, 91.44% idle
SharedLibs: 1648K resident, 0B data, 0B linkedit,
MemRegions: 31278 total, 1892M resident, 117M private, 564M shared.
PhysMem: 5893M used (1191M wired), 10G unused.
VM: 523G vsize, 1026M framework vsize, 0(0) swapins, 0(0) swapouts.
Networks: packets: 12105/8925K in, 11907/1964K out.
Disks: 00156/2205M read, 21235/425M written.

PID COMMAND %CPU TIME #TH #WQ #PORT MEM PURG CMPR PGRP PPID
592 screencaptur 0.0 00:00.02 7 5 55+ 1952K+ 20K+ 0B 262 262
590 mdworker 0.0 00:00.01 3 0 44 2032K 0B 0B 590 1
589 mdworker 0.0 00:00.01 3 0 44 1572K 0B 0B 589 1
588 top 1.7 00:00.51 1/1 0 22+ 2860K 0B 0B 588 584
584 bash 0.0 00:00.00 1 0 15 588K 0B 0B 584 583
583 login 0.0 00:00.01 3 1 28 1228K 0B 0B 583 402
574 auditd 0.0 00:00.00 2 0 25 560K 0B 0B 574 1
567 System Prefe 0.0 00:03.23 3 0 270 39M 8364K 0B 567 1
561 systemstatsd 0.0 00:00.01 2 1 19 1040K 0B 0B 561 1
560 com.apple.We 0.0 00:01.42 9 0 229 25M 0B 0B 560 1
558 com.apple.We 0.0 00:05.07 15 3 224 151M 1716K 0B 558 1
555 bash 0.0 00:00.00 1 0 15 604K 0B 0B 555 554
554 login 0.0 00:00.01 3 1 28 1176K 0B 0B 554 482
550 bash 0.0 00:00.00 1 0 15 608K 0B 0B 550 549
```

터미널을 실행하는 방법

터미널을 실행하는 방법

터미널을 실행하는 방법

터미널을 실행하는 방법...

터미널

Terminal

터미널을 실행하는 방법Terminal

WindowsMac

터미널을 실행하는 방법

터미널을 실행하는 방법"

Chapter 3. AWS

3.1. AWS

Amazon은 클라우드 컴퓨팅 서비스인 AWS를 통해 기업들에게 다양한 클라우드 서비스를 제공합니다.

AWS (Amazon Web Services)는 Amazon이 제공하는 클라우드 서비스입니다. AWS는 Amazon이 2006년에 시작했으며, 2021년에는 미국에서 총 매출의 32%를 차지했습니다. AWS는 Netflix, Slack, Amazon 등 다양한 기업들이 사용하고 있습니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다.

AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다.

3.2. AWS 서비스 카테고리

Figure 4 AWS 서비스 카테고리

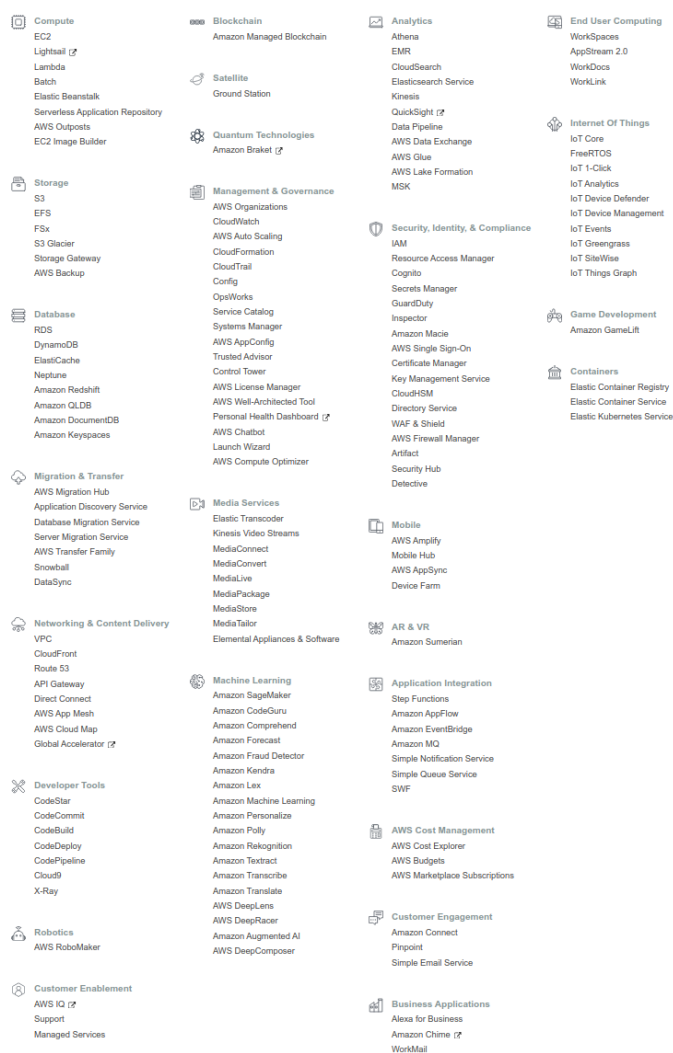


Figure 4. AWS 서비스 카테고리

이 다이어그램은 AWS 서비스의 다양한 카테고리를 보여줍니다. 각 카테고리는 아이콘과 함께 나열된 서비스 목록을 포함하고 있습니다.

アプリケーションを構築する際の考慮事項

アプリケーションがAR/VR 環境で動作する必要がある場合は、アプリケーションが 170FPS で動作する必要がある (図 3.2.1)

AWS には、アプリケーションを構築する際に考慮する必要がある多くのサービスがあります。AWS のサービスは、アプリケーションの構築に役立ちます。AWS のサービスは、アプリケーションの構築に役立ちます。AWS のサービスは、アプリケーションの構築に役立ちます。

図 3.2.1 AWS のサービスは、アプリケーションの構築に役立ちます。

3.2.1. 図 3.2.1



EC2 (Elastic Compute Cloud) は、アプリケーションを構築する際に考慮する必要があるサービスです。Chapter 4, Chapter 6, Chapter 9 を参照してください。



Lambda Function as a Service (FaaS) は、アプリケーションを構築する際に考慮する必要があるサービスです。(Chapter 11) を参照してください。

3.2.2. 図 3.2.2



EBS (Elastic Block Store) は、EC2 インスタンスに接続する必要があるサービスです。図 3.2.2 (OS) は、アプリケーションの構築に役立ちます。



S3 (Simple Storage Service) は、Object Storage サービスです。API は、アプリケーションの構築に役立ちます。(Chapter 11) を参照してください。

3.2.3. 図 3.2.3



DynamoDB NoSQL は、データベースサービスです。(図 3.2.3 **mongoDB** は、データベースサービスです) (Chapter 11) を参照してください。

3.2.4. 図 3.2.4



VPC (Virtual Private Cloud) は、AWS のサービスです。EC2 インスタンスは、VPC に接続する必要があります。



API Gateway は、API サービスです。(Lambda は、API サービスです) (Chapter 13) を参照してください。

3.3. Region & Availability Zone

AWS 提供多個地理區域（Region）及 Availability Zone (AZ) 服務 (Figure 5) 以確保服務的高可用性。

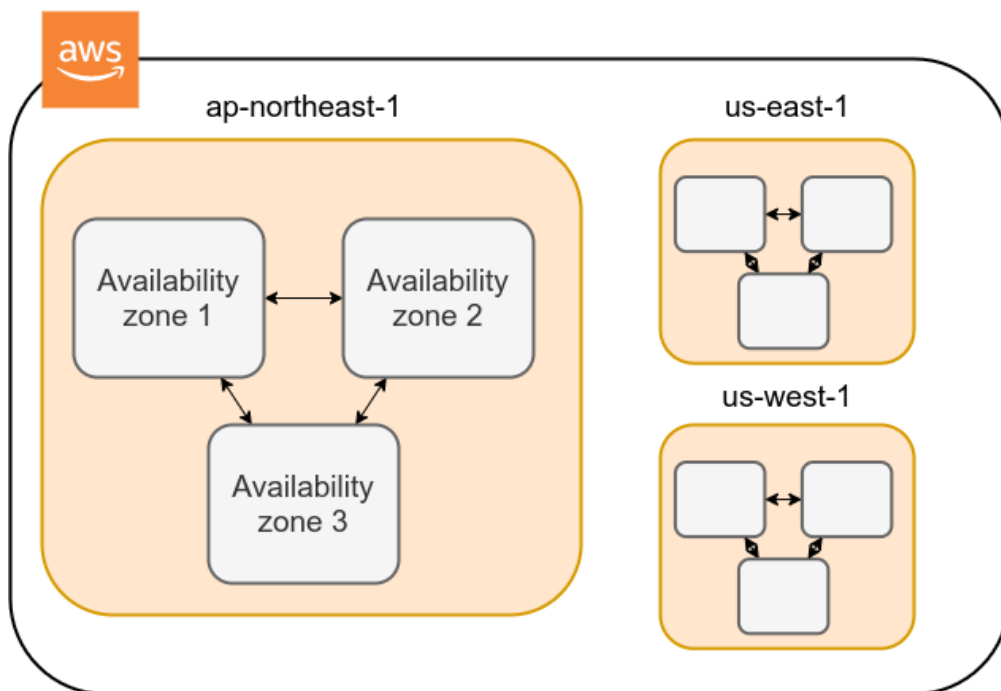


Figure 5. AWS Region & Availability Zones

每個 Region 包含多個 Availability Zones。AWS 目前提供 25 個 Region，如 Figure 6 所示。每個 Region 都有唯一的 ID，例如 `ap-northeast-1`，而 `us-east-2` 則是即將推出的 Region。



Figure 6. Regions in AWS (URL: <https://aws.amazon.com/about-aws/global-infrastructure/>)

AWS 提供多種服務，包括 EC2、S3 等 (Figure 7, 服務列表)。

這些服務可以部署在多個 Availability Zones 中，以確保高可用性 (i.e. 容錯能力)。

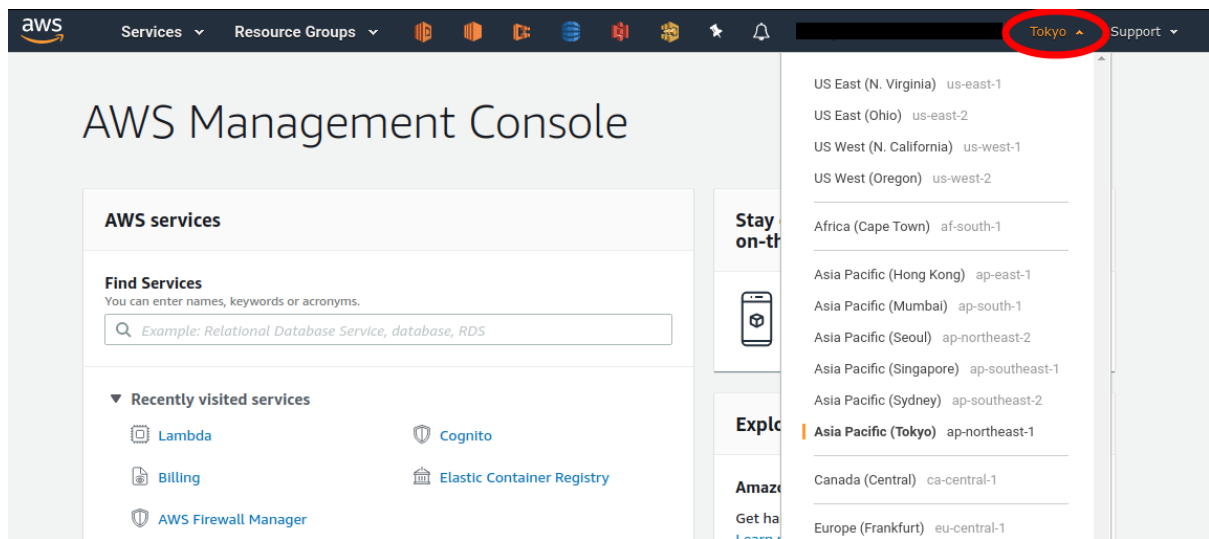


Figure 7. AWS Management Console

Availability Zone (AZ) is a physically separate location within the same Region, designed to be isolated from failures in other Availability Zones. Each Region has at least two Availability Zones, and some Regions have more than two. Availability Zones are connected to each other via low-latency, high-speed, fiber-optic networks. AWS services are distributed across multiple Availability Zones to ensure high availability and fault tolerance. For example, if one Availability Zone experiences a failure, the service can continue to operate in the other Availability Zones. This ensures that your application remains available even in the event of a regional outage.



AWS

AWS provides a variety of services that are distributed across multiple Availability Zones to ensure high availability and fault tolerance. For example, Amazon EC2 instances can be launched in multiple Availability Zones to ensure that your application remains available even in the event of a regional outage. This is achieved through a process called **multi-AZ deployment**, which involves distributing your application across multiple Availability Zones. This ensures that your application remains available even in the event of a regional outage.



AWS Educate Starter Account

When you create an AWS Educate Starter Account, you are automatically assigned to the **us-east-1** region. This region is located in the Northern Virginia area of the United States. It is important to note that the **us-east-1** region is not available in all countries. If you are located in a country where the **us-east-1** region is not available, you may need to create an account in a different region.

Further reading

- [AWS documentation "Regions, Availability Zones, and Local Zones"](#)

3.4. AWS Regions and Availability Zones

AWS provides a variety of services that are distributed across multiple Availability Zones to ensure high availability and fault tolerance. This is achieved through a process called **multi-AZ deployment**, which involves distributing your application across multiple Availability Zones.

AWS provides a variety of services that are distributed across multiple Availability Zones to ensure high availability and fault tolerance. This is achieved through a process called **multi-AZ deployment**, which involves distributing your application across multiple Availability Zones.

3.4.1. Multi-AZ deployment

AWS provides a variety of services that are distributed across multiple Availability Zones to ensure high availability and fault tolerance. This is achieved through a process called **multi-AZ deployment**, which involves distributing your application across multiple Availability Zones.

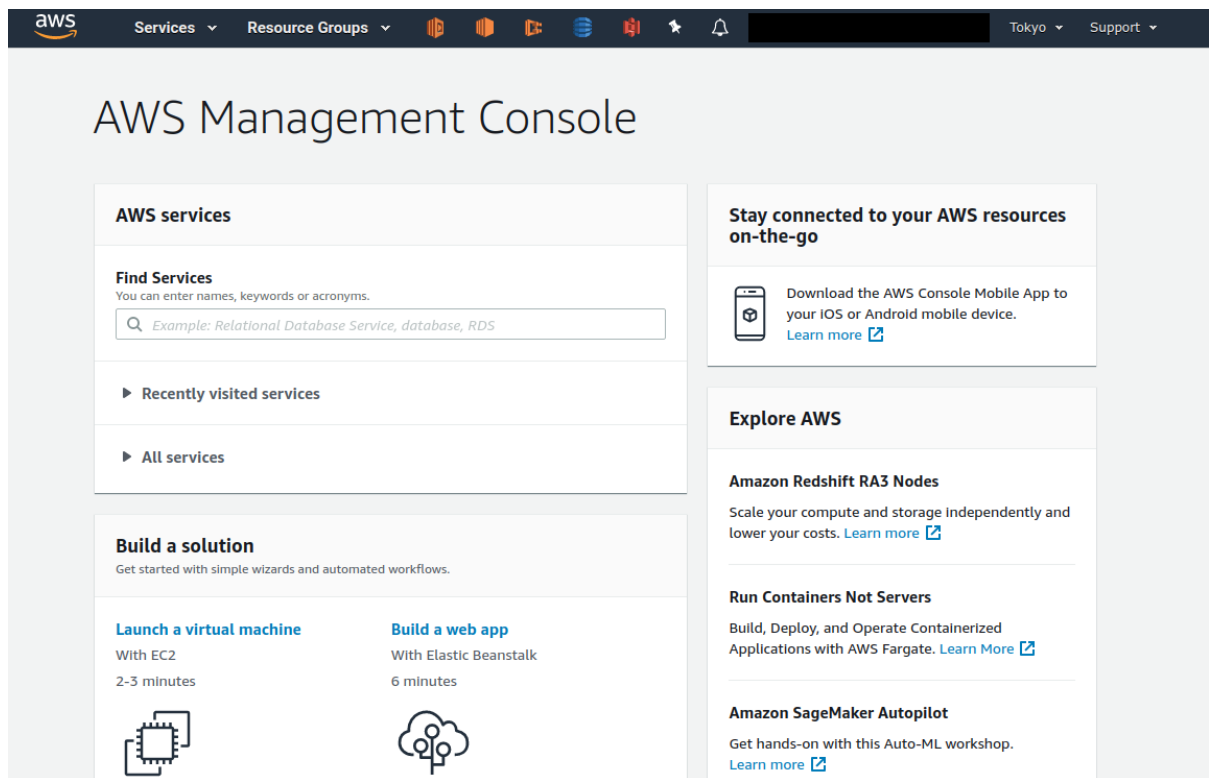


Figure 8. AWS

EC2 S3 AWS GUI (Graphical User Interface)

API AWS AWS

3.4.2. API

API (Application Programming Interface) AWS API GET, POST, DELETE REST API (REST API Section 10.2) REST API

AWS CLI UNIX AWS API CLI (Command Line Interface) CLI SDK (Software Development Kit)

- Python ⇒ boto3
- Ruby ⇒ AWS SDK for Ruby
- Node.js ⇒ AWS SDK for Node.js

API

S3 (Bucket) AWS CLI

```
$ aws s3 mb s3://my-bucket --region ap-northeast-1
```

my-bucket ap-northeast-1

Python boto3

```
1 import boto3
2
3 s3_client = boto3.client("s3", region_name="ap-northeast-1")
4 s3_client.create_bucket(Bucket="my-bucket")
```

EC2

```
$ aws ec2 run-instances --image-id ami-xxxxxxx --count 1 --instance-type t2.micro
--key-name MyKeyPair --security-group-ids sg-903004f8 --subnet-id subnet-6e7f829e
```

t2.micro (1 vCPU, 1.0 GB RAM) (Chapter 4)

Python

```
1 import boto3
2
3 ec2_client = boto3.client("ec2")
4 ec2_client.run_instances(
5     ImageId="ami-xxxxxxx",
6     MinCount=1,
7     MaxCount=1,
8     KeyName="MyKeyPair",
9     InstanceType="t2.micro",
10    SecurityGroupIds=["sg-903004f8"],
11    SubnetId="subnet-6e7f829e",
12 )
```

API

CPU RAM

API

3.4.3. AWS CLI

AWS CLI AWS CLI S3 (EC2) aws s3



AWS CLI Section 15.3



S3



~~~~~AWS~~~~~  
 ~~~~~ (AWS\_ACCESS\_KEY\_ID,  
 AWS_DEFAULT_REGION) ~~~~~ Section 15.3 ~~~~~

~/.aws/credentials
 AWS_SECRET_ACCESS_KEY,

~~~~~S3~~~~~ (Bucket ~~~~~ OS ~~~~~) ~~~~~

```
$ bucketName="mybucket-$(openssl rand -hex 12)"
$ echo $bucketName
$ aws s3 mb "s3://${bucketName}"
```

S3~~~~~ AWS ~~~~~bucketName ~~~~~  
 ~~~~~ aws s3 mb (mb ~ make bucket ~) ~~~~~

~~~~~

```
$ aws s3 ls

2020-06-07 23:45:44 mybucket-c6f9385550a72b5b66f5efe
```

~~~~~



~~~~~ \$ ~~~~~  
 ~~~~~&~~~~~ \$ ~~~~~

~~~~~

```
$ echo "Hello world" > hello_world.txt
$ aws s3 cp hello_world.txt "s3://${bucketName}/hello_world.txt"
```

~~~~~hello\_world.txt ~~~~~

~~~~~

```
$ aws s3 ls "s3://${bucketName}" --human-readable

2020-06-07 23:54:19    13 Bytes hello_world.txt
```

~~~~~

~~~~~

```
$ aws s3 rb "s3://${bucketName}" --force
```

rb ~ remove bucket ~~~~~ ~~~~~ --force

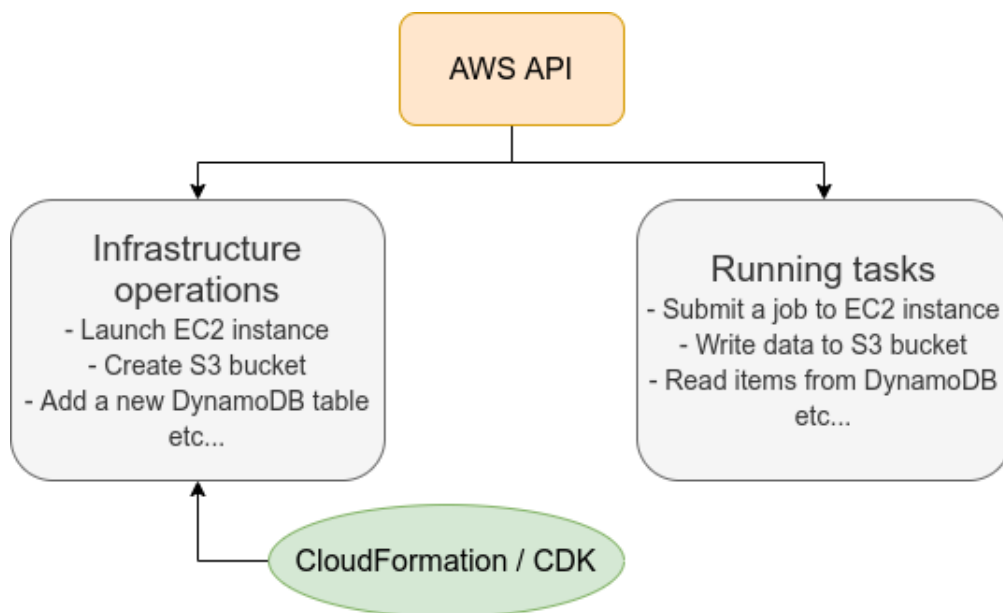
1. **インストール**  
 AWS CLI をインストールする。EC2 上の Lambda, DynamoDB などから AWS CLI を利用する。



S3      EC2      ARN      ARN

### 3.5.1. CloudFormation ☐☐☐ Infrastructure as Code (IaC)

Figure 9: AWS API request rate (requests per second) over time (Figure 9)

[illegible]

AWS CloudFormation CloudFormation CloudFormation  
AWS CloudFormation  
EC2 CloudFormation  
CloudFormation AWS  
CloudFormation  
Infrastructure as Code (IaC)

CloudFormation 使用 **JSON** (JavaScript Object Notation) 来描述云资源。JSON 是一种轻量级的数据交换格式，易于人阅读和机器生成/解析。

```

1  "Resources" : {
2      ...
3      "WebServer": {
4          "Type" : "AWS::EC2::Instance",
5          "Properties": {
6              "ImageId" : { "Fn::FindInMap" : [ "AWSRegionArch2AMI", { "Ref" :
7                  "AWS::Region" },
8                      { "Fn::FindInMap" : [ "AWSInstanceType2Arch", { "Ref" :
9                          "InstanceType" }, "Arch" ] } ] } },
10             "InstanceType" : { "Ref" : "InstanceType" },
11             "SecurityGroups" : [ { "Ref" : "WebServerSecurityGroup" } ],
12             "KeyName" : { "Ref" : "KeyName" },
13             "UserData" : { "Fn::Base64" : { "Fn::Join" : [ "", [
14                 "#!/bin/bash -xe\n",
15                 "yum update -y aws-cfn-bootstrap\n",
16                 "\n/opt/aws/bin/cfn-init -v ",
17                 "    --stack ", { "Ref" : "AWS::StackName" },
18                 "    --resource WebServer ",
19                 "    --configsets wordpress_install ",
20                 "    --region ", { "Ref" : "AWS::Region" }, "\n",
21                 "\n/opt/aws/bin/cfn-signal -e $? ",
22                 "    --stack ", { "Ref" : "AWS::StackName" },
23                 "    --resource WebServer ",
24                 "    --region ", { "Ref" : "AWS::Region" }, "\n"
25             ] } } }
26         } ,
27         ...
28     } ,
29     ...
30 } ,

```

```
"WebServer"                                EC2
OS EC2
```



## Further reading

- [AWS CDK Examples](#): CDKのサンプルコードをまとめたリポジトリ。AWS CDKの基本的なユースケースや、より高度なユースケースのサンプルコードが豊富に紹介されている。

# Chapter 4. Hands-on #1:

## EC2

この章では、AWS Cloud Development Kit (CDK) を使用して、Amazon EC2 インスタンスを構築し、SSH で接続できるようにします。

### 4.1. 準備

この章のコードは GitHub の [handson/ec2-get-started](https://github.com/tomomano/ec2-get-started) リポジトリにあります。



この章のコードは AWS のリポジトリにあります。

この章のコードを実行するには、以下の環境を準備する必要があります。

- **AWS Account:** AWS アカウントと AWS CLI をインストールしてください。 [Section 15.1](#) を参照してください。
- **Python** または **Node.js:** Python (3.6 以上) または Node.js (12.0 以上) をインストールしてください。
- **AWS CLI:** AWS CLI をインストールしてください。 [Section 15.3](#) を参照してください。
- **AWS CDK:** AWS CDK をインストールしてください。 [Section 15.4](#) を参照してください。
- **Git:** Git をインストールしてください。 [Section 15.8](#) を参照してください。

```
$ git clone https://github.com/tomomano/learn-aws-by-coding.git
```

この章のコードは <https://github.com/tomomano/learn-aws-by-coding> リポジトリにあります。

#### Docker

Python, Node.js, AWS CDK をインストールする必要がある場合、Docker image を使用して Docker をインストールすることもできます。

この章のコードは [Section 15.8](#) を参照してください。

### 4.2. SSH

SSH (secure shell) は Unix 系オペレーティングシステムで標準的に提供される、リモートホストに接続するためのプロトコルです。

SSH は Linux/Mac 上で標準的に提供されています。Windows 上では WSL (Windows Subsystem for Linux) を使用して SSH を実行できます。 ( [Section 1.4](#) を参照 )

SSH を実行するには、ホスト名 ( `<host name>` )、IP アドレス、DNS 名、ユーザー名 ( `<user name>` ) を指定する必要があります。

```
$ ssh <user name>@<host name>
```

SSH (Public Key Cryptography) EC2 (Public key)(Private key) EC2 (Public key) (Private key) EC2

SSH `-i` `--identity_file`

```
$ ssh -i Ec2SecretKey.pem <user name>@<host name>
```

### 4.3.

Figure 10

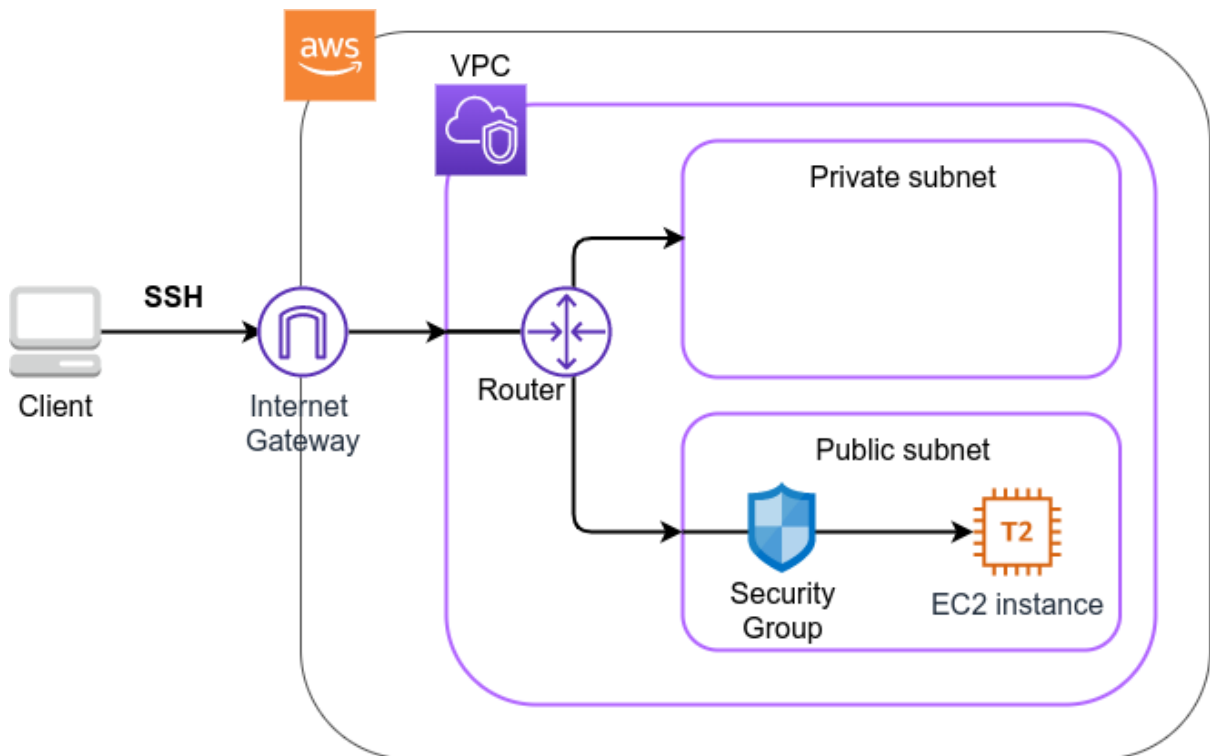


Figure 10. #1

VPC (Virtual Private Cloud) public subnet EC2 (Elastic Compute Cloud) Security Group EC2 SSH

Figure 10 CDK

([handson/ec2-get-started/app.py](https://github.com/hands-on-aws/ec2-get-started/app.py))

```
1 class MyFirstEc2(Stack):
```

```

2
3     def __init__(self, scope: Construct, construct_id: str, key_name: str, **
kwargs) -> None:
4         super().__init__(scope, construct_id, **kwargs)
5
6         ①
7         vpc = ec2.Vpc(
8             self, "MyFirstEc2-Vpc",
9             max_azs=1,
10            ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),
11            subnet_configuration=[
12                ec2.SubnetConfiguration(
13                    name="public",
14                    subnet_type=ec2.SubnetType.PUBLIC,
15                )
16            ],
17            nat_gateways=0,
18        )
19
20        ②
21        sg = ec2.SecurityGroup(
22            self, "MyFirstEc2Vpc-Sg",
23            vpc=vpc,
24            allow_all_outbound=True,
25        )
26        sg.add_ingress_rule(
27            peer=ec2.Peer.any_ipv4(),
28            connection=ec2.Port.tcp(22),
29        )
30
31        ③
32        host = ec2.Instance(
33            self, "MyFirstEc2Instance",
34            instance_type=ec2.InstanceType("t2.micro"),
35            machine_image=ec2.MachineImage.latest_amazon_linux(),
36            vpc=vpc,
37            vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
38            security_group=sg,
39            key_name=key_name
40        )

```

① VPC

② security group (SG) 22 (SSH) 4

③ VPC SG EC2 t2.micro Amazon Linux OS



### 4.3.1. VPC (Virtual Private Cloud)

VPC 是什么



VPC

□

AWS

Amazon Virtual Private Cloud (VPC) is a service that lets you provision a virtual private cloud within a public cloud (e.g., Amazon Web Services) to host your Amazon EC2 instances and other AWS resources. A VPC is a logically isolated section of the AWS cloud where you can launch resources in your own virtual network.

Amazon VPC is a service that lets you provision a virtual private cloud within a public cloud (e.g., Amazon Web Services) to host your Amazon EC2 instances and other AWS resources. A VPC is a logically isolated section of the AWS cloud where you can launch resources in your own virtual network.

Amazon VPC is a service that lets you provision a virtual private cloud within a public cloud (e.g., Amazon Web Services) to host your Amazon EC2 instances and other AWS resources. A VPC is a logically isolated section of the AWS cloud where you can launch resources in your own virtual network.

```
1 vpc = ec2.Vpc(  
2     self, "MyFirstEc2-Vpc",  
3     max_azs=1,  
4     ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),  
5     subnet_configuration=[  
6         ec2.SubnetConfiguration(  
7             name="public",  
8             subnet_type=ec2.SubnetType.PUBLIC,  
9         )  
10    ],  
11    nat_gateways=0,  
12 )
```



- `max_azs=1` : Availability zone (AZ) is a logically isolated part of the AWS cloud. You can specify the number of AZs that you want your VPC to span. The default is 1.
- `ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23")` : Amazon VPC uses IPv4 CIDR notation to specify the IP address range for your VPC. The default is `10.10.0.0/23`, which provides 512 IP addresses. You can specify a different CIDR range, but it must be a /23 or larger. For example, `10.10.0.0/24` provides 256 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses.
- `subnet_configuration=...` : Amazon VPC uses subnets to divide the VPC into smaller, logically isolated sections of the AWS cloud. You can specify the configuration for each subnet. The default is a public subnet. You can also specify a private subnet. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses.
- `natgateways=0` : Amazon VPC uses NAT Gateways to provide internet access to instances in a private subnet. You can specify the number of NAT Gateways that you want to create. The default is 0. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses. You can also specify a range of IP addresses, such as `10.10.1.255`, which provides 512 IP addresses.

### 4.3.2. Security Group

Security group (SG) は EC2 インスタンスに適用されるネットワークセキュリティグループです。IP アドレス範囲 (IP アドレス範囲) を指定して、特定の IP アドレス範囲 (IP アドレス範囲) から特定のポート (ポート) への接続を許可または拒否します。

以下は、SG を作成する Python コードです。

```
1 sg = ec2.SecurityGroup(
2     self, "MyFirstEc2Vpc-Sg",
3     vpc=vpc,
4     allow_all_outbound=True,
5 )
6 sg.add_ingress_rule(
7     peer=ec2.Peer.any_ipv4(),
8     connection=ec2.Port.tcp(22),
9 )
```

このコードは、SSH (ポート 22) を許可する SG を作成します。sg.add\_ingress\_rule(peer=ec2.Peer.any\_ipv4(), connection=ec2.Port.tcp(22)) は、任意の IPv4 アドレスからポート 22 への接続を許可します。allow\_all\_outbound=True は、EC2 インスタンスから任意の IP アドレスへの任意のポートへの接続を許可します。



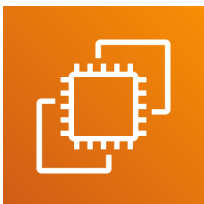
SSH はポート 22 で実行されます。



このコードは、SSH を許可する SG を作成します。

### 4.3.3. EC2 (Elastic Compute Cloud)

EC2 インスタンス



EC2 は AWS のクラウドコンピューティングサービスです。EC2 インスタンス (instance) は、仮想マシン (VM) のように動作します。

EC2 インスタンスの仕様は [Table 2](#) に示されています。 (Table 2) EC2 インスタンスの仕様は ["Amazon EC2 Instance Types"](#) に示されています。

Table 2. EC2 instance types

Instance	vCPU	Memory (GiB)	Network bandwidth (Gbps)	Price per hour (\$)
t2.micro	1	1	-	0.0116
t2.small	1	2	-	0.023



### 4.4.1. Python 環境構築

Python 環境構築は、Python 環境を `venv` で構築する。

手元で `handson/ec2-get-started` ディレクトリに移動する。

```
$ cd handson/ec2-get-started
```

Python 環境を `venv` で構築する。

```
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt
```

Python 環境構築完了。



`venv` の詳細は [Section 15.7](#) を参照してください。



Python 環境構築時に `pip` が `pip3` として `python3 -m pip` で呼び出される。

### 4.4.2. AWS 環境構築

AWS CLI と AWS CDK をインストールし、AWS 環境構築に必要な設定を行う。詳細は [Section 15.2](#) を参照してください。また、[Section 15.3](#) の設定も参照してください。

環境構築に必要な `AWS_ACCESS_KEY_ID` と `AWS_SECRET_ACCESS_KEY` は、`~/.aws/credentials` ファイルに記述する。AWS CLI/CDK を実行する際に `aws configure` コマンドで設定を行う。

### 4.4.3. SSH 環境構築

EC2 インスタンスに SSH 接続するための `SSH` キーペアを作成する。

AWS CLI を使って `HirakeGoma` という名前のキーペアを作成する。

```
$ export KEY_NAME="HirakeGoma"
$ aws ec2 create-key-pair --key-name ${KEY_NAME} --query 'KeyMaterial' --output text >
${KEY_NAME}.pem
```

作成された `HirakeGoma.pem` ファイルを `~/.ssh/` ディレクトリに移動し、`SSH` 接続に必要な権限 `400` を設定する。

```
$ mv HirakeGoma.pem ~/.ssh/
$ chmod 400 ~/.ssh/HirakeGoma.pem
```

#### 4.4.4. CDK deploy

CDK deploy 実行には、EC2 インスタンスの作成に必要な AWS クレジットカードの登録と、AWS CLI のインストールが必要です。-c key\_name="HirakeGoma" オプションは、キーペアの名前を指定します。

```
$ cdk deploy -c key_name="HirakeGoma"
```

CDK deploy 実行後、VPC に EC2 インスタンスが作成されます。Figure 11 は、CDK deploy 実行後の出力結果を示しています。InstancePublicIp は、EC2 インスタンスの公開 IP アドレスです。IP アドレスは、EC2 インスタンスの IP アドレスです。

```
✓ MyFirstEc2
Outputs:
MyFirstEc2.InstancePublicIp = 54.238.112.5
MyFirstEc2.InstancePublicDnsName = ec2-54-238-112-5.ap-northeast-1.compute.amazonaws.com
Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/MyFirstEc2/46ed0490-aa2d-
```

Figure 11. CDK deploy 実行結果

#### 4.4.5. SSH 接続

SSH 接続を行うには、EC2 インスタンスの IP アドレスと、キーペアの名前が必要です。

```
$ ssh -i ~/.ssh/HirakeGoma.pem ec2-user@<IP address>
```

-i オプションは、キーペアのファイルパスを指定します。ec2-user は、EC2 インスタンスのユーザー名です。<IP address> は、EC2 インスタンスの IP アドレスです。(12.345.678.9 など)

Figure 12 は、SSH 接続成功後の画面を示しています。[ec2-user@ip-10-10-1-217 ~]\$ は、SSH 接続成功後のプロンプトです。

```
(.env) tomoyuki@eiffel:01-ec2$ ssh -i ~/.ssh/HirakeGoma.pem ec2-user@54.238.112.5
Last login: Tue Jun  9 09:18:09 2020 from 157.82.122.171

 _ _ | _ _ | _ _ )
 _ | ( _ _ /   Amazon Linux AMI
 _ _ | _ _ | _ _ |

https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/
5 package(s) needed for security, out of 7 available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-10-10-1-217 ~]$
```

Figure 12. SSH 接続成功後の画面

SSH 接続成功後、AWS CLI を使用して EC2 インスタンスの情報を取得することができます。

#### 4.4.6. EC2 インスタンスの起動

EC2 インスタンスの起動には、AWS CLI を使用します。

EC2 インスタンスの起動には、CPU のタイプを指定する必要があります。

```
$ cat /proc/cpuinfo
```

processor	: 0
vendor_id	: GenuineIntel
cpu family	: 6
model	: 63
model name	: Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz
stepping	: 2
microcode	: 0x43
cpu MHz	: 2400.096
cache size	: 30720 KB

[illegible]

```
$ top -n 1

top - 09:29:19 up 43 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  76 total,    1 running,  51 sleeping,    0 stopped,    0 zombie
Cpu(s):  0.3%us,  0.3%sy,  0.1%ni, 98.9%id,  0.2%wa,  0.0%hi,  0.0%si,  0.2%st
Mem:   1009140k total,  270760k used,  738380k free,   14340k buffers
Swap:        0k total,    0k used,    0k free,  185856k cached


  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0 19696 2596 2268 S   0.0   0.3   0:01.21 init
    2 root        20   0     0     0     0 S   0.0   0.0   0:00.00 kthreadd
    3 root        20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/0:0
```

t2.micro 1009140k = 1GB

Python 2 Python 3 Python 3.6

```
$ sudo yum update -y
$ sudo yum install -y python36
```

```
$ sudo yum update -y
$ sudo yum install -y python36
```

## Python 環境構築

```
$ python3
Python 3.6.10 (default, Feb 10 2020, 19:55:14)
[GCC 4.8.5 20150623 (Red Hat 4.8.5-28)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Python において、`Ctrl + D` は `exit()` を呼び出すために使用されます。

これは、標準出力 (STDOUT) に EOF (End of File) を送信するのと同じです。

\$ exit

#### 4.4.7. AWS 環境構築

環境構築には、EC2 インスタンスを起動し、EC2インスタンスにネットワークを接続し、AWS VPCにSGを接続する。

環境構築には、AWS Management ConsoleのServicesからEC2を選択し、Instancesを選択する。図13に示すように、EC2インスタンスのリストが表示される。VPCにSGを接続する。

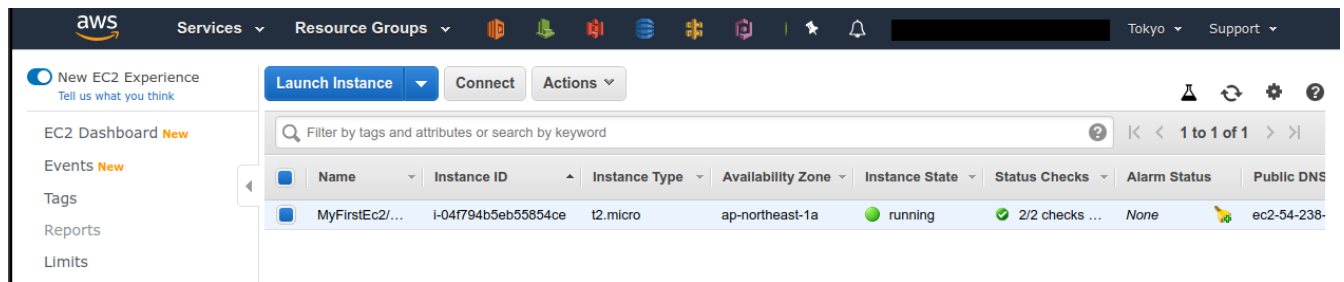


Figure 13. EC2 インスタンス



環境構築には、EC2インスタンスを起動し、EC2インスタンスにネットワークを接続し、AWS VPCにSGを接続する。

CloudFormation テンプレートを使用して、CloudFormation スタック (stack) を作成し、AWS VPC/EC2/SG を作成する。図14に示すように、CloudFormation スタックのリストが表示される。

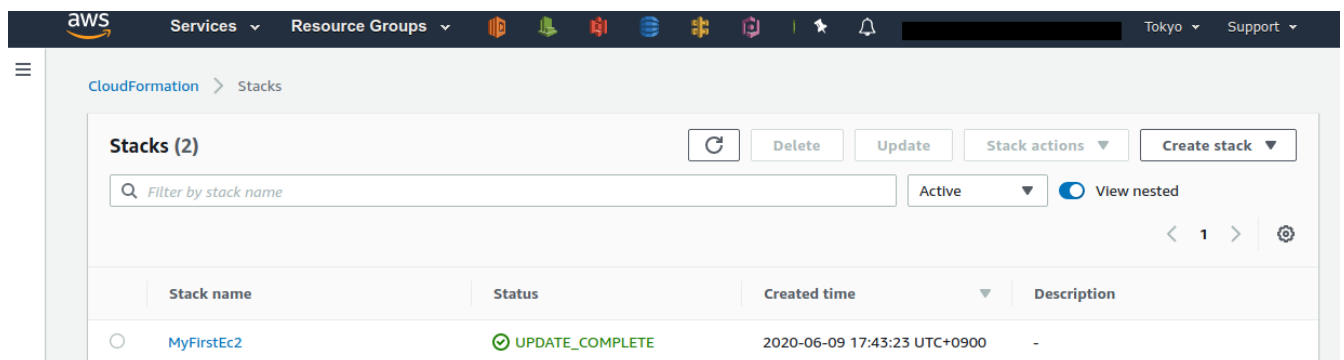


Figure 14. CloudFormation スタック

"MyFirstEc2" スタックは、EC2、VPC、SGを含む環境構築のためのテンプレートです。

#### 4.4.8. テンプレート

環境構築には、CloudFormation テンプレートを使用します。

CloudFormation テンプレート "Delete" を実行すると、図15に示すように、"DELETE\_IN\_PROGRESS" ステータスでCloudFormation テンプレートが削除されます。

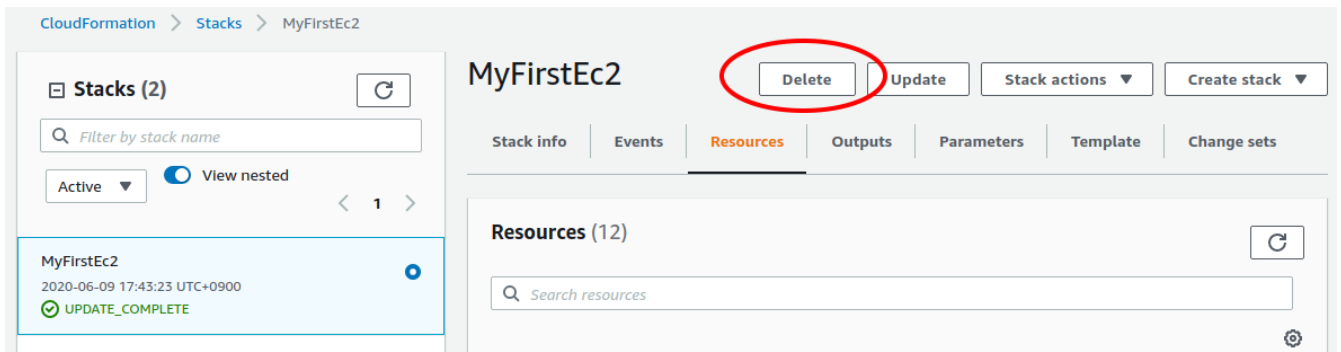


Figure 15. CloudFormation

```
$ cdk destroy
```

CloudFormation VPC, EC2 CloudFormation AWS



EC2

SSH EC2

EC2 Key Pairs HirakeGoma Actions Delete (Figure 16)

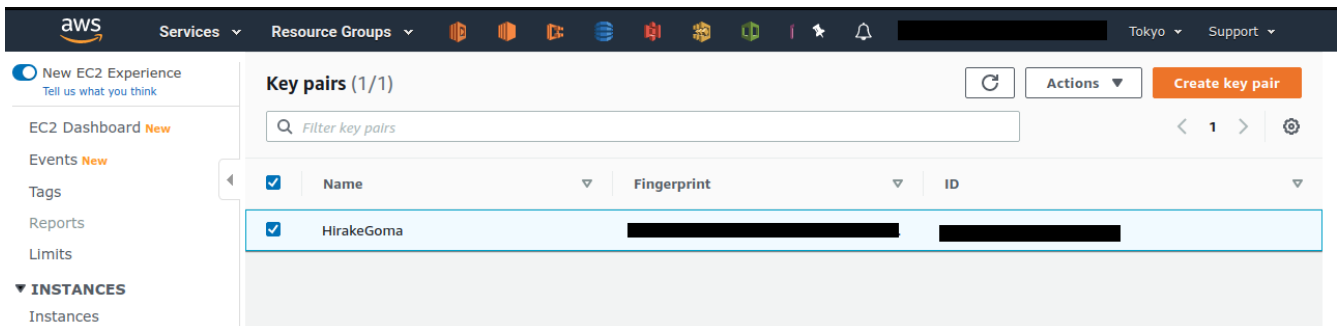
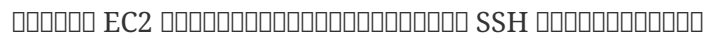


Figure 16. EC2 SSH

```
$ aws ec2 delete-key-pair --key-name "HirakeGoma"
```

```
$ rm -f ~/.ssh/HirakeGoma.pem
```



[illegible]

---

Chapter 2

EC2

[illegible]

# Chapter 5. 環境構築と学習環境の準備

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。

本章では、Python、PyTorch、GPU のインストールと設定を行います。PyTorch は、AI 学習のためのフレームワークであり、GPU を利用して学習を行うことができます。

## 5.1. 環境構築と学習環境の準備

2010年代後半から AI 学習が盛んになり、多くの企業が AI 学習のための環境構築と学習環境の準備を行っています。

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。

GPT-3 は、SOTA (State-of-the-art) の AI 学習モデルであり、多くの企業が GPT-3 を利用して AI 学習を行っています。

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。

## 5.2. GPU のインストールと設定

本章では、GPU (Graphics Processing Unit) のインストールと設定について説明します。

GPU は、AI 学習のための重要なハードウェアであり、CPU (Central Processing Unit) と比較して、AI 学習のための処理能力が大幅に向上しています。



Figure 17. GPU architecture (source: <https://devblogs.nvidia.com/nvidia-turing-architecture-in-depth/>)

GPU is designed for **General-purpose computing on GPU; GPGPU**. CPU is designed for **General-purpose computing on CPU; GPGPU**. GPU is designed for **General-purpose computing on GPU; GPGPU**.

GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18) GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18) GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18)

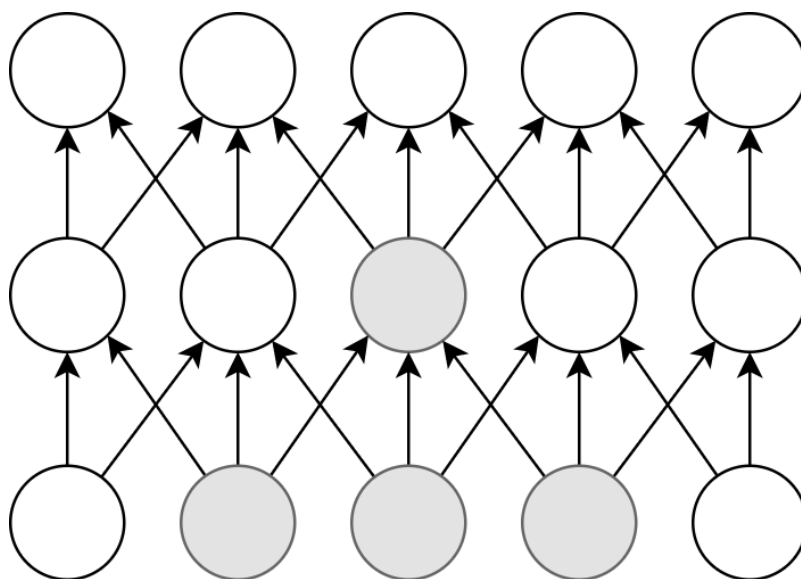


Figure 18. Convolution operation

GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18) GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18) GPU is designed for **General-purpose computing on GPU; GPGPU** (Convolution) (Figure 18)



Amazon EC2 V100 GPU instances (NVIDIA GeForce V100) are available in the us-east-1 region. The V100 GPU has 16 GB of memory and 8 vCPUs. The price per hour is \$3.06. The V100 GPU is available in the p3.2xlarge instance type. The V100 GPU is available in the p3n.16xlarge instance type. The V100 GPU is available in the p2.xlarge instance type. The V100 GPU is available in the g4dn.xlarge instance type.

Amazon EC2 GPU instances (P3, P2, G3, G4) are available in the us-east-1 region. Table 3 shows the GPU instances and their specifications.

Table 3. GPU instances on EC2

Instance	GPUs	GPU model	GPU Mem (GiB)	vCPU	Mem (GiB)	Price per hour (\$)
p3.2xlarge	1	NVIDIA V100	16	8	61	3.06
p3n.16xlarge	8	NVIDIA V100	128	64	488	24.48
p2.xlarge	1	NVIDIA K80	12	4	61	0.9
g4dn.xlarge	1	NVIDIA T4	16	4	16	0.526

Table 3 shows the GPU instances and their specifications. The CPU instances are p3.2xlarge, p3n.16xlarge, p2.xlarge, and g4dn.xlarge. The GPU instances are V100, K80, and T4. The price per hour is shown in the last column.

The g4dn.xlarge instance type is the most cost-effective for training GPT-3. It has 16 GB of GPU memory and 4 vCPUs. The price per hour is \$0.526.



Table 3 shows the GPU instances and their specifications. The CPU instances are p3.2xlarge, p3n.16xlarge, p2.xlarge, and g4dn.xlarge. The GPU instances are V100, K80, and T4. The price per hour is shown in the last column.



The V100 GPU is available in the p3.2xlarge instance type. The price per hour is \$3.06. The V100 GPU has 16 GB of memory and 8 vCPUs. The V100 GPU is available in the p3n.16xlarge instance type. The V100 GPU is available in the p2.xlarge instance type. The V100 GPU is available in the g4dn.xlarge instance type.

GPT-3 is available on the Lambda platform. The Lambda platform is a serverless computing platform. The Lambda platform is available in the us-east-1 region. The Lambda platform is available in the us-east-1 region.



The Lambda platform is a serverless computing platform. The Lambda platform is available in the us-east-1 region. The Lambda platform is available in the us-east-1 region. The Lambda platform is available in the us-east-1 region.

## Further reading

For more information on the Lambda platform, see the AWS Lambda documentation.

- [Deep Learning \(Ian Goodfellow, Yoshua Bengio and Aaron Courville\)](#) is a book that covers the fundamentals of deep learning. It is available on Amazon.

- [Intro to Deep Learning \(CS 231n\)](#) 斯坦福大学深度学习课程 包含理论和实践 每周都有作业和考试
- [Dive into Deep Learning \(Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola\)](#)  
斯坦福大学深度学习课程 包含理论和实践 每周都有作业和考试 1000 多个幻灯片  
包含理论和实践 每周都有作业和考试

# Chapter 6. Hands-on #2: AWS 環境構築

## 6.1. 準備

このHands-onでは GPU 搭載の EC2 インスタンスを使用します。

このHands-onのコードは GitHub の [handson/mnist](https://github.com/handson/mnist) リポジトリにあります。

このHands-onは AWS の [Section 4.1](#) の環境構築の手順に従って行います。



このHands-onでは AWS の GPU 搭載の G インスタンスを使用します。AWS の EC2 インスタンスの **Limits** ページを確認し、**Running On-Demand All G instances** の制限を確認してください。



このHands-onでは 0 台の EC2 インスタンスを使用します。AWS の [Amazon EC2 service quotas](#) ページを確認してください。



このHands-onでは **g4dn.xlarge** の EC2 インスタンスを使用します (**ap-northeast-1** リージョン) の料金は 0.71 \$/hour です。

AWS Educate Starter Account の制限: Starter Account の GPU 搭載の EC2 インスタンスの使用は制限されています。Starter Account の制限を確認してください。

## 6.2. ネットワーク環境構築

このHands-onのネットワーク環境は [Figure 19](#) のように構築します。

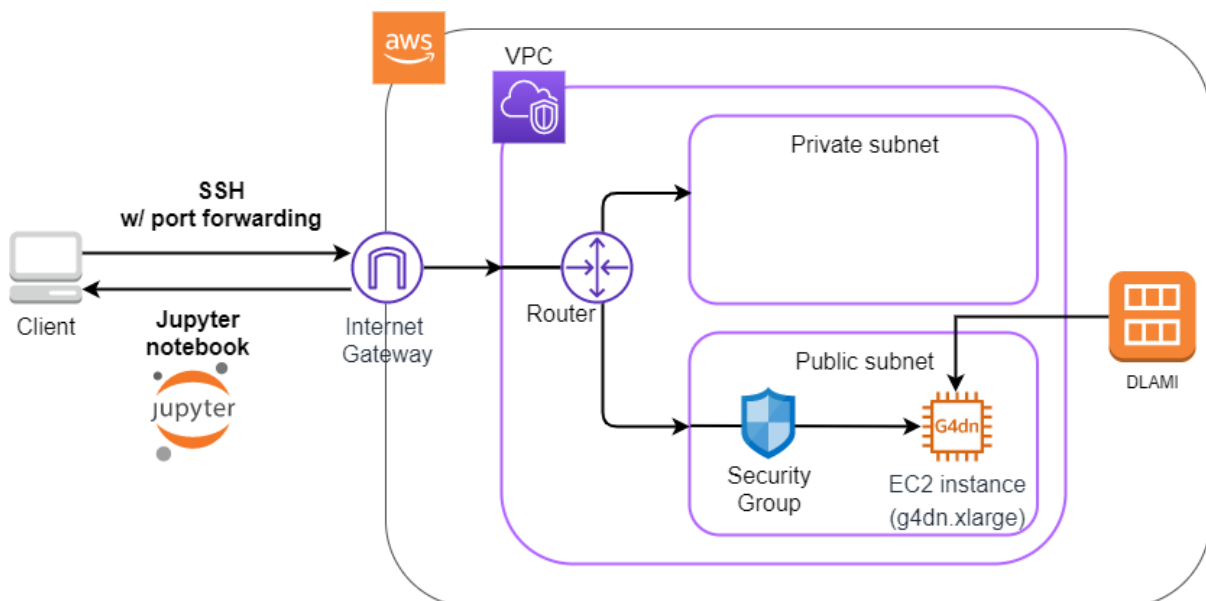


Figure 19. Hands-on #2 のネットワーク環境構築

このHands-onでは、AWS の **g4dn.xlarge** の EC2 インスタンスを使用します。

- GPU `g4dn.xlarge` を利用する
- 必要に応じて `DLAMI` (AMI) を指定
- SSH 接続可能にする (Jupyter Notebook (AMI) を利用する)

必要に応じて `handson/mnist/app.py` を利用する

```

1 class Ec2ForDL(Stack):
2
3     def __init__(self, scope: Construct, construct_id: str, key_name: str, **
4         kwargs) -> None:
5         super().__init__(scope, construct_id, **kwargs)
6
7         vpc = ec2.Vpc(
8             self, "Ec2ForDL-Vpc",
9             max_azs=1,
10            ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),
11            subnet_configuration=[
12                ec2.SubnetConfiguration(
13                    name="public",
14                    subnet_type=ec2.SubnetType.PUBLIC,
15                )
16            ],
17            nat_gateways=0,
18        )
19
20        sg = ec2.SecurityGroup(
21            self, "Ec2ForDL-Sg",
22            vpc=vpc,
23            allow_all_outbound=True,
24        )
25        sg.add_ingress_rule(
26            peer=ec2.Peer.any_ipv4(),
27            connection=ec2.Port.tcp(22),
28        )
29
30        host = ec2.Instance(
31            self, "Ec2ForDL-Instance",
32            instance_type=ec2.InstanceType("g4dn.xlarge"), ①
33            machine_image=ec2.MachineImage.generic_linux({
34                "us-east-1": "ami-060f07284bb6f9faf",
35                "ap-northeast-1": "ami-09c0c16fc46a29ed9"
36            }), ②
37            vpc=vpc,
38            vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
39            security_group=sg,
40            key_name=key_name
41        )

```

① `g4dn.xlarge` は GPU を搭載したインスタンス (標準的な CPU インスタンス `t2.micro` など) より `g4dn.xlarge` は高価である





```

tomoyuki@balthasar:02-ec2-dnn$ aws ec2 describe-images --owners amazon --image-ids "ami-09c0c16fc46a29ed9"
{
  "Images": [
    {
      "Architecture": "x86_64",
      "CreationDate": "2020-05-20T14:47:04.000Z",
      "ImageId": "ami-09c0c16fc46a29ed9",
      "ImageLocation": "amazon/Deep Learning AMI (Amazon Linux 2) Version 29.0",
      "ImageType": "machine",
      "Public": true,
      "OwnerId": "898082745236",
      "PlatformDetails": "Linux/UNIX",
      "UsageOperation": "RunInstances",
      "State": "available",
      "BlockDeviceMappings": [
        {
          "DeviceName": "/dev/xvda",
          "Ebs": {
            "DeleteOnTermination": true,
            "SnapshotId": "snap-0bd381ab76e5a6146",
            "VolumeSize": 90,
            "VolumeType": "gp2",
            "Encrypted": false
          }
        }
      ],
      "Description": "MXNet-1.6.0, Tensorflow-2.1.0 & 1.15.2, PyTorch-1.4.0 & 1.5.0, Neuron, & other frameworks, NVIDIA CUDA, cuDNN, NCCL, Intel MKL-DNN, Docker, NVIDIA-Docker & EFA support. For fully managed experience, check: https://aws.amazon.com/sagemaker",
      "EnaSupport": true,
      "Hypervisor": "xen",
      "ImageOwnerAlias": "amazon",
      "Name": "Deep Learning AMI (Amazon Linux 2) Version 29.0",
      "RootDeviceName": "/dev/xvda",
      "RootDeviceType": "ebs",
      "SriovNetSupport": "simple",
      "VirtualizationType": "hvm"
    }
  ]
}

```

Figure 20. AMI ID = ami-09c0c16fc46a29ed9

Figure 20 DLAMI PyTorch 1.4.0 1.5.0 DLAMI



DLAMI (何: ["What Is the AWS Deep Learning AMI?"](#))

low-level GPU OS GPU CUDA cuDNN NVIDIA GPU C++ cuDNN CUDA n "Base" DLAMI

"Conda" "Base" TensorFlow PyTorch Anaconda TensorFlow PyTorch MxNet Jupyter Notebook

## 6.3.

1. 環境構築 (Section 15.3)

```
# 環境構築
$ cd handson/mnist

# venv 環境構築
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# SSH
$ export KEY_NAME="HirakeGoma"
$ aws ec2 create-key-pair --key-name ${KEY_NAME} --query 'KeyMaterial' --output text >
${KEY_NAME}.pem
$ mv HirakeGoma.pem ~/.ssh/
$ chmod 400 ~/.ssh/HirakeGoma.pem

# CDK
$ cdk deploy -c key_name="HirakeGoma"
```



1. SSH 環境構築 (Section 15.3)

Figure 21 AWS インスタンスの IP アドレス (InstancePublicIp) を取得する

```
✓ Ec2ForDL
Outputs:
Ec2ForDL.InstancePublicIp = 52.192.211.12
Ec2ForDL.InstancePublicDnsName = ec2-52-192-211-12.ap-northeast-1.compute.amazonaws.com
Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/Ec2ForDL/dd8361d0-...
```

Figure 21. CDK の出力結果

## 6.4. 接続

SSH 環境構築後、Jupyter Notebook を起動するためのポート転送 (port forwarding) を設定する (-L) を実行する

```
$ ssh -i ~/.ssh/HirakeGoma.pem -L localhost:8931:localhost:8888 ec2-user@<IP address>
```

SSH 環境構築後、Jupyter Notebook を起動するためのポート転送 (port forwarding) を設定する (-L localhost:8931:localhost:8888) を実行する。localhost:8931 は Jupyter Notebook が起動するポート (TCP/IP) であり、localhost:8888 は Jupyter Notebook が起動するポート (localhost:8931) である。Jupyter Notebook を起動する (Figure 22) の SSH 環境構築後、

```
ssh -L localhost:8931:localhost:8888 ec2-user@0.0.0.0
```

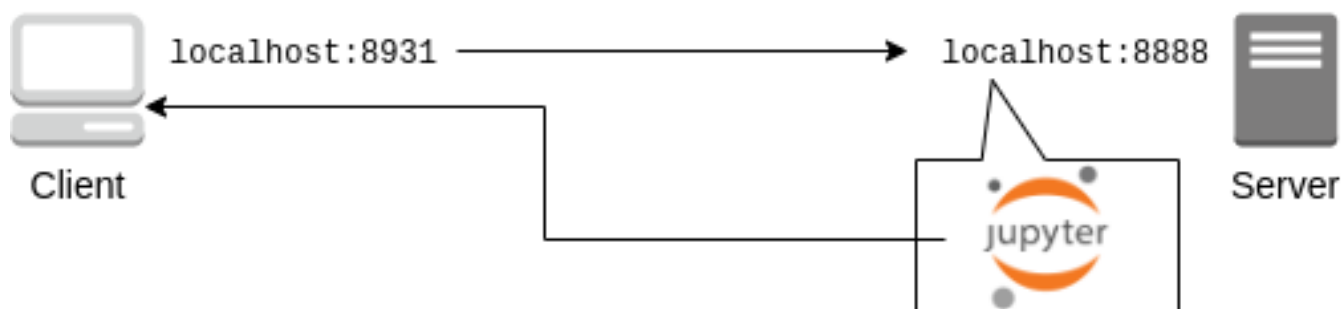


Figure 22. SSH 接続による Jupyter Notebook 接続



接続する際のポート番号 (:8931, :8888) は 1065535 の範囲内である。また、22 (SSH) と 80 (HTTP) のポート番号は、Jupyter Notebook のポート番号 8888 と異なる。また、8888 のポート番号は



SSH 接続する <IP address> は、接続する IP である。



また、Docker を使用する。

SSH 接続する Docker は、(接続する IP) の Jupyter Notebook の Docker である。

また、Docker を使用する。また、cat ~/.ssh/HirakeGoma の内容を確認する。また、-v を使用する。また、Docker を使用する。また、"Use volumes" を使用する。

SSH 接続する GPU を使用する。

```
$ nvidia-smi
```

Figure 23 接続する GPU を使用する。また、Tesla T4 の GPU を使用する。また、GPU Driver と CUDA を使用する。また、GPU を使用する。

```
[ec2-user@ip-10-10-1-172 ~]$ nvidia-smi
Sat Jun 13 04:55:22 2020

+-----+
| NVIDIA-SMI 440.33.01      Driver Version: 440.33.01      CUDA Version: 10.2      |
+-----+-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
+-----+-----+-----+
|  0 Tesla T4               On         | 00000000:00:1E.0 Off  |          0          |
| N/A   31C    P8      11W /  70W |  0MiB / 15109MiB |    0%      Default   |
+-----+-----+-----+

+-----+
| Processes:                                     GPU Memory |
|  GPU       PID    Type    Process name                        Usage      |
+-----+-----+-----+
| No running processes found                  |
+-----+
```

Figure 23. nvidia-smi

## 6.5. Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and explanatory text. It is a GUI for running Python code in a web browser. (Figure 24) Python code is executed in the background and the results are displayed in the notebook.

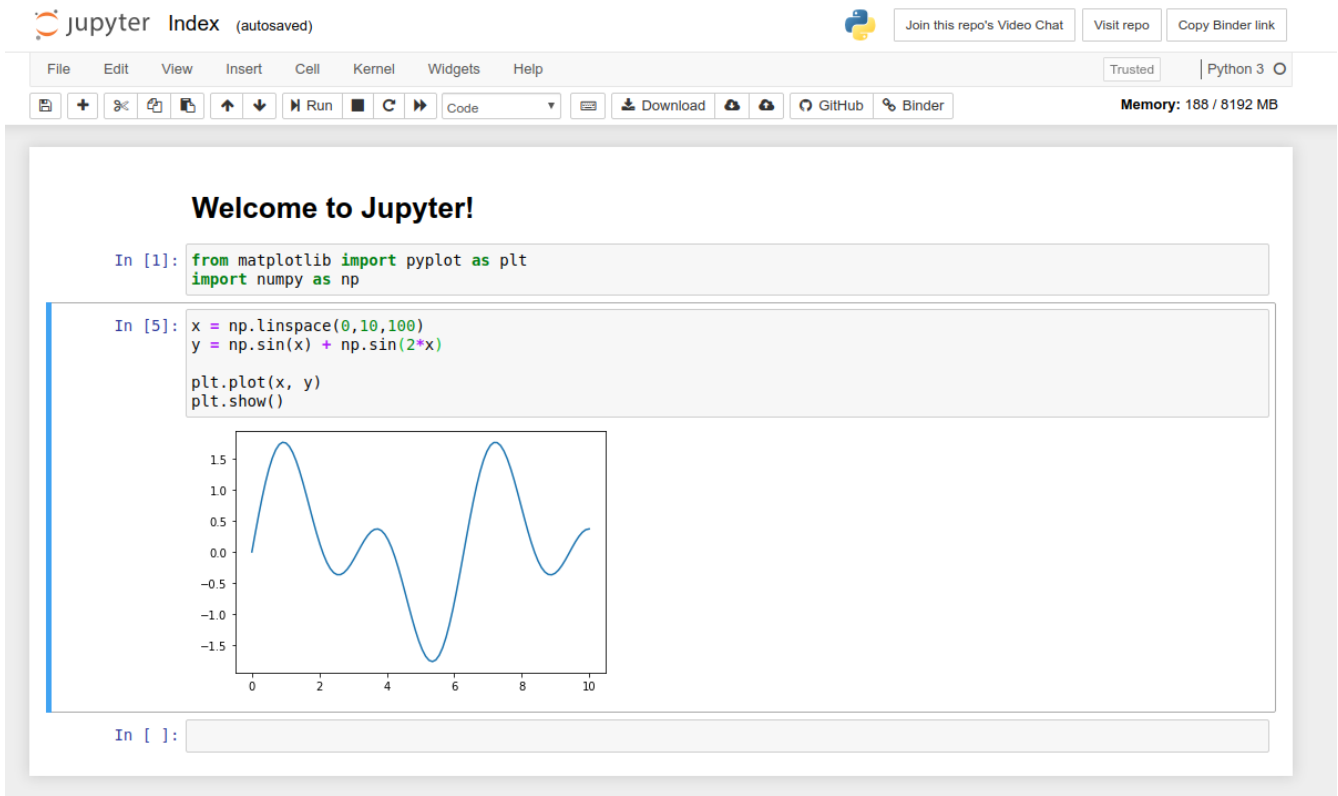


Figure 24. Jupyter Notebook

Jupyter Notebook is a web application that allows you to create and share documents that contain live code, equations, visualizations and explanatory text. It is a GUI for running Python code in a web browser. DLAMI is a Jupyter Notebook environment that is pre-installed on the EC2 instance.

Jupyter Notebook can be accessed via SSH on the EC2 instance.

```
$ cd ~ # go to home directory
$ jupyter notebook
```

Figure 25 Jupyter EC2 localhost:8888  
localhost:8888 ?token=XXXX

```
[I 06:01:10.466 NotebookApp] The Jupyter Notebook is running at:
[I 06:01:10.466 NotebookApp] http://localhost:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
[I 06:01:10.466 NotebookApp] or http://127.0.0.1:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
[I 06:01:10.466 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[W 06:01:10.469 NotebookApp] No web browser found: could not locate runnable browser.
[C 06:01:10.469 NotebookApp]

To access the notebook, open this file in a browser:
    file:///home/ec2-user/.local/share/jupyter/runtime/nbserver-11720-open.html
Or copy and paste one of these URLs:
    http://localhost:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
    or http://127.0.0.1:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
```

Figure 25. Jupyter Notebook



Jupyter Notebook  
AWS GPU

SSH Jupyter localhost:8888  
localhost:8931 Jupyter (Chrome, FireFox)

http://localhost:8931/?token=XXXX

?token=XXXX Jupyter

Jupyter (Figure 26) Jupyter

Quit Logout

Files Running IPython Clusters Conda

Select items to perform actions on them. Upload New

	Name	Last Modified	File size
0 /			
<input type="checkbox"/>	anaconda3	25 days ago	
<input type="checkbox"/>	data	16 hours ago	
<input type="checkbox"/>	examples	20 hours ago	
<input type="checkbox"/>	examples_	25 days ago	
<input type="checkbox"/>	src	25 days ago	
<input type="checkbox"/>	tools	a month ago	
<input type="checkbox"/>	tutorials	25 days ago	

Figure 26. Jupyter



Jupyter Notebook ( )

- **Shift + Enter**: 실행
- **Esc: Command mode** 모드
- 셀에 "+" 버튼을 클릭하면 Command mode → A ⇒ 삽입
- 셀에 "X" 버튼을 클릭하면 Command mode → X ⇒ 실행

이제 Ventsislav Yordanov 님의 블로그를 살펴보겠습니다.

## 6.6. PyTorch를 설치합니다

**PyTorch**는 Facebook AI Research LAB (FAIR)에서 개발한 딥러닝 프레임워크입니다. PyTorch는 Tesla에서 개발한 딥러닝 프레임워크인 Caffe2와 유사합니다. PyTorch는 Caffe2와 유사한 API를 제공합니다.



PyTorch를 설치합니다.

Facebook은 PyTorch를 Caffe2보다 더 강력하고 사용하기 쉬운 프레임워크로 (Caffe는 UC Berkeley에서 개발한 Yangqing Jia 님의 프레임워크) Caffe2를 2018년 PyTorch로 대체했습니다.

2019년 12월 현재 Preferred Networks에서 개발한 Chainer는 PyTorch를 대체할 것으로 예상됩니다. (Chainer는 Preferred Networks에서 개발한 딥러닝 프레임워크) PyTorch는 Chainer보다 더 강력하고 사용하기 쉬운 API를 제공합니다. Chainer는 DNA에서 개발한 PyTorch를 대체할 것으로 예상됩니다...!

PyTorch를 설치하려면 GPU를 사용해야 합니다. GPU를 사용하지 않으면 PyTorch를 설치할 수 없습니다.

Jupyter Notebook을 사용하여 "conda\_pytorch\_p36" 환경을 생성합니다. (Figure 27) "conda\_pytorch\_p36" 환경을 생성하면 PyTorch를 사용할 수 있습니다.

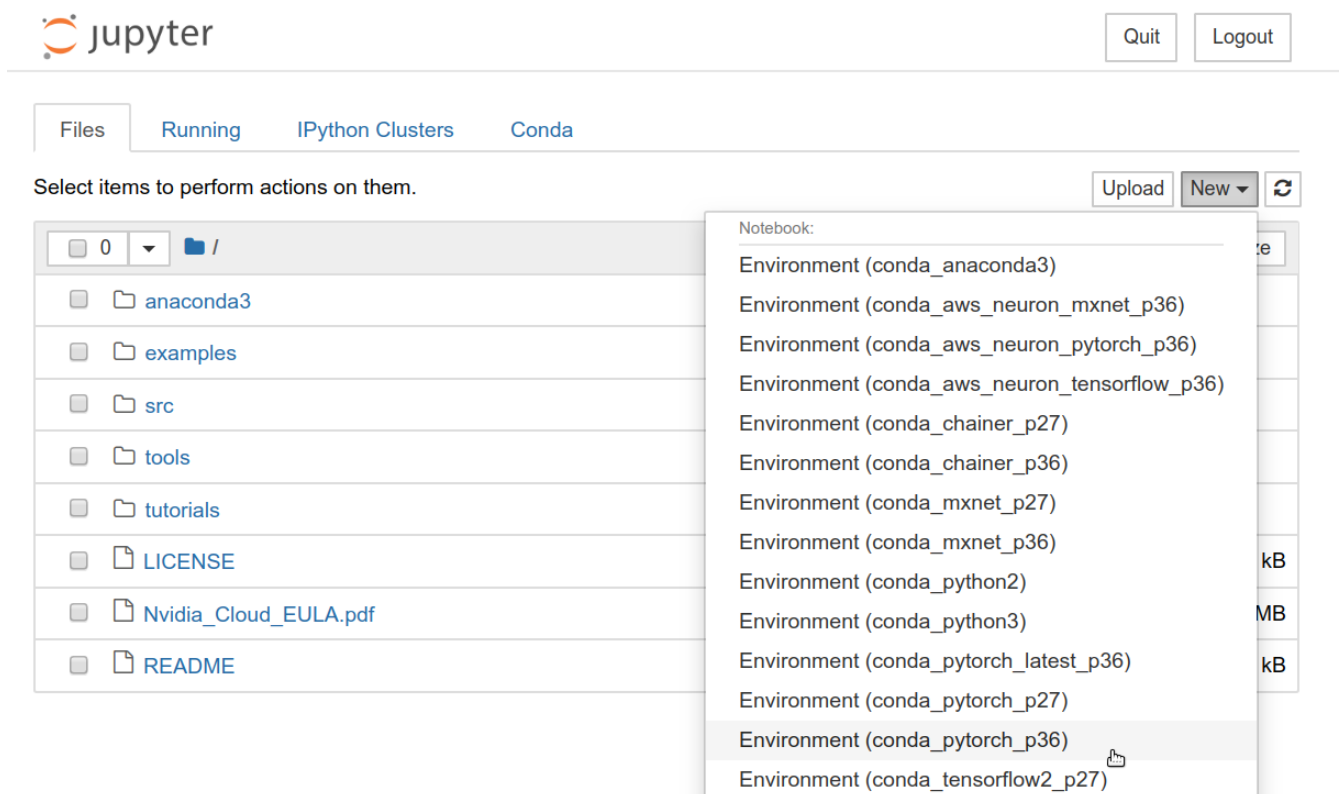


Figure 27. "conda\_pytorch\_p36" 환경을 생성합니다

Figure 28

```
In [1]: import torch
        print("Is CUDA ready?", torch.cuda.is_available())
        Is CUDA ready? True

In [3]: # create a random array in CPU
        x = torch.rand(3,3)
        print(x)
        tensor([[0.6896, 0.2428, 0.3269],
                  [0.0533, 0.3594, 0.9499],
                  [0.9764, 0.5881, 0.0203]])

In [4]: # create another array in GPU device
        y = torch.ones_like(x, device="cuda")
        # move 'x' from CPU to GPU
        x = x.to("cuda")

In [5]: # run addition operation in GPU
        z = x + y
        print(z)
        tensor([[1.6896, 1.2428, 1.3269],
                  [1.0533, 1.3594, 1.9499],
                  [1.9764, 1.5881, 1.0203]], device='cuda:0')

In [6]: # move z from GPU to CPU
        z = z.to("cpu")
        print(z)
        tensor([[1.6896, 1.2428, 1.3269],
                  [1.0533, 1.3594, 1.9499],
                  [1.9764, 1.5881, 1.0203]])
```

Figure 28. PyTorch

PyTorch GPU

```
1 import torch
2 print("Is CUDA ready?", torch.cuda.is_available())
```

:

Is CUDA ready? True

3x3 CPU

```
1 x = torch.rand(3,3)
2 print(x)
```

:

```
tensor([[0.6896, 0.2428, 0.3269],
        [0.0533, 0.3594, 0.9499],
        [0.9764, 0.5881, 0.0203]])
```

GPU

```
1 y = torch.ones_like(x, device="cuda")
2 x = x.to("cuda")
```

변수 `x` 와 `y` 모두 GPU로 이동

```
1 z = x + y
2 print(z)
```

결과:

```
tensor([[1.6896, 1.2428, 1.3269],
        [1.0533, 1.3594, 1.9499],
        [1.9764, 1.5881, 1.0203]], device='cuda:0')
```

변수 GPU로 이동하면 CPU로 이동

```
1 z = z.to("cpu")
2 print(z)
```

결과:

```
tensor([[1.6896, 1.2428, 1.3269],
        [1.0533, 1.3594, 1.9499],
        [1.9764, 1.5881, 1.0203]])
```

변수 GPU로 이동하면 CPU로 이동 GPU로 이동하면 CPU로 이동 3x3  
변수 GPU로 이동하면 CPU로 이동 GPU로 이동하면 CPU로 이동



Jupyter Notebook 에서 [/handson/mnist/pytorch/pytorch\\_get\\_started.ipynb](/handson/mnist/pytorch/pytorch_get_started.ipynb) 에서  
Jupyter 인터프리터 "Upload" 버튼을 클릭하여 업로드합니다

업로드된 파일을 실행합니다

변수 GPU로 이동하면 CPU로 이동 GPU로 이동하면 CPU로 이동 Jupyter 인터프리터에서 `%time` 명령어를 사용합니다

변수 CPU로 이동하면 10000x10000 행렬 곱셈을 수행합니다

```
1 s = 10000
2 device = "cpu"
3 x = torch.rand(s, s, device=device, dtype=torch.float32)
4 y = torch.rand(s, s, device=device, dtype=torch.float32)
5
6 %time z = torch.matmul(x, y)
```



このコードを実行すると、CPUの処理時間が約11.6秒、GPUの処理時間が約5.8秒で実行されます。

```
CPU times: user 11.5 s, sys: 140 ms, total: 11.6 s
Wall time: 5.8 s
```

GPUの処理時間を短縮するために、CUDAの同期を明示的に呼び出す必要があります。

```
1 s = 10000
2 device = "cuda"
3 x = torch.rand(s, s, device=device, dtype=torch.float32)
4 y = torch.rand(s, s, device=device, dtype=torch.float32)
5 torch.cuda.synchronize()
6
7 %time z = torch.matmul(x,y); torch.cuda.synchronize()
```

このコードを実行すると、CPUの処理時間が約554ms、GPUの処理時間が約553msで実行されます。

```
CPU times: user 334 ms, sys: 220 ms, total: 554 ms
Wall time: 553 ms
```



PyTorch の GPU 処理は asynchronous (非同期) であるため、`torch.cuda.synchronize()` を明示的に呼び出す必要があります。



このコードでは、`dtype=torch.float32` を指定しているため、GPUの精度は32bitで実行されます。32bit/16bitの精度を指定することで、精度を16bitに下げることができます。32bit/16bitの精度を指定することで、精度を16bitに下げることができます。

このコードを実行すると、GPUの処理時間が約10秒で実行されます。これは、GPUの精度が32bitであるためです。精度を16bitに下げると、処理時間がさらに短縮されます。

## 6.7. MNISTの学習と推論

この章では、AWSのEC2インスタンスを使用して、MNISTの学習と推論を実行します。

この章の図表は、MNISTの学習と推論の結果を示しています。(Figure 29)



Figure 29. MNIST 데이터셋

이번 MNIST 데이터셋을 사용하여 (Convolutional Neural Network; CNN) 모델을 구현하고 </handson/minist/pytorch/>에서 `mnist.ipynb`와 `simple_mnist.py` 파일을 사용하여 PyTorch의 [Example Project](#)를 실행해 보겠습니다.

이제 `simple_mnist.py` 파일을 업로드합니다 (Figure 30). "Upload" 버튼을 클릭하여 Python 코드를 업로드하고 CNN 모델을 실행합니다.

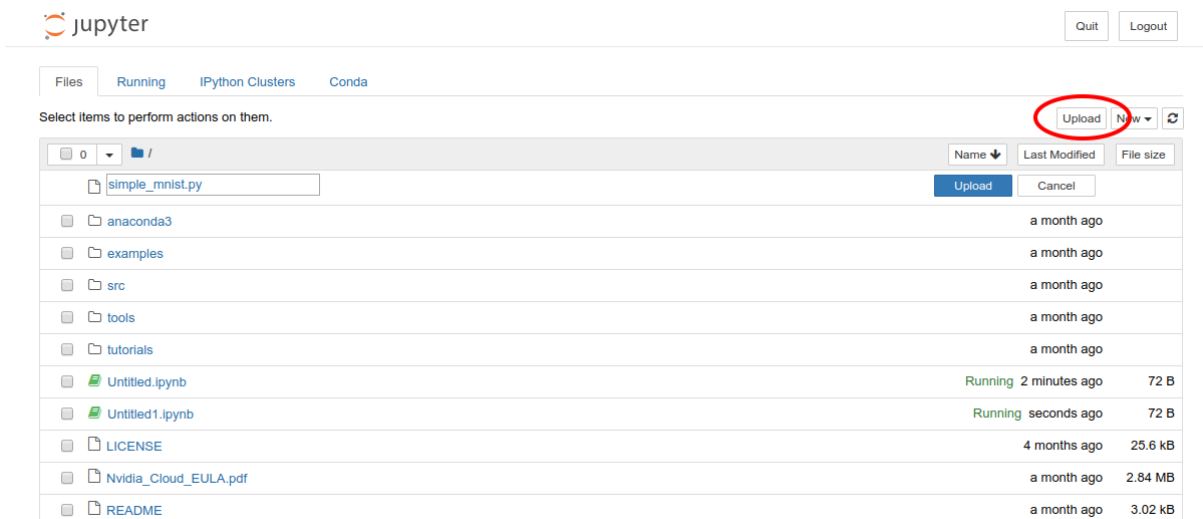


Figure 30. `simple_mnist.py` 업로드

`simple_mnist.py` 파일을 업로드한 후 notebook에서 "conda\_pytorch\_p36" 환경을 선택합니다.

이제 모델을 실행하고 결과를 확인합니다.

```

1 import torch
2 import torch.optim as optim
3 import torchvision
4 from torchvision import datasets, transforms
5 from matplotlib import pyplot as plt
6
7 # custom functions and classes
8 from simple_mnist import Model, train, evaluate

```

`torchvision` `datasets.MNIST` `torch.utils.data.DataLoader` `torchvision.transforms.Compose` `torchvision.transforms.ToTensor` `torchvision.transforms.Normalize` (`Model`, `train`, `evaluate`)

`datasets.MNIST` `torch.utils.data.DataLoader`

```

1 transf = transforms.Compose([transforms.ToTensor(),
2                               transforms.Normalize((0.1307,), (0.3081,))])
3
4 trainset = datasets.MNIST(root='./data', train=True, download=True, transform=
    transf)
5 trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
6
7 testset = datasets.MNIST(root='./data', train=False, download=True, transform=
    transf)
8 testloader = torch.utils.data.DataLoader(testset, batch_size=1000, shuffle=True)

```

`datasets.MNIST` `torch.utils.data.DataLoader` `torchvision.transforms.Compose` `torchvision.transforms.ToTensor` `torchvision.transforms.Normalize` (`Model`, `train`, `evaluate`)

Figure 31

```

1 examples = iter(testloader)
2 example_data, example_targets = examples.next()
3
4 print("Example data size:", example_data.shape)
5
6 fig = plt.figure(figsize=(10,4))
7 for i in range(10):
8     plt.subplot(2,5,i+1)
9     plt.tight_layout()
10    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
11    plt.title("Ground Truth: {}".format(example_targets[i]))
12    plt.xticks([])
13    plt.yticks([])
14 plt.show()

```

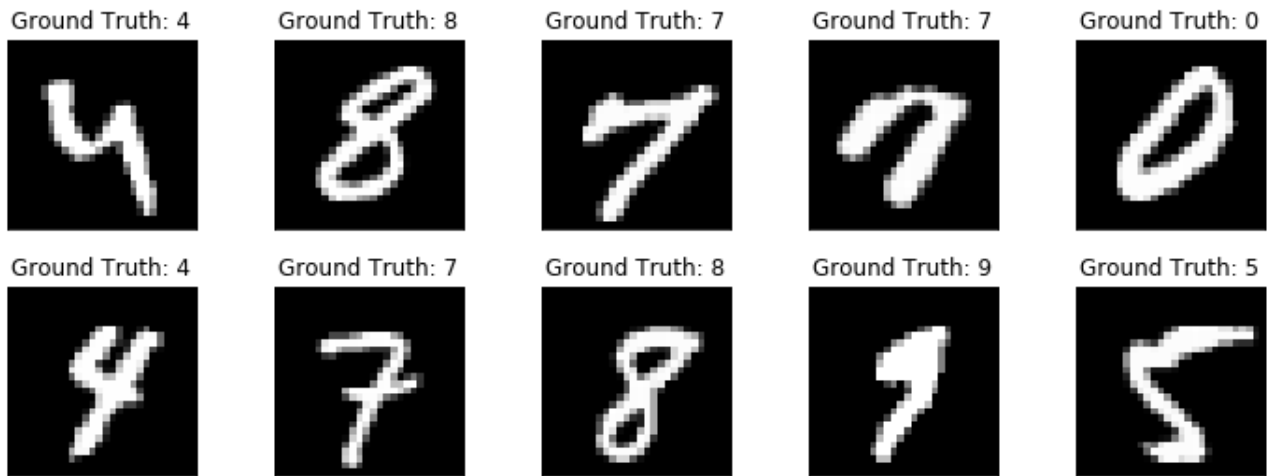


Figure 31. MNIST 데이터셋의 예시 이미지

이제 CNN 모델을 구축해 보겠습니다.

```
1 model = Model()
2 model.to("cuda") # load to GPU
```

이제 `Model` 클래스를 `simple_mnist.py` 파일에 정의합니다. Figure 32에 표시된 대로, 모델은 입력층 (output layer)을 포함하며, Softmax 손실 함수 (Loss function)를 사용하여 Negative log likelihood (NLL) 손실을 계산합니다.

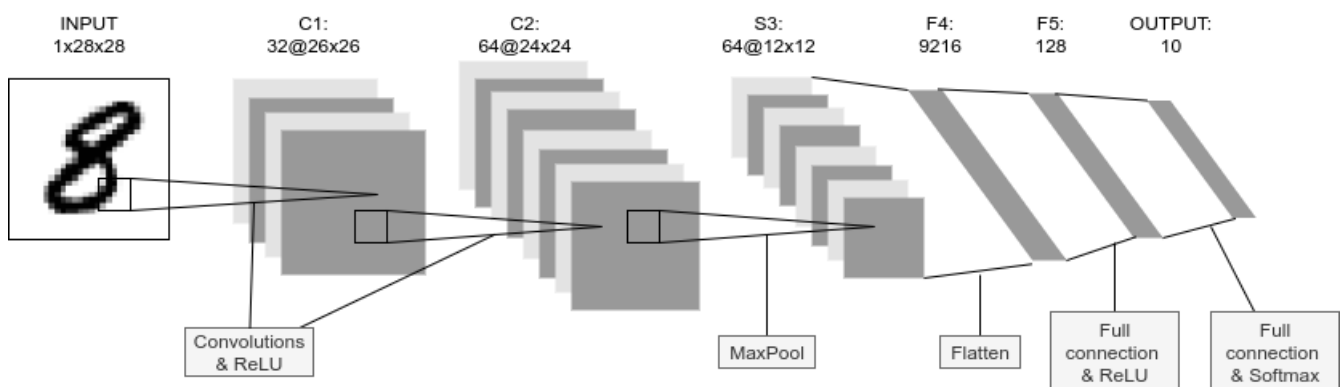


Figure 32. CNN 모델의 구조

이제 CNN 모델을 학습시키기 위해 Stochastic Gradient Descent (SGD) 옵티마이저를 사용합니다.

```
1 optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

이제 CNN 모델을 학습시켜 보겠습니다!

```
1 train_losses = []
2 for epoch in range(5):
3     losses = train(model, trainloader, optimizer, epoch)
4     train_losses = train_losses + losses
5     test_loss, test_accuracy = evaluate(model, testloader)
```

```

6     print(f"\nTest set: Average loss: {test_loss:.4f}, Accuracy: {test_accuracy:.1f}%\n")
7
8     plt.figure(figsize=(7,5))
9     plt.plot(train_losses)
10    plt.xlabel("Iterations")
11    plt.ylabel("Train loss")
12    plt.show()

```

5000 iterations GPU 100%

Figure 33 shows the training loss function (Loss function) over iterations (5000 iterations).

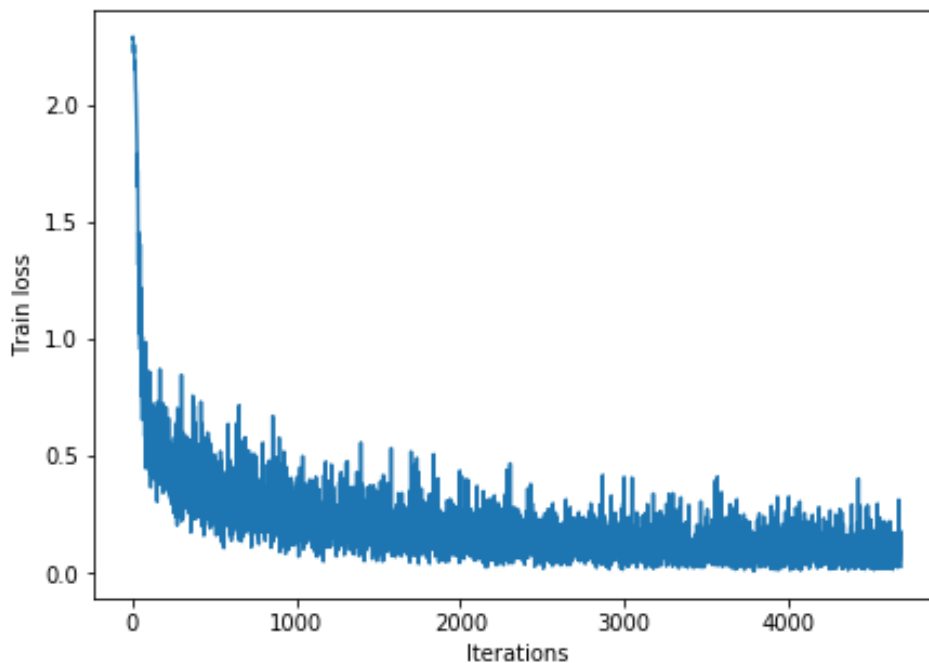


Figure 33. Training loss plot

The final accuracy of the model is 98% (Figure 34).

[mnist\_final\_score] | imgs/handson-jupyter/mnist\_final\_scdk.png

Figure 34. Final accuracy of the model (5000 iterations)

CNN model performance is shown in Figure 35. The model achieves an accuracy of 100% on the test set.

```

1 model.eval()
2
3 with torch.no_grad():
4     output = model(example_data.to("cuda"))
5
6 fig = plt.figure(figsize=(10,4))
7 for i in range(10):
8     plt.subplot(2,5,i+1)

```

```

9 plt.tight_layout()
10 plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
11 plt.title("Prediction: {}".format(output.data.max(1, keepdim=True)[1][i].
    item()))
12 plt.xticks([])
13 plt.yticks([])
14 plt.show()

```



Figure 35. `mnist_cnn` CNN model on MNIST dataset

Save the model to a file named `mnist_cnn.pt` using the following code:

```

1 torch.save(model.state_dict(), "mnist_cnn.pt")

```

Now, we can use the `mnist_cnn` model to predict the digits in the `MNIST` dataset. We can use the `GPU` instance to speed up the training process.

## 6.8. Deployment

We can deploy the `mnist_cnn` model to an `EC2` instance using the following code:

We can use the `AWS` CLI to create a `CloudFormation` stack using the `AWS CLI` (see [Section 4.4.8](#)).

```
$ cdk destroy
```



The `g4dn.xlarge` instance is expensive, costing \$0.71 / hour. We can use the `g4dn.xlarge` instance to speed up the training process.

### AWS Budgets

AWS Budgets (AWS Budgets) allows you to set limits on your AWS spending. You can create a budget to track your spending and receive alerts when you reach a certain threshold. For more information, see [AWS Budgets](#).



# Chapter 7. Docker

SSH  
Chapter 2

Chapter 7, Chapter 8, Chapter 9)  
Docker  
Figure 38) Docker AWS

## 7.1.

Chapter 5 GPT-3 Figure 36

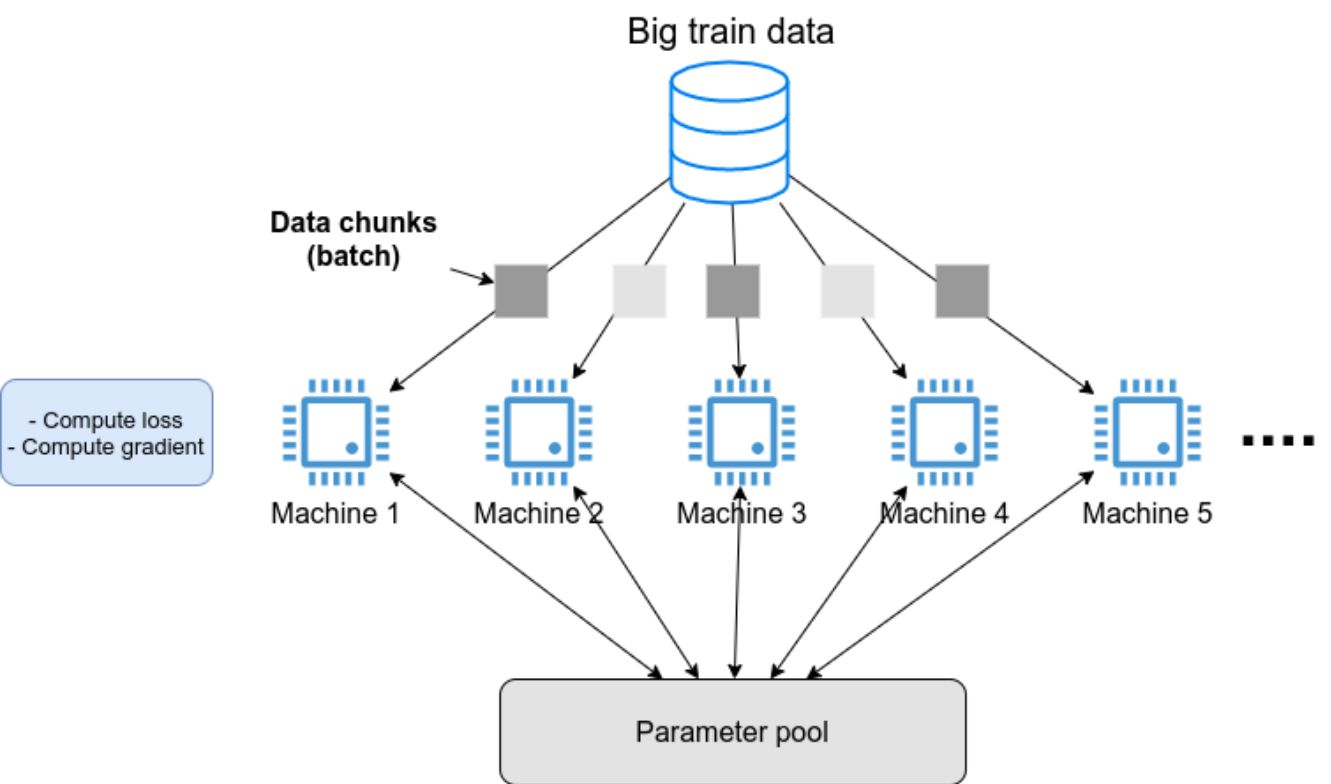


Figure 36.

SNS Figure 37



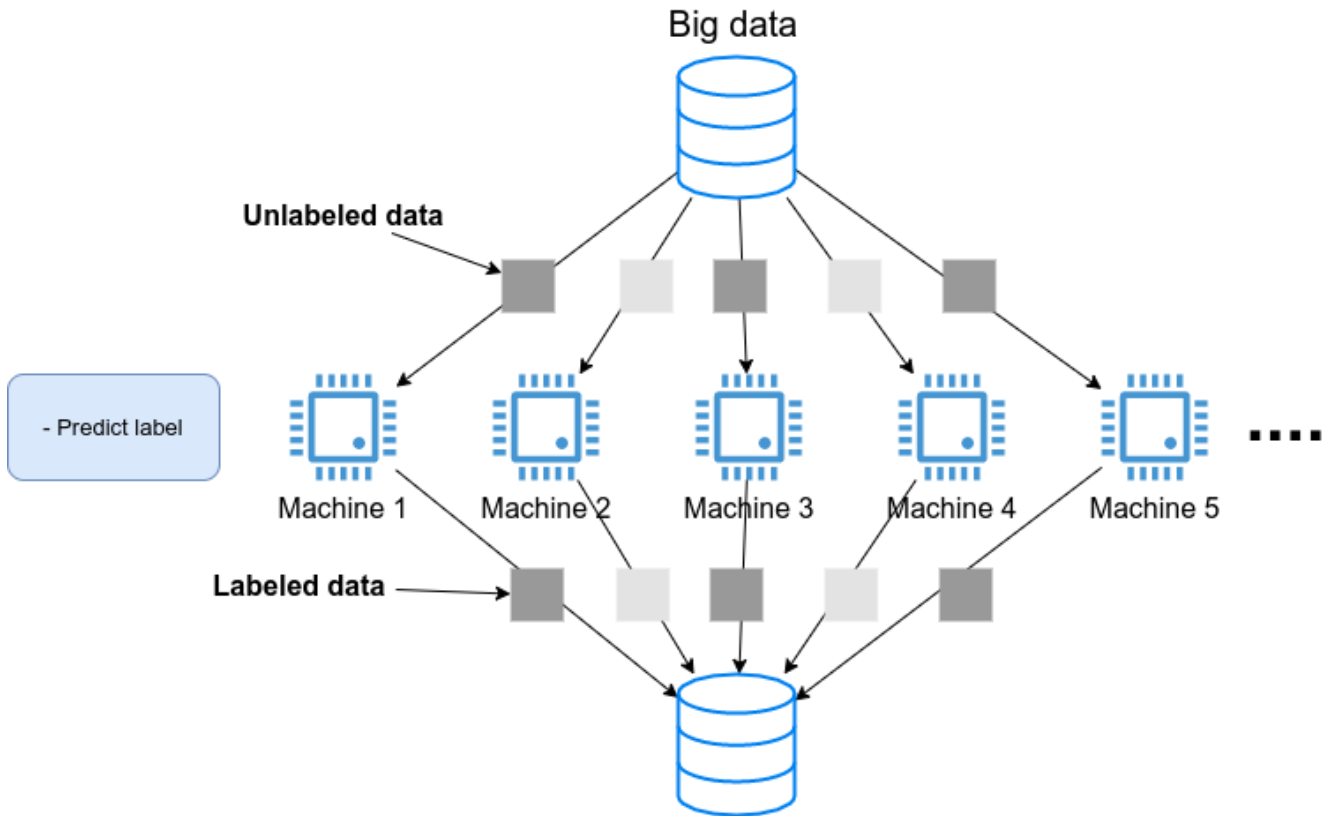


Figure 37. Distributed machine learning architecture

Figure 37 illustrates a distributed machine learning architecture where data is partitioned across multiple machines for processing and prediction.

Figure 36 and Figure 37 show the architecture of a distributed machine learning system. The system consists of multiple machines (Machine 1 to Machine 5) that process data and predict labels. The data is stored in a database (Big data) and is distributed across the machines. The machines are connected to a central database and a prediction server.

The architecture is based on Docker, which allows for the deployment of containerized applications.

## 7.2. Docker



Figure 38. Docker logo

Docker is a container management system that allows for the deployment of containerized applications. It is based on the Linux kernel's namespaces and cgroups. Docker containers are lightweight and can be deployed across different operating systems (OS). The architecture is shown in Figure 37.

Docker is a container management system that allows for the deployment of containerized applications. It is based on the Linux kernel's namespaces and cgroups. Docker containers are lightweight and can be deployed across different operating systems (OS). The architecture is shown in Figure 37. Docker is a container management system that allows for the deployment of containerized applications. It is based on the Linux kernel's namespaces and cgroups. Docker containers are lightweight and can be deployed across different operating systems (OS). The architecture is shown in Figure 37.

VM (Virtual Machine) は、OSを仮想化する (Figure 39) VM は Hypervisor (Hypervisor) によって管理される (CPU, RAM, network など) VM は OS (CPU 4コア、メモリ 2GB) VM OS OS-A OS-B CPU isolation VM VMware VirtualBox Xen EC2 VM

Docker VM OS OS Docker (Figure 39) Docker OS OS Docker OS VM (=) OS Docker VM (Docker VM) Docker

VM Docker Docker 2013

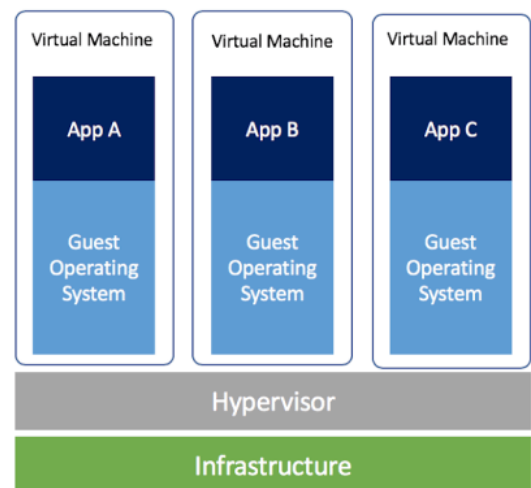
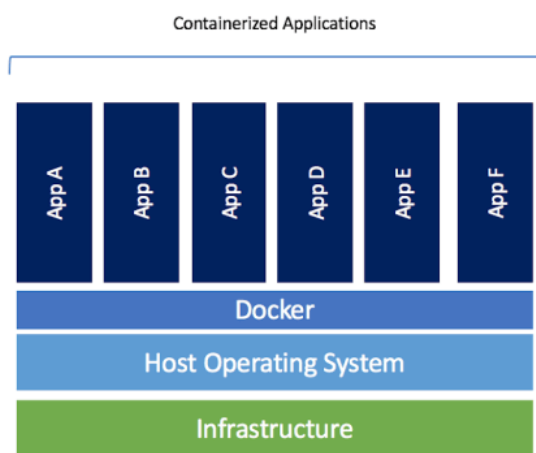


Figure 39. Docker ( ) VM ( ) (URL: <https://www.docker.com/blog/containers-replacing-virtual-machines/>)

## Git: 開発者にとって?

開発者にとって "Git, Vim Docker" は

Git は Linux Linus Torvalds 2005

Vim 1991 30 Stackoverflow 2019

Docker Docker OS Docker

[illegible]

### 7.3.3. 実行

Pull したイメージを実行する `run` コマンド

```
$ docker run -it ubuntu:18.04
```

実行 `-it` オプションは `shell` を実行するイメージを実行する

実行したイメージは `Ubuntu` イメージを実行するイメージ (Figure 41) のイメージを実行する (イメージ) のイメージ **Container** (イメージ) のイメージ

```
tomoyuki@eiffel:~$ docker run -it ubuntu:18.04
root@3d7e5903b640:/#
```

Figure 41. Docker で `ubuntu:18.04` を実行する

実行した `ubuntu:18.04` イメージは `Ubuntu OS` のイメージを実行するイメージ (Chapter 6) の `DLAMI` のイメージを実行するイメージ `PyTorch` のイメージを実行するイメージ `PyTorch` のイメージ `Docker Hub` のイメージを実行するイメージ

実行する

```
$ docker run -it pytorch/pytorch
```



`docker run` のイメージを実行するイメージ `Docker Hub` のイメージを実行するイメージ

`pytorch` のイメージを実行する `Python` のイメージを実行する `pytorch` のイメージを実行する

```
$ python3
Python 3.7.7 (default, May 7 2020, 21:25:33)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.cuda.is_available()
False
```

実行した `Docker` のイメージを実行する `OS` のイメージを実行するイメージ

### 7.3.4. 実行イメージ

実行したイメージを実行するイメージ

実行した `docker` のイメージを実行するイメージ `Python`, `Node.js`, `AWS CLI`, `AWS CDK` のイメージを実行するイメージ

実行した `docker` のイメージを実行する `Dockerfile` のイメージを実行するイメージ

実行した `Docker` のイメージを実行する (`docker/Dockerfile`)

```
FROM node:12
LABEL maintainer="Tomoyuki Mano"

RUN apt-get update \
    && apt-get install nano

①
RUN cd /opt \
    && curl -q "https://www.python.org/ftp/python/3.7.6/Python-3.7.6.tgz" -o Python-3.7.6.tgz \
    && tar -xzf Python-3.7.6.tgz \
    && cd Python-3.7.6 \
    && ./configure --enable-optimizations \
    && make install

RUN cd /opt \
    && curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" \
    && unzip awscliv2.zip \
    && ./aws/install

②
RUN npm install -g aws-cdk@1.100

# clean up unnecessary files
RUN rm -rf /opt/*

# copy hands-on source code in /root/
COPY handson/ /root/handson
```

**Dockerfile** の書き方について、**<1>** は **Python 3.7** をインストールするコマンド、**<2>** は **AWS CDK** をインストールするコマンド。また、**OS** は **Docker** で実行されるコンテナのオペレーティングシステムを指定する。

"oooooooooooooooooooo..."      oooooooooooooooooooooooooooooo      Docker      oooooooooooooooooooooooooooooo  
oooooooooooooooooooooooooooo Docker oooooooooooooooooooooo

## □□□: Is Docker alone?

1. 在 Docker 中安装 API 接口  
 2. 在 Docker 中安装 Docker

**Singularity**  **HPC (High Performance Computing)**  **Singularity**  
 **HPC**  **Docker**  **root**   
**Singularity**  (root)  **root**  **Web**  
 **HPC**  **Singularity**  **Docker**  **Singularity**

podman Red Hat podman Docker  
 Red Hat podman Singularity  
 HPC pod  
 Docker

## 7.4. Elastic Container Service (ECS)



Figure 42. ECS

Docker AWS Docker

**Elastic Container Service (ECS)** Docker AWS (Figure 42) ECS Docker (=)

ECS Figure 43 ECS (Task) ECS Docker Docker Hub AWS Docker ECR (Elastic Container Registry)

ECS Docker "Docker Hub" Docker

ECS ECS (80%) scale-out (scale-in) ECS AWS Auto scaling group (ASG) Fargate ASG Chapter 9, Fargate Chapter 8

ECS (Docker Hub) Docker

Docker AWS

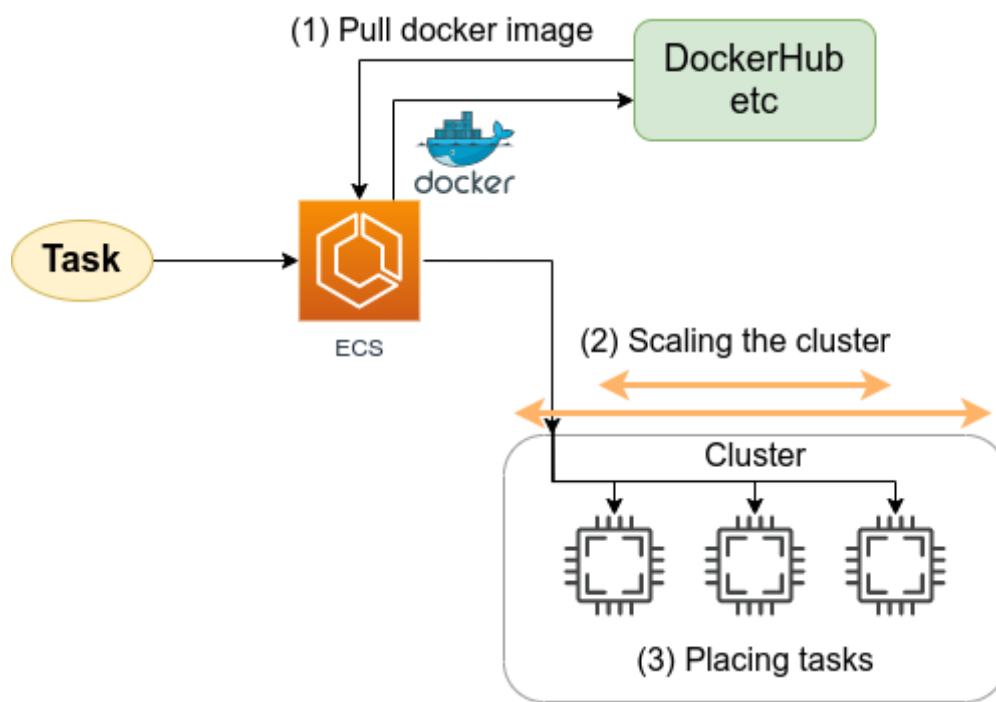


Figure 43. ECS



## 8.1. Fargate



64



## 8.2.

이 레시피는 GitHub 에서 [handson/qa-bot](#) 이라는

컨테이너화된 질문-답변 서비스 (Section 4.1) 를 배포하는 방법을 보여줍니다. 이 서비스는 Docker



이 서비스는 1CPU/4GB RAM 을 필요로 하며 Fargate 에 배포할 경우 가격은 0.025 \$/hour 입니다.

## 8.3. Transformer 기반 question-answering 서비스

이 서비스는 컨텍스트(context) 텍스트와 질문(question) 텍스트를 기반으로

context: Albert Einstein (14 March 1879 - 18 April 1955) was a German-born theoretical physicist who developed the theory of relativity, one of the two pillars of modern physics (alongside quantum mechanics). His work is also known for its influence on the philosophy of science. He is best known to the general public for his mass-energy equivalence formula  $E = mc^2$ , which has been dubbed "the world's most famous equation". He received the 1921 Nobel Prize in Physics "for his services to theoretical physics, and especially for his discovery of the law of the photoelectric effect", a pivotal step in the development of quantum theory.

question: In what year did Einstein win the Nobel prize?

이 서비스는 컨텍스트(context) 텍스트를 기반으로 질문(question) 텍스트에 대한

answer: 1921

이 서비스는 컨텍스트(context) 텍스트를 기반으로 질문(question) 텍스트에 대한

이 서비스는 [huggingface/transformers](#) 라이브러리를 사용하여 Q&A 서비스를 제공합니다. 이 서비스는 Q&A 서비스 Transformer (Figure 45)를 사용하여 Docker 컨테이너로 Docker Hub에 배포할 수 있습니다.

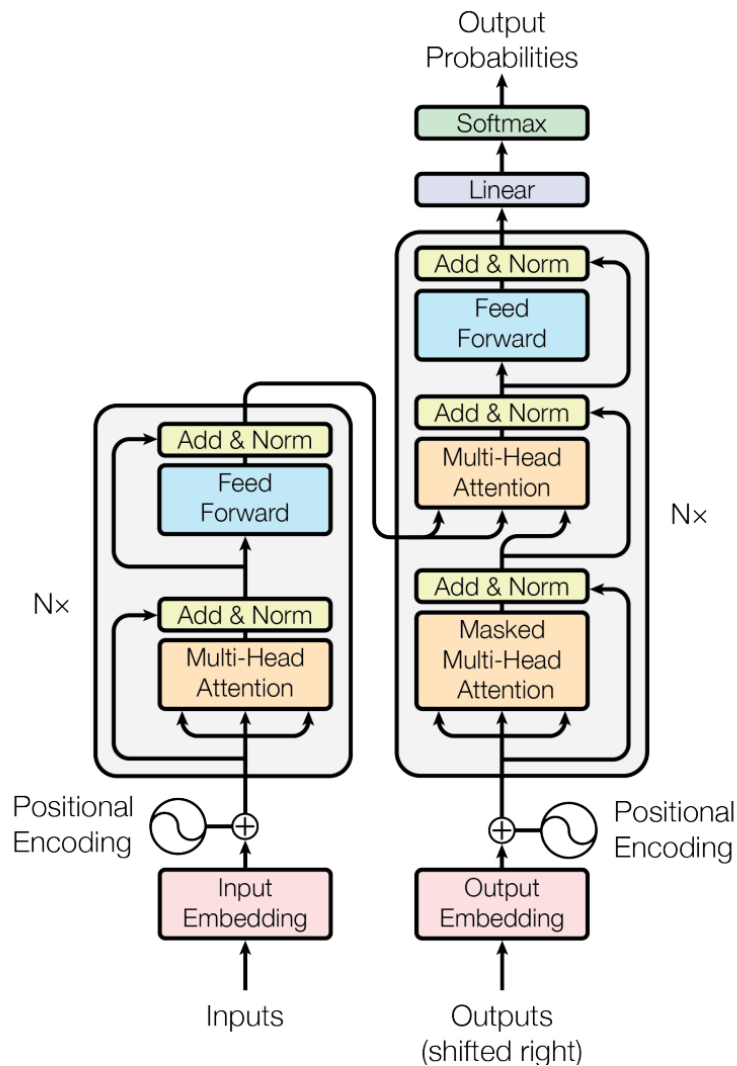


Figure 45. Transformer (Source: Vaswani+ 2017)



Transformer is a deep learning architecture that can be used for both sequence-to-sequence (Seq2Seq) and sequence-to-text (Seq2Text) tasks. It is a type of neural network that uses self-attention mechanisms to process input sequences. The architecture is designed to be highly parallelizable, allowing it to be trained on large datasets using multiple GPUs. The Transformer model is composed of two main parts: an encoder and a decoder. The encoder processes the input sequence and the decoder generates the output sequence. The model is trained using a combination of supervised and unsupervised learning.

Transformer Docker image (pull) command

```
$ docker pull tomomano/qabot:latest
```

pull Transformer Docker image context question

```
$ context="Albert Einstein (14 March 1879 - 18 April 1955) was a German-born theoretical physicist who developed the theory of relativity, one of the two pillars of modern physics (alongside quantum mechanics). His work is also known for its influence on the philosophy of science. He is best known to the general public for his mass-energy equivalence formula  $E = mc^2$ , which has been dubbed the world's most famous equation. He received the 1921 Nobel Prize in Physics for his services to theoretical physics, and especially for his discovery of the law of the photoelectric effect, a pivotal step in the development of quantum theory."
$ question="In what year did Einstein win the Nobel prize ?"
```





```

11         name="item_id", type=dynamodb.AttributeType.STRING
12     ),
13     billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
14     removal_policy=cdk.RemovalPolicy.DESTROY
15 )
16
17 ②
18 vpc = ec2.Vpc(
19     self, "EcsClusterQaBot-Vpc",
20     max_azs=1,
21 )
22
23 ③
24 cluster = ecs.Cluster(
25     self, "EcsClusterQaBot-Cluster",
26     vpc=vpc,
27 )
28
29 ④
30 taskdef = ecs.FargateTaskDefinition(
31     self, "EcsClusterQaBot-TaskDef",
32     cpu=1024, # 1 CPU
33     memory_limit_mib=4096, # 4GB RAM
34 )
35
36 # grant permissions
37 table.grant_read_write_data(taskdef.task_role)
38 taskdef.add_to_task_role_policy(
39     iam.PolicyStatement(
40         effect=iam.Effect.ALLOW,
41         resources=["*"],
42         actions=["ssm:GetParameter"]
43     )
44 )
45
46 ⑤
47 container = taskdef.add_container(
48     "EcsClusterQaBot-Container",
49     image=ecs.ContainerImage.from_registry(
50         "tomomano/qabot:latest"
51     ),
52 )

```

① DynamoDB の作成

② #1, #2 VPC の作成

③ ECS クラスター (cluster) の作成

④ タスク定義 (task definition) の作成

⑤ コンテナイメージ Docker の作成

## 8.4.1. ECS の Fargate

ECS の Fargate を使ってみよう

```
1 cluster = ecs.Cluster(  
2     self, "EcsClusterQaBot-Cluster",  
3     vpc=vpc,  
4 )  
5  
6 taskdef = ecs.FargateTaskDefinition(  
7     self, "EcsClusterQaBot-TaskDef",  
8     cpu=1024, # 1 CPU  
9     memory_limit_mib=4096, # 4GB RAM  
10 )  
11  
12 container = taskdef.add_container(  
13     "EcsClusterQaBot-Container",  
14     image=ecs.ContainerImage.from_registry(  
15         "tomomano/qabot:latest"  
16     ),  
17 )
```

`cluster` = ECS クラスターオブジェクト

`taskdef=ecs.FargateTaskDefinition` は Fargate を使ったタスク定義で、1 CPU、4GB RAM のコンテナを動かすことができる。1024 CPU、4096 MB RAM の設定になっている。

`container` = Docker image を指定して、Docker Hub から image を取得する。

このようにして ECS Fargate を使ったタスク定義を作成する。



`cpu=1024` は CPU の数で、1 CPU は 1024 vCPU (virtual CPU; vCPU) に相当する。0.25 vCPU から 0.5 vCPU、1 vCPU、2 vCPU、4 vCPU、8 vCPU、16 vCPU、32 vCPU、64 vCPU、128 vCPU、256 vCPU、512 vCPU、1024 vCPU、2048 vCPU のように指定できる。1024 CPU は 2 GB から 8 GB のメモリに相当する。詳しくは ["Amazon ECS on AWS Fargate"](#) を参照。

Table 4. CPU とメモリの関係

CPU の数	メモリ
256 (.25 vCPU)	0.5 GB, 1 GB, 2 GB
512 (.5 vCPU)	1 GB, 2 GB, 3 GB, 4 GB
1024 (1 vCPU)	2 GB, 3 GB, 4 GB, 5 GB, 6 GB, 7 GB, 8 GB
2048 (2 vCPU)	Between 4 GB and 16 GB in 1-GB increments

4096 (4 vCPU)

Between 8 GB and 30 GB in 1-GB increments

## 8.5. 環境構築

環境構築の手順を説明します。

環境構築の手順は、SSH で環境に接続し、以下のコマンドを実行します。 (# はプロンプト、\$ はコマンドラインプロンプト) 1, 2 は、必要に応じて (Section 15.3) のように変更してください。

```
# 環境構築
$ cd handson/qa-bot

# venv の作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# デプロイ
$ cdk deploy
```

環境構築の手順は Figure 47 のように実行します。

```
✓ EcsClusterQaBot
Outputs:
EcsClusterQaBot.ClusterName = EcsClusterQaBot-EcsClusterQaBotCluster6488E31F-XXXX
EcsClusterQaBot.TaskDefinitionArn = arn:aws:ecs:ap-northeast-1:606887060834:task-definition/EcsClusterQaBotEcsClusterQaBotTaskDef:XXXX
Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/EcsClusterQaBot/f89b5980-b42d-XXXX-ac70
(.env) tomoyuki@balthasar:03-qa-bot$
```

Figure 47. CDK の実行結果

AWS のコンソールで、ECS クラスタを確認します。 Figure 48 のように、EcsClusterQaBot-XXXX というクラスタが作成されています。

クラスタのタスクを確認すると、 Figure 48 のように FARGATE タスクが 0 Running tasks, 0 Pending tasks と表示されています。

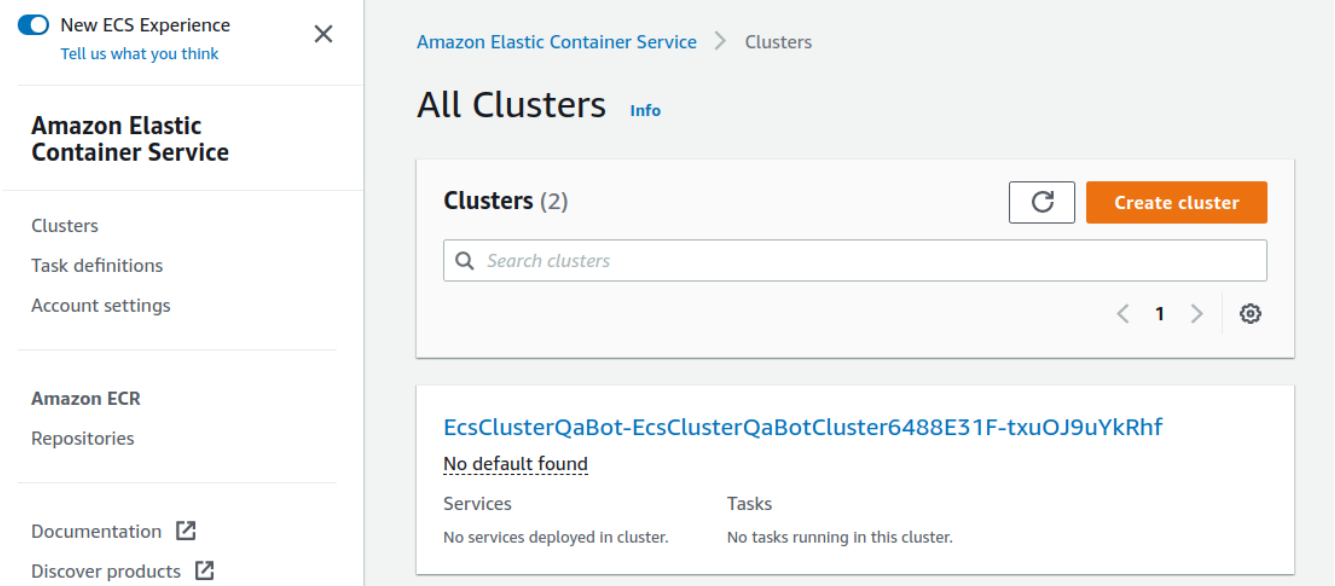


Figure 48. ECS console

Task Definitions

EcsClusterQaBotEcsClusterQaBotTaskDefXXXX

CPU Docker container Task Definition

### Task size ?

The task size allows you to specify a fixed size for your task. Task size is required for tasks using the Fargate launch type and is optional for the EC2 or External launch type. Container level memory settings are optional when task size is set. Task size is not supported for Windows containers.

**Task memory (MiB)** 4096

**Task CPU (unit)** 1024

**Task memory maximum allocation for container memory reservation**

0 4096 shared of 4096 MiB

**Task CPU maximum allocation for containers**

0 1024 shared of 1024 CPU units

### Task Placement

**Constraint** No constraints

### Container Definitions

Container ...	Image	CP...	GP...	Infe...	Hard/Soft memory limits (MiB)	Ess...
▼ EcsClu...	tomomano/qabot:latest	0			--/--	true

Figure 49. Task definition



## 8.6. 実装

実装の概要を説明します。

ECS のタスク定義 (run\_task.py) は [\(handson/qa-bot/run\\_task.py\)](#) を

参照してください。ECS のタスク定義は次のようになります。

```
$ python run_task.py ask "A giant peach was flowing in the river. She picked it up and brought it home. Later, a healthy baby was born from the peach. She named the baby Momotaro." "What is the name of the baby?"
```



run\_task.py はタスク定義を AWS の ECS に登録します。

"ask" は context (context) と question (question) を受け取ります。

タスク定義は "Waiting for the task to finish..." というメッセージを返します。AWS の ECS は Fargate を使用して Docker コンテナを実行し、AWS の ECS コンソールでタスクを確認できます。

ECS のタスク定義を確認するには、AWS の ECS コンソールで "Tasks" をクリックします (Figure 50)。

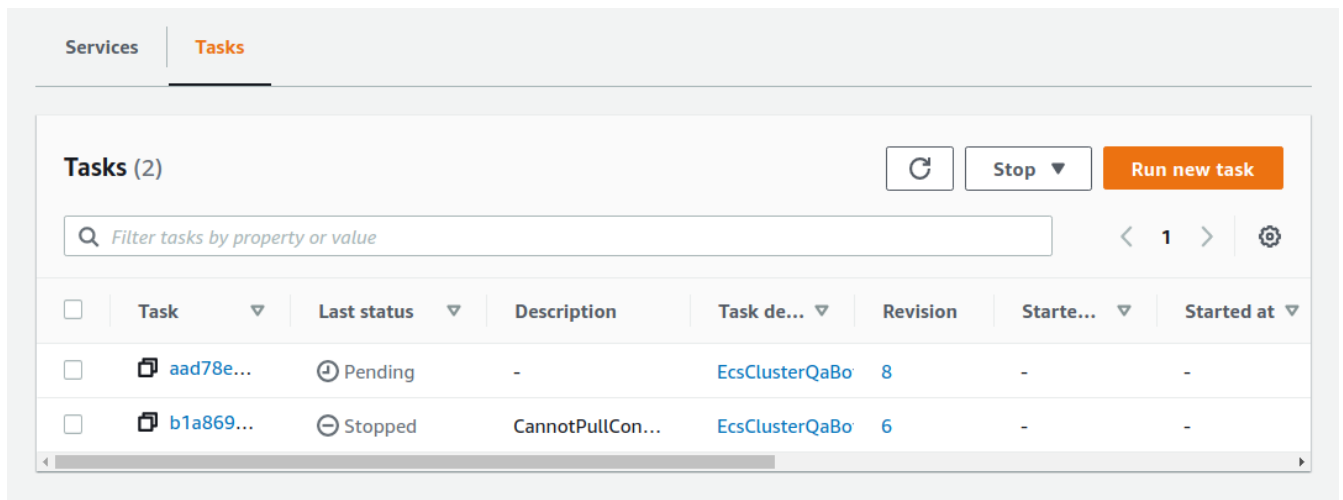


Figure 50. ECS のタスク定義を確認する

Figure 50 の "Last status = Pending" はタスク定義が実行中であることを示します。Fargate は Docker image を使用してタスクを実行します。

タスクの Status が "RUNNING" であることを確認します。Status が "STOPPED" である ECS のタスクは Fargate のタスク定義を確認してください。

Figure 50 の "Task" はタスクの ID を示します (Figure 51)。

"Last status", "Platform version" はタスクのステータスとプラットフォームバージョンを示します。

"Logs" はタスクのログを確認できます。



```
$ python run_task.py ask_many
```

Amazon ECS を使ったタスクの実行 (Figure 53) は、Amazon Fargate を使ったタスクの実行と似ています。

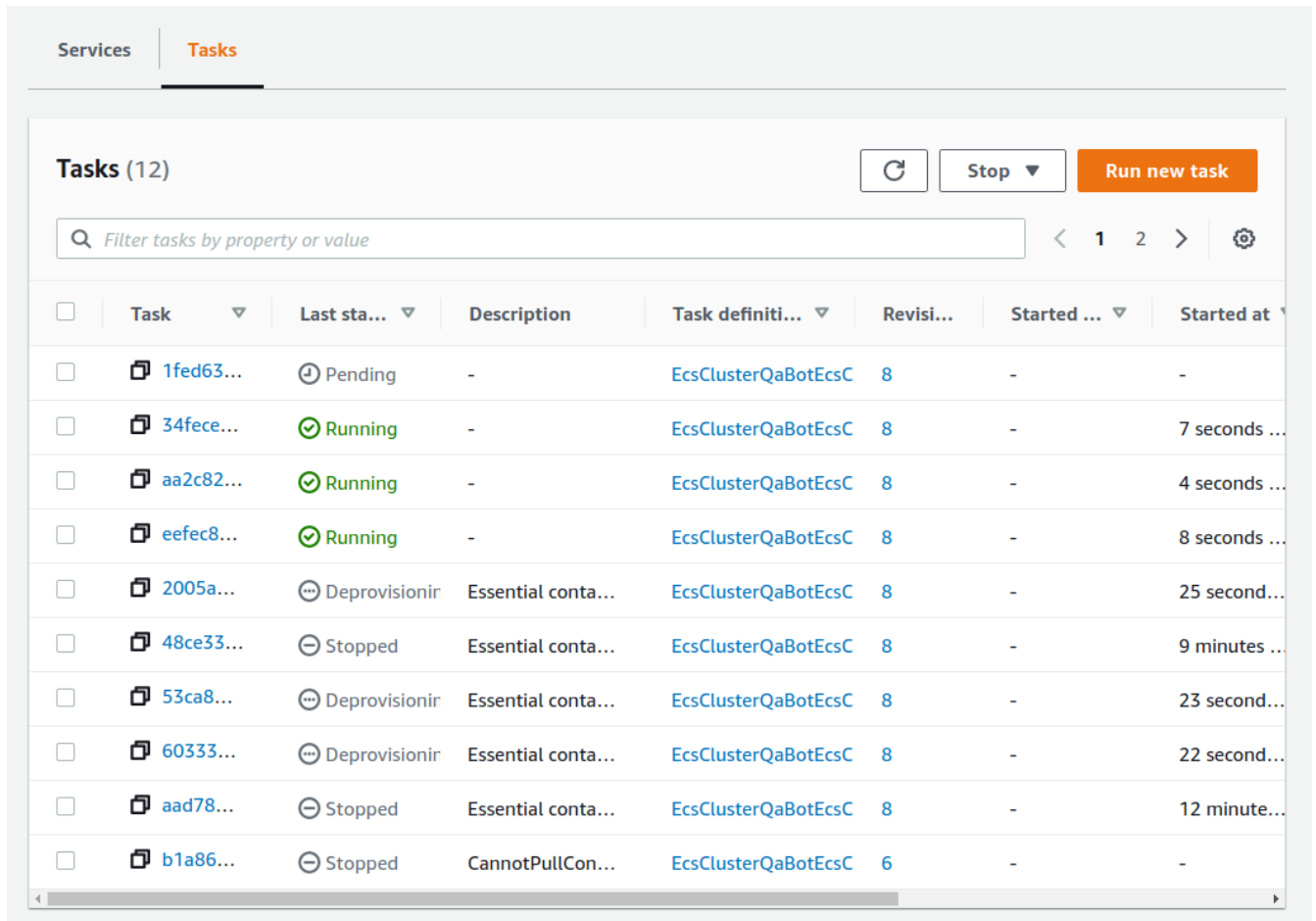


Figure 53. タスクの実行状況

タスクが "STOPPED" の状態にある場合は、タスクのステータスを "STOPPED" に変更する必要があります。

```
$ python run_task.py list_answers
```

Figure 54 は、タスクの実行状況を確認するためのコマンドです。







Figure 55. AWS Batch

EC2 ASG ECS ASG  
ECS ASG  
AWS Batch

AWS Batch (Batch) ( )  
AWS Batch  
ECS/ASG/EC2

AWS Batch (Figure 56) Job AWS Batch  
Job definitions Docker  
CPU/RAM Job definition Job queues  
Job queues priority queue queue  
priority ( ) priority queue  
( " " ) Compute environment  
(EC2 Fargate ) Compute environment EC2  
Job queues Compute environment  
Compute environment

AWS Batch

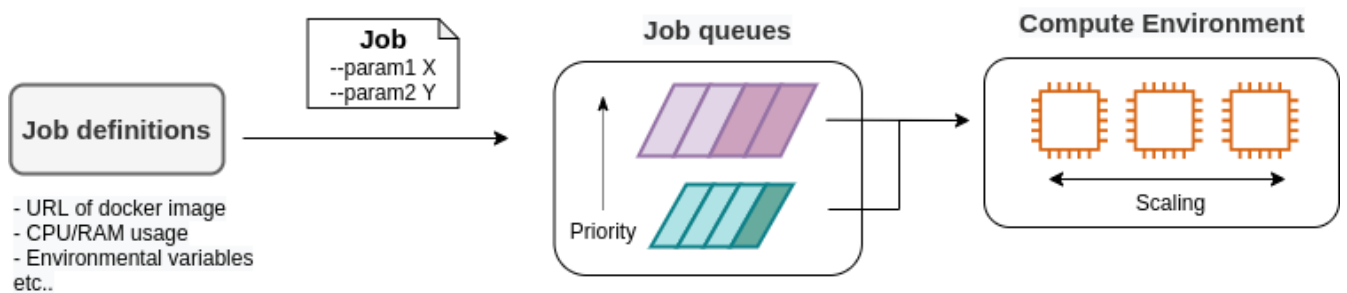


Figure 56. AWS Batch

### EC2 or Fargate?



ECS EC2 Fargate  
Table 5  
Fargate

Table 5. EC2 vs Fargate

	EC2	Fargate
Compute capacity	Medium to large	Small to medium

	EC2	Fargate
GPU	Yes	No
Launch speed	Slow	Fast
Task placement flexibility	Low	High
Programming complexity	High	Low

EC2 CPU GPU  
Fargate CPU 4  
Fargate  
Fargate  
2  
CPU  
Fargate

EC2 Fargate EC2 Fargate

### 9.3.

GitHub [handson/aws-batch](#)

(Section 4.1) Docker



`g4dn.xlarge` EC2 (`us-east-1`) 0.526 \$/hour  
(`ap-northeast-1`) 0.71 \$/hour



Section 6.1 G AWS EC2  
Section 9.5

### 9.4. MNIST ( )

Section 6.7 MNIST  
Section 6.7  
(SGD)

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

(`lr=0.01`) (`momentum=0.5`)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

[illegible][illegible][illegible]

0000000000000000000000000000000000000000000000000000000000000000  
000000000000000000000000000000000000000000000000000000000000

[illegible]

Embarassingly parallel

Embarassingly parallel

optuna  
GPU

# ○○○○○○○○○○ Docker ○○○○○○○○○○○○○○○○○○○

Docker      [handson/aws-batch/docker](#)      Section 6.7

00000000 Docker 000000000000000000000000 Dockerfile 00000000000000000000 mymnist 000000 (Tag)  
 000000000000000000

```
$ cd hands-on/aws-batch/docker
$ docker build -t mymnist .
```



**docker**   **build**        **MNIST**        

<http://yann.lecun.com/exdb/mnist/>   



~~~~~ Docker Hub ~~~ pull ~~~~~

```
$ docker pull tomomano/mymnist:latest
```

□□□□□□□□□□□□□□□□□□□□□□□□ MNIST □□□□□□□□□□


```
$ docker run mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

`--lr` `--momentum` `--epochs` Chapter 6 Loss (Figure 57)

```

Train Epoch: 0 [0/48000 (0.0%)] Loss: 2.297341
Train Epoch: 0 [6400/48000 (13.3%)] Loss: 0.298299
Train Epoch: 0 [12800/48000 (26.7%)] Loss: 0.083849
Train Epoch: 0 [19200/48000 (40.0%)] Loss: 0.252933
Train Epoch: 0 [25600/48000 (53.3%)] Loss: 0.160509
Train Epoch: 0 [32000/48000 (66.7%)] Loss: 0.082315
Train Epoch: 0 [38400/48000 (80.0%)] Loss: 0.153711
Train Epoch: 0 [44800/48000 (93.3%)] Loss: 0.222485

Val set: Average loss: 0.0733, Accuracy: 97.8%

Train Epoch: 1 [0/48000 (0.0%)] Loss: 0.165711
Train Epoch: 1 [6400/48000 (13.3%)] Loss: 0.136394
Train Epoch: 1 [12800/48000 (26.7%)] Loss: 0.089186
Train Epoch: 1 [19200/48000 (40.0%)] Loss: 0.095106
Train Epoch: 1 [25600/48000 (53.3%)] Loss: 0.025505
Train Epoch: 1 [32000/48000 (66.7%)] Loss: 0.061345
Train Epoch: 1 [38400/48000 (80.0%)] Loss: 0.163712
Train Epoch: 1 [44800/48000 (93.3%)] Loss: 0.122928

Val set: Average loss: 0.0552, Accuracy: 98.4%

```

Figure 57. Docker □□□□□□□□

1. CPU 環境で Docker をインストールし、`nvidia-docker` をインストールして GPU を Docker にマウントする。

```
$ docker run --gpus all mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

```
python3 --gpus all
```

CPU/GPU 時間 (Train 時間) 1 Loss 時間 (Validation 時間) 2 Loss 時間 Accuracy 時間

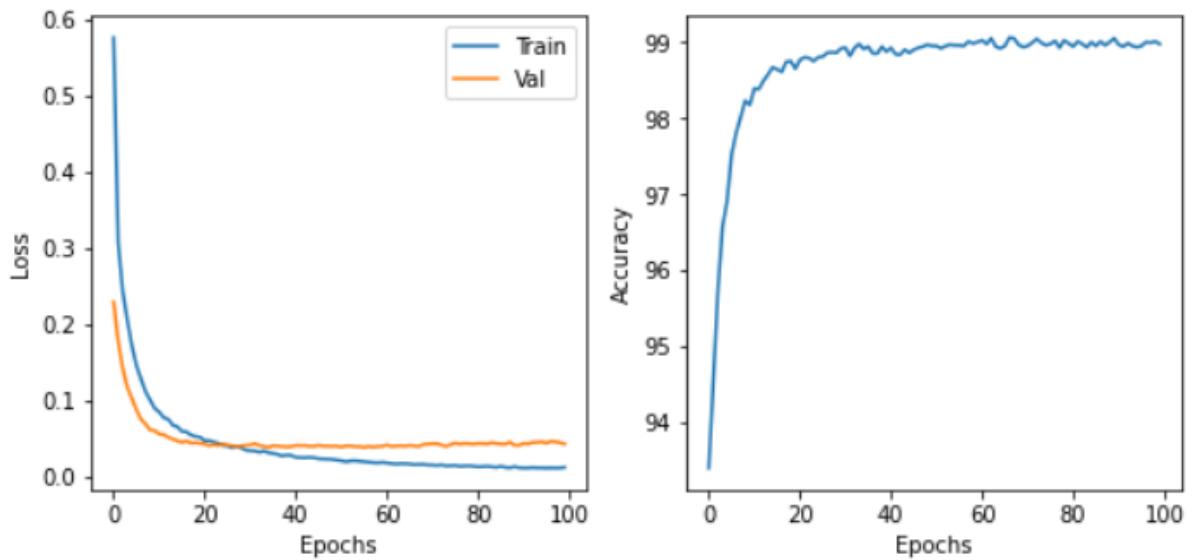


Figure 58. (a) Train/Validation Loss (b) Validation Accuracy

Early stopping is a technique used to prevent overfitting. It involves monitoring the validation loss (or accuracy) and stopping the training process when the performance on the validation set starts to decline (or plateau). This helps to ensure that the model is not overfitting to the training data and is able to generalize to new, unseen data.



MNIST dataset is often used for early stopping experiments. For example, training for 60,000 steps (10,000 epochs) and then evaluating on a validation set. If the accuracy is 80%, it might be a good idea to stop training. Alternatively, training for 48,000 steps (8,000 epochs) and then evaluating on a validation set. If the accuracy is 20%, it might be a good idea to stop training.

9.5. Early stopping

Figure 59 shows the training and validation loss curves for the MNIST dataset. The training loss (blue line) decreases rapidly, while the validation loss (orange line) starts to increase after approximately 40 epochs, indicating overfitting.


```

14         self, "vpc",
15         # other parameters...
16     )
17
18     ②
19     managed_env = batch_alpha.ComputeEnvironment(
20         self, "managed-env",
21         compute_resources=batch_alpha.ComputeResources(
22             vpc=vpc,
23             allocation_strategy=batch_alpha.AllocationStrategy.BEST_FIT,
24             desiredv_cpus=0,
25             maxv_cpus=64,
26             minv_cpus=0,
27             instance_types=[
28                 ec2.InstanceType("g4dn.xlarge")
29             ],
30         ),
31         managed=True,
32         compute_environment_name=self.stack_name + "compute-env"
33     )
34
35     ③
36     job_queue = batch_alpha.JobQueue(
37         self, "job-queue",
38         compute_environments=[
39             batch_alpha.JobQueueComputeEnvironment(
40                 compute_environment=managed_env,
41                 order=100
42             )
43         ],
44         job_queue_name=self.stack_name + "job-queue"
45     )
46
47     ④
48     job_role = iam.Role(
49         self, "job-role",
50         assumed_by=iam.CompositePrincipal(
51             iam.ServicePrincipal("ecs-tasks.amazonaws.com")
52         )
53     )
54     # allow read and write access to S3 bucket
55     bucket.grant_read_write(job_role)
56
57     ⑤
58     repo = ecr.Repository(
59         self, "repository",
60         removal_policy=cdk.RemovalPolicy.DESTROY,
61     )
62
63     ⑥
64     job_def = batch_alpha.JobDefinition(

```

```

65         self, "job-definition",
66         container=batch_alpha.JobDefinitionContainer(
67             image=ecs.ContainerImage.from_ecr_repository(repo),
68             command=["python3", "main.py"],
69             vcpus=4,
70             gpu_count=1,
71             memory_limit_mib=12000,
72             job_role=job_role,
73             environment={
74                 "BUCKET_NAME": bucket.bucket_name
75             }
76         ),
77         job_definition_name=self.stack_name + "job-definition",
78         timeout=cdk.Duration.hours(2),
79     )

```

- ① `BucketName` S3 `BucketName`
- ② `ComputeEnvironmentName` `g4dn.xlarge` `vCPU` `64` `0` `0`
- ③ `<2>` `ComputeEnvironmentName` `Job queue`
- ④ `S3` `IAM` (`IAM` [Section 13.2.5](#))
- ⑤ `Docker image` `ECR`
- ⑥ `Job definition` `4` `vCPU` `12000 MB (=12GB)` `RAM` (`BUCKET_NAME`) `<4>` `IAM`

[illegible]

AWS EC2 Limits (Figure 60) g4dn.xlarge
 (EC2 G Running On-Demand All G instances)
 AWS Running On-Demand All G instances
 "Amazon EC2 service quotas"



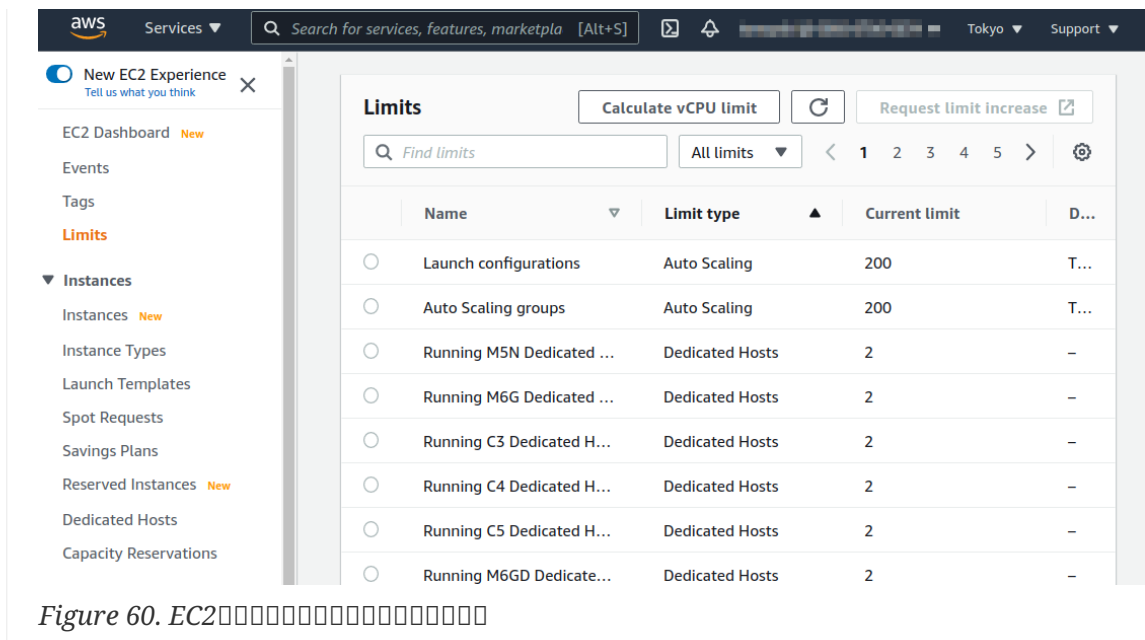


Figure 60. EC2の制限を確認する

9.6. 環境構築

このセクションでは、AWS Batch の環境構築を行います。

このセクションでは、AWS Batch の環境構築を行います (# 環境構築) (Section 15.3)

```
# 環境構築
$ cd handson/aws-batch

# venv の作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# デプロイ
$ cdk deploy
```

このセクションでは、AWS Batch の環境構築を行います。AWS Batch の環境構築には、`batch` という AWS Batch のコンストラクタ (Figure 61)

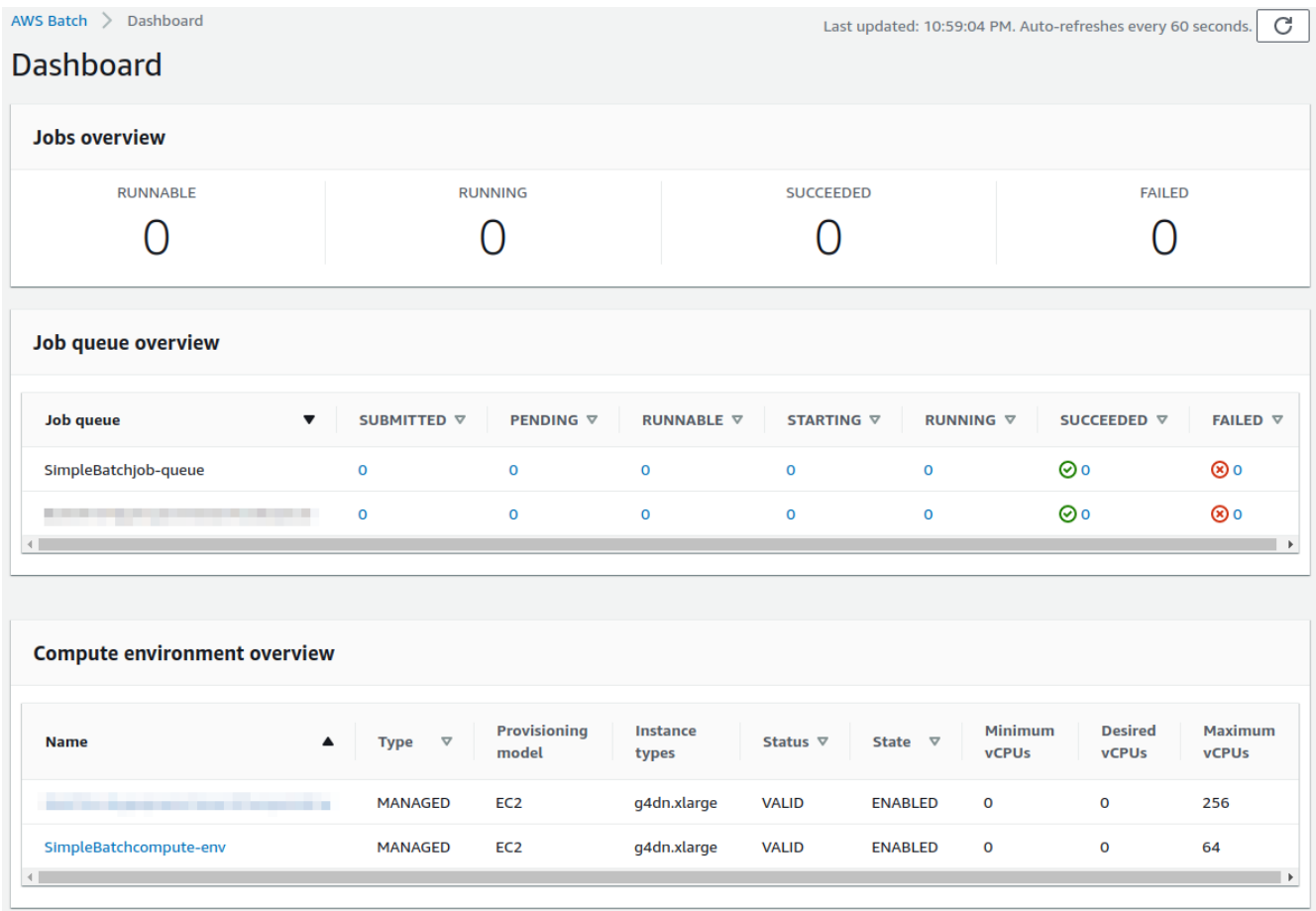


Figure 61. AWS Batch 画面 (画面)

画面の Compute environment overview には SimpleBatchcompute-env という Compute environment が表示されています (画面)。この Compute environment は g4dn.xlarge インスタンスタイプを使用し、Minimum vCPUs は 0、Maximum vCPUs は 64、Desired vCPUs は 0 になっています。Compute environment の詳細は、画面の Compute environment overview を参照してください。

画面の Job queue overview には SimpleBatch-queue という Job queue が表示されています。この Job queue は PENDING, RUNNING, SUCCEEDED, FAILED の状態のジョブを表示しています。

画面の Job definition には SimpleBatchjob-definition という Job definition が表示されています (Figure 62)。この Job definition は vCPUs, Memory, GPU のリソースを指定し、Docker イメージとして ECR からイメージを取得しています。Docker イメージは ECR から取得されています。

| Container properties | |
|------------------------|----------------|
| Command | Image |
| ["python3", "main.py"] | |
| User | vCpus |
| -- | 4 |
| Read only filesystem | Memory |
| -- | 12000 |
| Instance type | Number of GPUs |
| | 1 |

Figure 62. AWS Batch の Job definition

9.7. Docker image を ECR にアップロード

この Batch のコンテナイメージは Docker イメージ (pull) によって提供される (Chapter 8) のイメージを Docker Hub から pull するのではなく、AWS のコンテナイメージレジストリ **ECR (Elastic Container Registry)** から image を提供するように Batch が ECR から pull するように設定する (Figure 59)。

コンテナイメージを ECR にアップロード

```

①
repo = ecr.Repository(
    self, "repository",
    removal_policy=cdk RemovalPolicy.DESTROY,
)

job_def = batch_alpha.JobDefinition(
    self, "job-definition",
    container=batch_alpha.JobDefinitionContainer(
        image=ecs.ContainerImage.from_ecr_repository(repo), ②
        ...
    ),
    ...
)

```

① ECR レジストリを作成

② Job definition のコンテナイメージとして <1> の ECR レジストリにアップロードされたイメージを Job definition のコンテナイメージとして IAM ロールに付与する

コンテナイメージを ECR レジストリにアップロードする Docker push コマンド

コンテナイメージを AWS の ECR レジストリ (Elastic Container Registry) にアップロードする Private レジストリとして simplebatch-repositoryXXXXXX を作成する (Figure 63)。

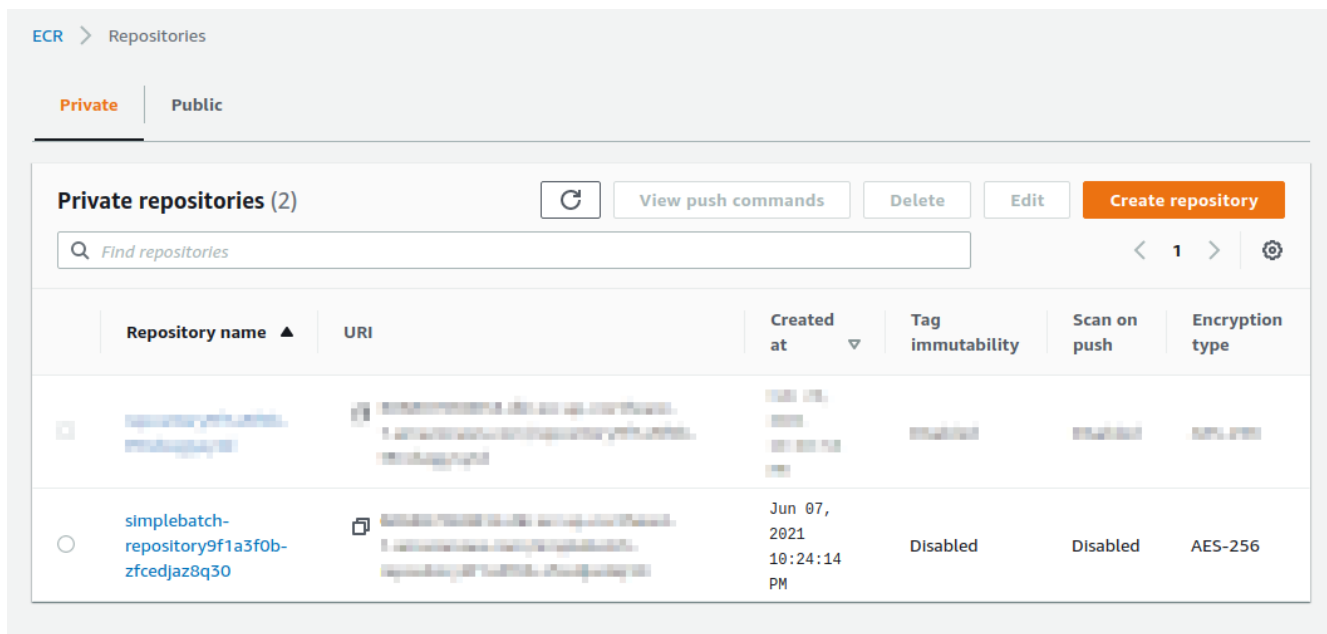


Figure 63. ECR Private repositories

The screenshot shows the ECR Private repositories page. The first repository is 'simplebatch-repository9f1a3f0b-zfcedjaz8q30', created on Jun 07, 2021, with tag immutability disabled and scan on push disabled. The second repository is 'simplebatch-repository9f1a3f0b-zfcedjaz8q30', created on Jun 07, 2021, with tag immutability disabled and scan on push disabled. The page includes a search bar, a 'Find repositories' button, and a 'Create repository' button.

Push commands for simplebatch-repository9f1a3f0b-zfcedjaz8q30

macOS / Linux

Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

- Retrieve an authentication token and authenticate your Docker client to your registry.
Use the AWS CLI:

```
aws ecr get-login-password --region ap-northeast-1 | docker login --username AWS --password-
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.
- Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t simplebatch-repository9f1a3f0b-zfcedjaz8q30 .
```
- After the build completes, tag your image so you can push the image to this repository:

```
docker tag simplebatch-repository9f1a3f0b-zfcedjaz8q30:latest
```
- Run the following command to push this image to your newly created AWS repository:

```
docker push
```

Close

Figure 64. ECR push commands

Before you can push an image to Amazon ECR, you must first build the image using Docker. Then, you must push the image to the repository using the `docker push` command. For more information, see [Pushing an image to Amazon ECR](#).



2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t XXXXX .
```

The `XXXXX` is the name of the image. The `.` is the directory containing the `Dockerfile`.

After the build completes, tag your image so you can push the image to this repository:

```
docker tag XXXXX:latest
```

Then, run the following command to push this image to your newly created AWS repository:

```
docker push XXXXX:latest
```

(Figure 65)

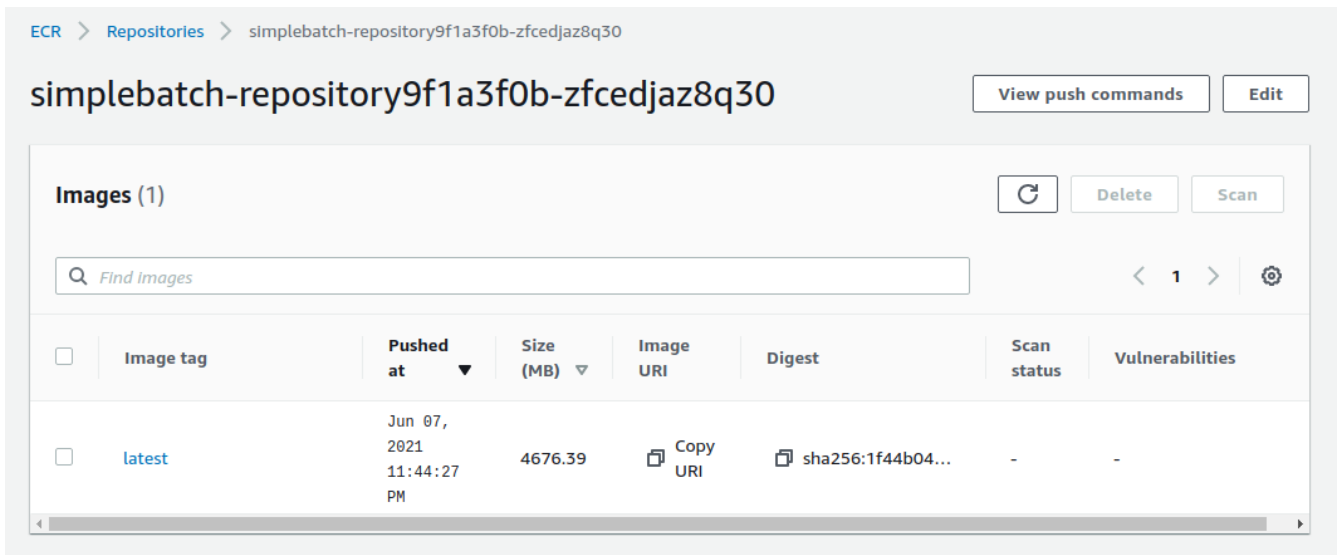


Figure 65. ECR の image の一覧

9.8. バッチジョブの実行

バッチジョブを実行するには AWS Batch を利用します。

まず、`notebook/` ディレクトリに移動し、`run_single.ipynb` ファイルを実行します (`.ipynb` は Jupyter notebook ファイル形式)。

次に、`venv` ディレクトリに移動し、Jupyter notebook を起動します。起動した Jupyter notebook からバッチジョブを実行します。

```
# .env ディレクトリに移動
(.env) $ cd notebook
(.env) $ jupyter notebook
```

Jupyter notebook から `run_single.ipynb` を実行します。

コード [1], [2], [3] はバッチジョブを実行するための `submit_job()` 関数の定義です。

```
1 # [1]
2 import boto3
3 import argparse
4
5 # [2]
6 # AWS プロファイル ...
7
8 # [3]
9 def submit_job(lr:float, momentum:float, epochs:int, profile_name="default"):
10     if profile_name is None:
11         session = boto3.Session()
12     else:
13         session = boto3.Session(profile_name=profile_name)
14     client = session.client("batch")
```

```

15
16 title = "lr" + str(lr).replace(".", "") + "_m" + str(momentum).replace(".", "")
17 resp = client.submit_job(
18     jobName=title,
19     jobQueue="SimpleBatchjob-queue",
20     jobDefinition="SimpleBatchjob-definition",
21     containerOverrides={
22         "command": ["--lr", str(lr),
23                     "--momentum", str(momentum),
24                     "--epochs", str(epochs),
25                     "--uploadS3", "true"]
26     }
27 )
28 print("Job submitted!")
29 print("job name", resp["jobName"], "job ID", resp["jobId"])

```

`submit_job()` の詳細は [Section 9.4](#) の MNIST の Docker の部分を見てください。

```
$ docker run -it mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

この `--lr 0.1 --momentum 0.5 --epochs 10` の部分は、

AWS Batch の `ContainerOverrides` の `command` の部分です。

```

1 containerOverrides={
2     "command": ["--lr", str(lr),
3                 "--momentum", str(momentum),
4                 "--epochs", str(epochs),
5                 "--uploadS3", "true"]
6 }

```

この [4] の部分は、`submit_job()` の引数 `lr = 0.01`, `momentum=0.1`, `epochs=100` の部分です。

```

# [4]
submit_job(0.01, 0.1, 100)

```



AWS の Jupyter Notebook の部分で、Notebook の [2] の部分 (AWS の部分) の `aws secret key` の部分 (Jupyter の部分) は、AWS の部分です。

この `submit_job()` の `profile_name` は、`~/.aws/credentials` の部分 (Section 15.3) の `profile_name` の部分です。

[4] 確認作業が完了したことを確認し、AWS Batch コンソールに移動し、AWS Batch ジョブのステータスを確認する。Figure 66

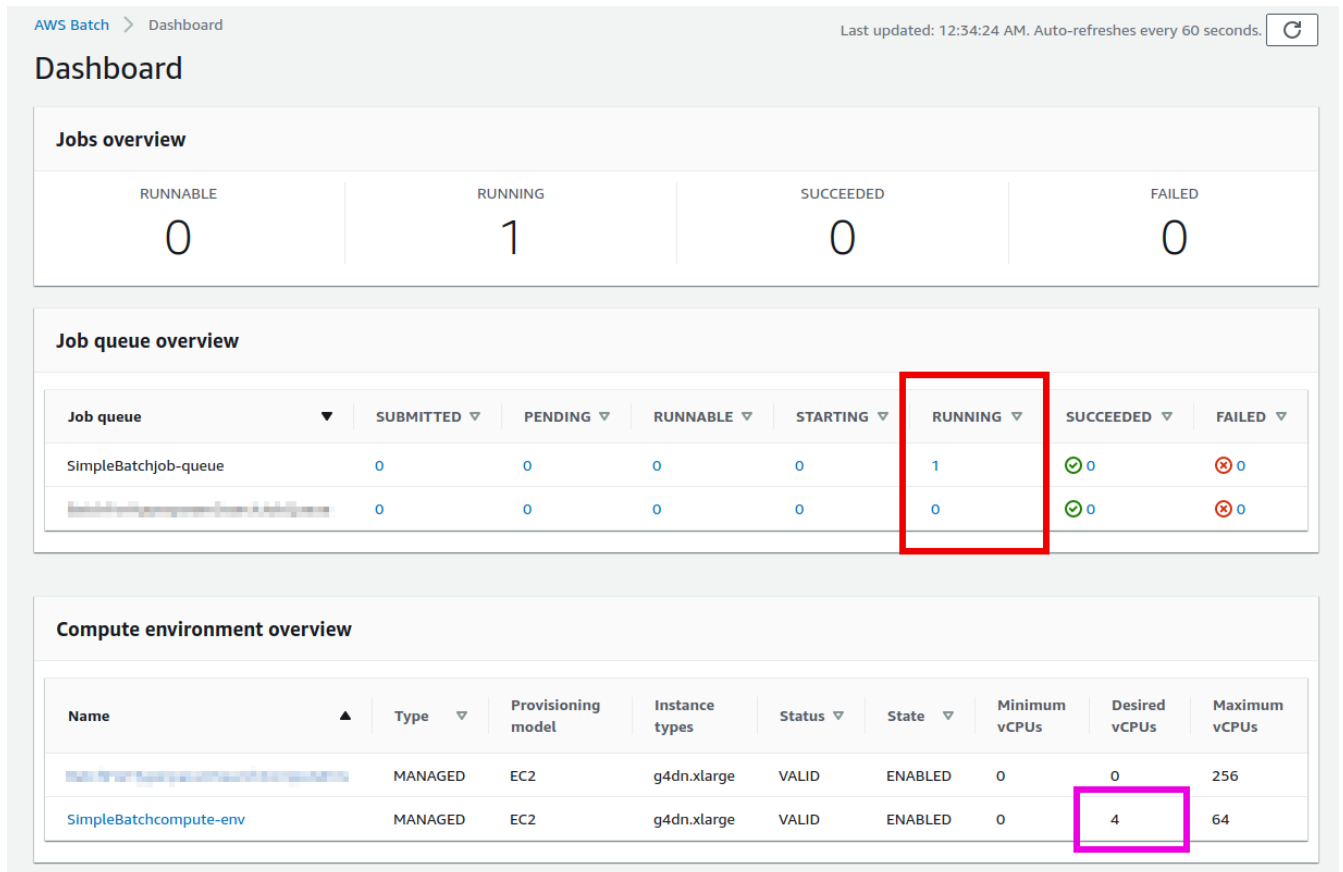


Figure 66. AWS Batch のジョブとコンピュート環境のステータス

Figure 66 のように、AWS Batch のジョブのステータスが SUBMITTED から RUNNING に移行していることが確認できる。また、Compute environment のステータスが VALID であり、Desired vCPU が 4 に設定されていることが確認できる。

次に、AWS Batch のジョブのステータスが RUNNING になっていることを確認する。Compute environment の Desired vCPU が 4 に設定されていることが確認できる。また、g4dn.xlarge インスタンスタイプが使用されていることが確認できる。vCPU の数は、EC2 インスタンスの仕様に基づいて決定される。

ジョブのステータスが RUNNING になっていることを確認し、SUCCEEDED (完了) または FAILED (失敗) のステータスになるまで待つ。MNIST データセットの 10 万枚の画像が SUCCEEDED のステータスになっていることを確認する。

ジョブの完了を確認し、Loss と Accuracy の結果を CSV ファイルとして S3 にアップロードする。AWS Batch のジョブの出力を確認する。

S3 にアップロードされた simplebatch-bucketXXXX (XXXX はバケット名) に metrics_lr0.0100_m0.1000.csv という CSV ファイルがアップロードされていることが確認できる (Figure 67)。このファイルは、Loss = 0.01, Accuracy = 0.1 の結果を示している。

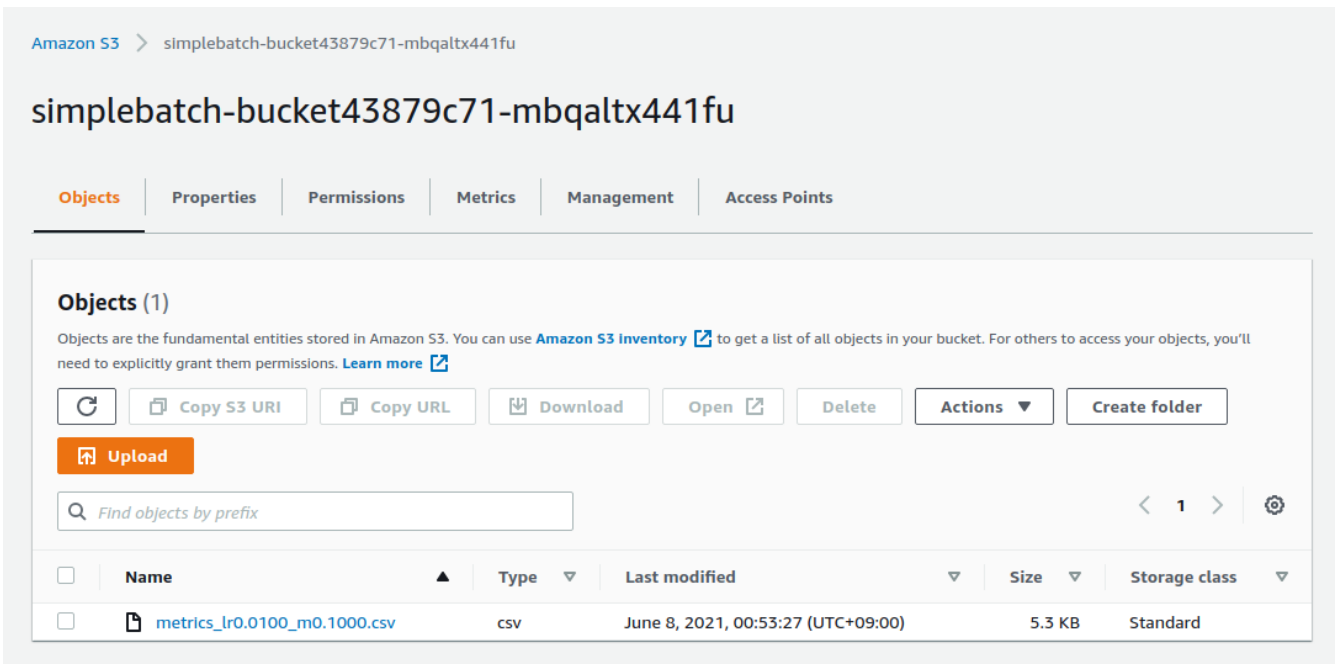


Figure 67. Amazon S3 console

The `run_single.ipynb` notebook [5] in [7] demonstrates how to read a CSV file from Amazon S3.

```

1 # [5]
2 import pandas as pd
3 import io
4 from matplotlib import pyplot as plt
5
6 # [6]
7 def read_table_from_s3(bucket_name, key, profile_name=None):
8     if profile_name is None:
9         session = boto3.Session()
10    else:
11        session = boto3.Session(profile_name=profile_name)
12    s3 = session.resource("s3")
13    bucket = s3.Bucket(bucket_name)
14
15    obj = bucket.Object(key).get().get("Body")
16    df = pd.read_csv(obj)
17
18    return df
19
20 # [7]
21 bucket_name = "simplebatch-bucket43879c71-mbqaltx441fu"
22 df = read_table_from_s3(
23     bucket_name,
24     "metrics_lr0.0100_m0.1000.csv"
25 )

```

[6] is a function that reads a CSV file from Amazon S3 using `pandas` and returns a `DataFrame`. The function takes three arguments: `bucket_name` (the name of the S3 bucket), `key` (the key of the object in the bucket), and `profile_name` (the name of the AWS profile to use). The function returns a `DataFrame` object.

Figure 68) AWS Batch MNIST

```
1 # [9]
2 fig, (ax1, ax2) = plt.subplots(1,2, figsize=(9,4))
3 x = [i for i in range(df.shape[0])]
4 ax1.plot(x, df["train_loss"], label="Train")
5 ax1.plot(x, df["val_loss"], label="Val")
6 ax2.plot(x, df["val_accuracy"])
7
8 ax1.set_xlabel("Epochs")
9 ax1.set_ylabel("Loss")
10 ax1.legend()
11
12 ax2.set_xlabel("Epochs")
13 ax2.set_ylabel("Accuracy")
14
15 print("Best loss:", df["val_loss"].min())
16 print("Best loss epoch:", df["val_loss"].argmin())
17 print("Best accuracy:", df["val_accuracy"].max())
18 print("Best accuracy epoch:", df["val_accuracy"].argmax())
```

```
Best loss: 0.2320499013662338
Best loss epoch: 0
Best accuracy: 99.15833333333332
Best accuracy epoch: 67
```

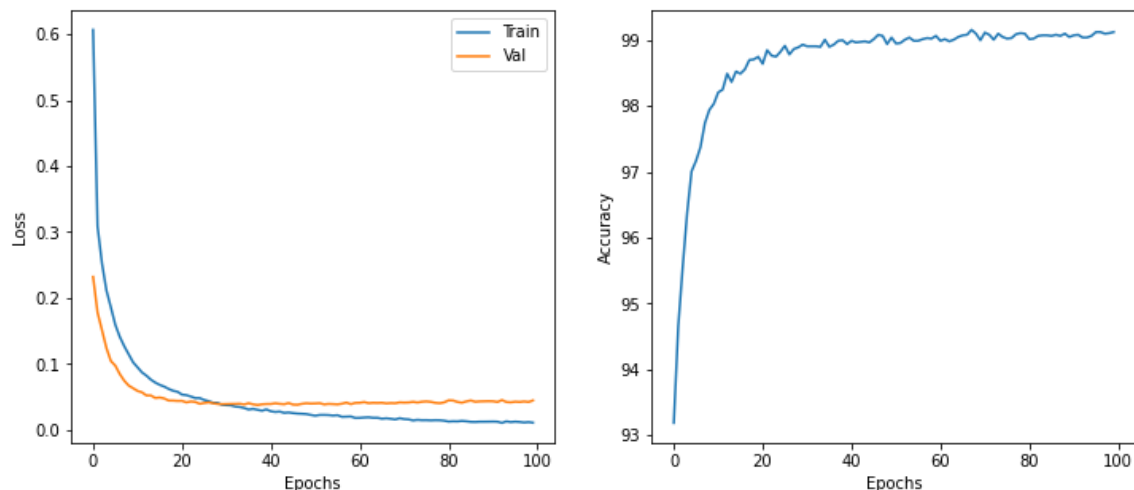


Figure 68. AWS Batch MNIST

9.9. Job

AWS Batch

run_single.ipynb run_sweep.ipynb

[1], [2], [3] run_single.ipynb

```
1 # [1]
```

```
2 import boto3
3 import argparse
4
5 # [2]
6 # AWS ...
7
8 # [3]
9 def submit_job(lr:float, momentum:float, epochs:int, profile_name=None):
10     # ...
```

[4] for batch size 3x3=9

```
1 # [4]
2 for lr in [0.1, 0.01, 0.001]:
3     for m in [0.5, 0.1, 0.05]:
4         submit_job(lr, m, 100)
```

`[4]` Batch `SUBMITTED > RUNNABLE > STARTING > RUNNING`
`9` `RUNNING` ([Figure 69](#)) Compute
environment `Desired vCPUs = 4x9=36` ([Figure 69](#))

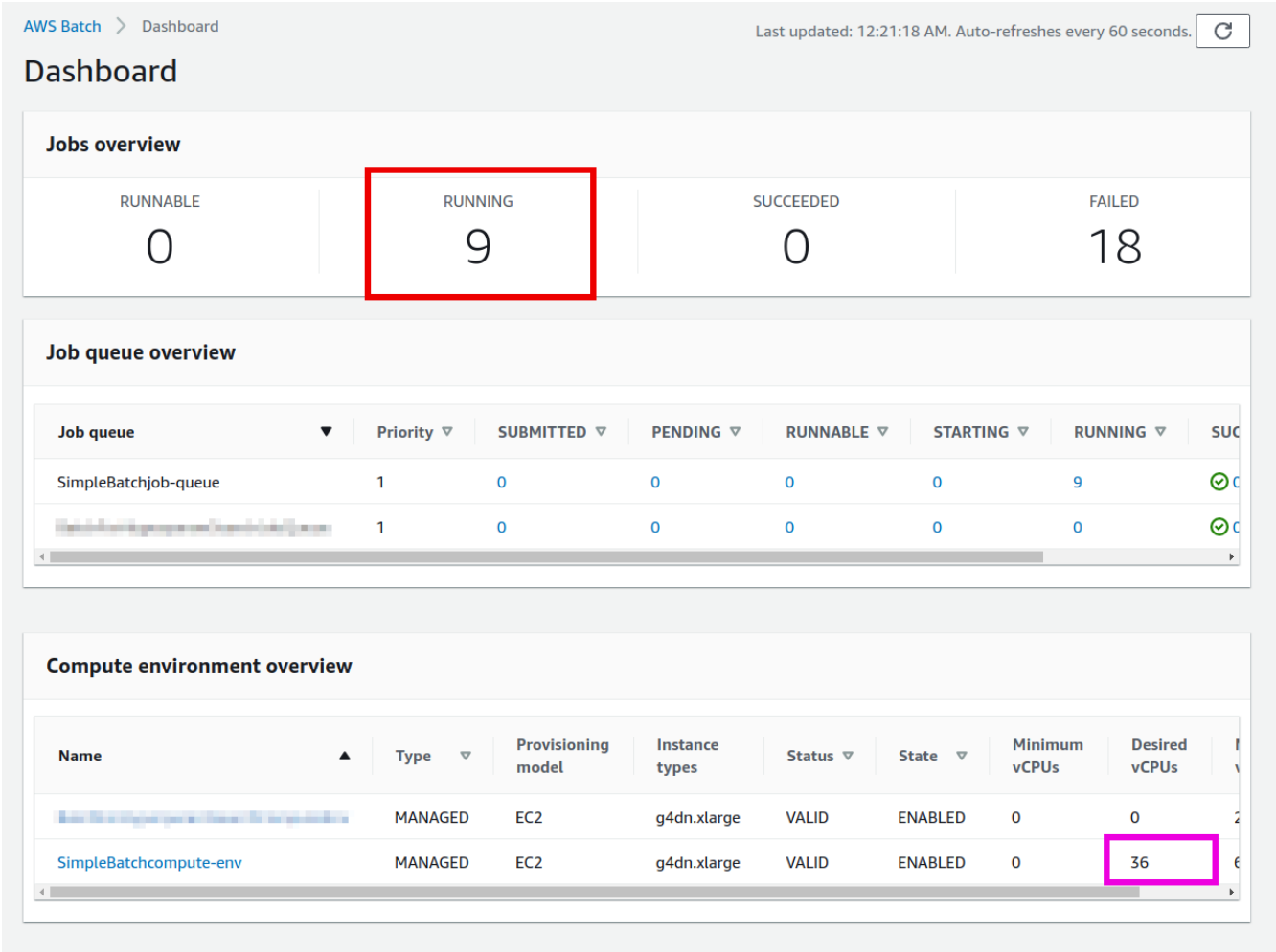
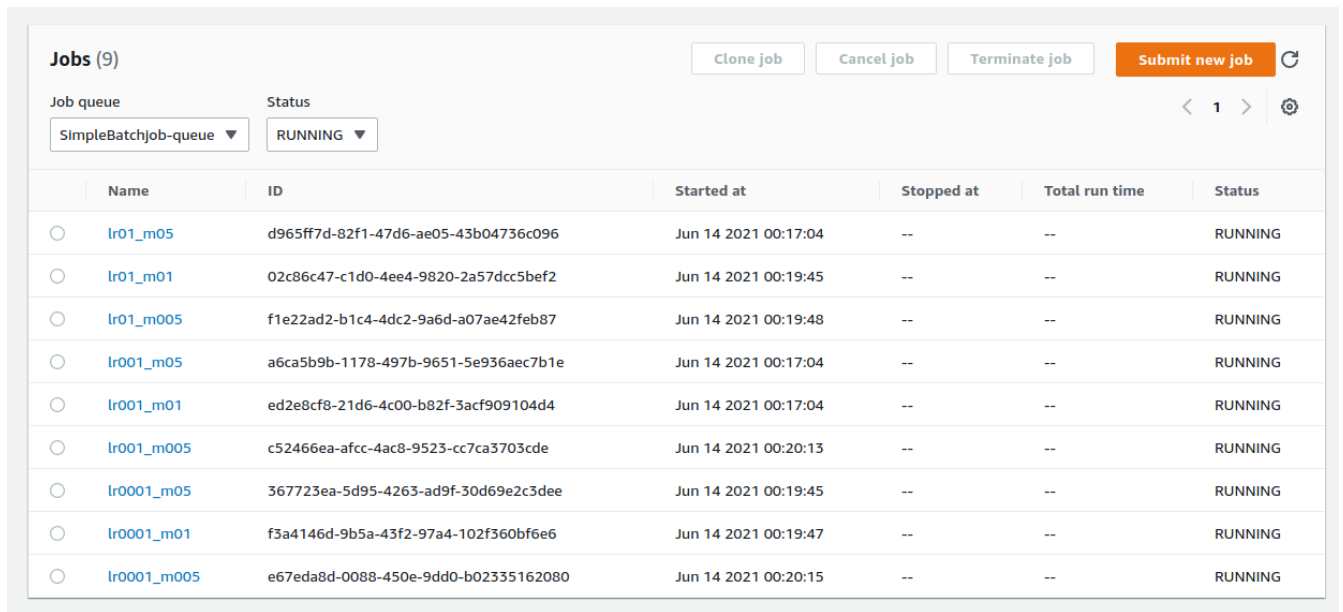


Figure 69. Batch

Batch Jobs (Figure 70)

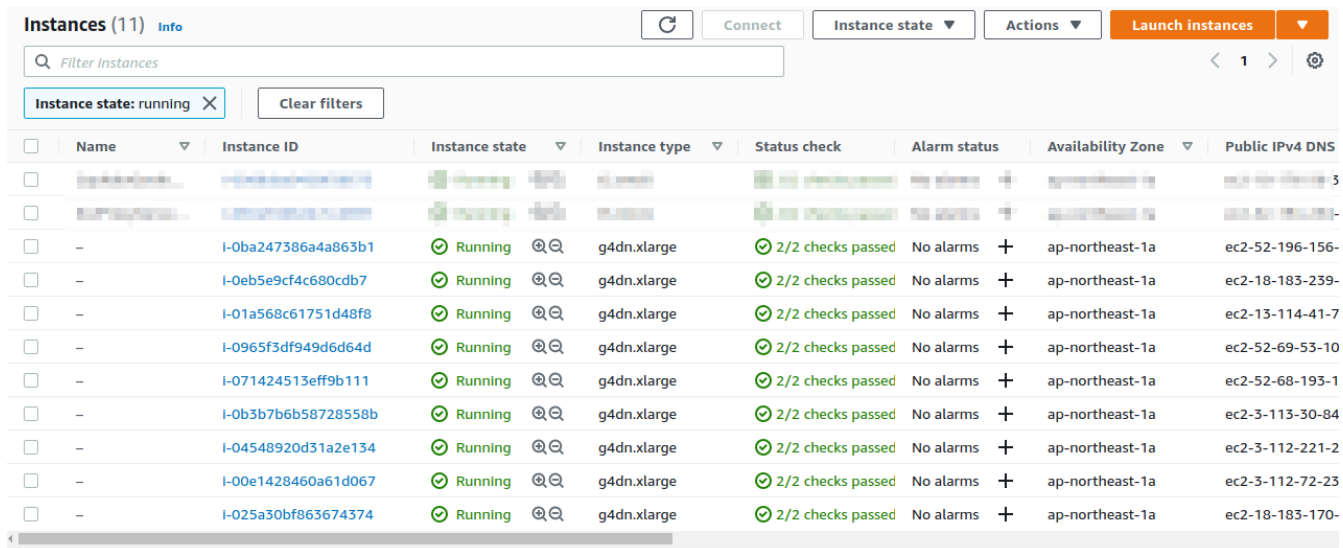
ジョブが9個 RUNNING 状態にあることを確認



| Name | ID | Started at | Stopped at | Total run time | Status |
|-------------|--------------------------------------|----------------------|------------|----------------|---------|
| lr01_m05 | d965ff7d-82f1-47d6-ae05-43b04736c096 | Jun 14 2021 00:17:04 | -- | -- | RUNNING |
| lr01_m01 | 02c86c47-c1d0-4ee4-9820-2a57dcc5bef2 | Jun 14 2021 00:19:45 | -- | -- | RUNNING |
| lr01_m005 | f1e22ad2-b1c4-4dc2-9a6d-a07ae42feb87 | Jun 14 2021 00:19:48 | -- | -- | RUNNING |
| lr001_m05 | a6ca5b9b-1178-497b-9651-5e936aec7b1e | Jun 14 2021 00:17:04 | -- | -- | RUNNING |
| lr001_m01 | ed2e8cf8-21d6-4c00-b82f-3acf909104d4 | Jun 14 2021 00:17:04 | -- | -- | RUNNING |
| lr001_m005 | c52466ea-afcc-4ac8-9523-cc7ca3703cde | Jun 14 2021 00:20:13 | -- | -- | RUNNING |
| lr0001_m05 | 367723ea-5d95-4263-ad9f-30d69e2c3dee | Jun 14 2021 00:19:45 | -- | -- | RUNNING |
| lr0001_m01 | f3a4146d-9b5a-43f2-97a4-102f360bf6e6 | Jun 14 2021 00:19:47 | -- | -- | RUNNING |
| lr0001_m005 | e67eda8d-0088-450e-9dd0-b02335162080 | Jun 14 2021 00:20:15 | -- | -- | RUNNING |

Figure 70. AWS Batch のジョブを確認

EC2 インスタンスを確認するために **Instances** ページを開く。Figure 71 のように、g4dn.xlarge の 9 インスタンスが Batch のジョブによって起動されている。



| Name | Instance ID | Instance state | Instance type | Status check | Alarm status | Availability Zone | Public IPv4 DNS |
|------|---------------------|----------------|---------------|-------------------|--------------|-------------------|-----------------|
| - | I-0ba247386a4a863b1 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-52-196-156- |
| - | I-0eb5e9cf4c680cdb7 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-18-183-239- |
| - | I-01a568c61751d48f8 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-13-114-41-7 |
| - | I-0965f3df949d6d64d | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-52-69-53-10 |
| - | I-071424513eff9b111 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-52-68-193-1 |
| - | I-0b3b7b6b58728558b | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-3-113-30-84 |
| - | I-04548920d31a2e134 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-3-112-221-2 |
| - | I-00e1428460a61d067 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-3-112-72-23 |
| - | I-025a30bf863674374 | Running | g4dn.xlarge | 2/2 checks passed | No alarms | ap-northeast-1a | ec2-18-183-170- |

Figure 71. AWS Batch の EC2 インスタンスを確認

ジョブのステータスを確認する (約 10-15 分程度)。ジョブのステータスが **SUCCEEDED** の 9 インスタンスが **Compute environment** の **Desired vCPUs** が 0 であることを確認。EC2 インスタンスが g4dn であることも確認。

AWS Batch のジョブが EC2 インスタンスによって実行されていることを確認。AWS Batch のジョブが 10 個実行されている (90 分程度) であることを確認。AWS Batch のジョブが 10 個実行されている (90 分程度) であることを確認。

run_sweep.ipynb を実行し [5] を実行する。

1 # [5]

```

2 import pandas as pd
3 import numpy as np
4 import io
5 from matplotlib import pyplot as plt
6
7 # [6]
8 def read_table_from_s3(bucket_name, key, profile_name=None):
9     if profile_name is None:
10         session = boto3.Session()
11     else:
12         session = boto3.Session(profile_name=profile_name)
13     s3 = session.resource("s3")
14     bucket = s3.Bucket(bucket_name)
15
16     obj = bucket.Object(key).get().get("Body")
17     df = pd.read_csv(obj)
18
19     return df
20
21 # [7]
22 grid = np.zeros((3,3))
23 for (i, lr) in enumerate([0.1, 0.01, 0.001]):
24     for (j, m) in enumerate([0.5, 0.1, 0.05]):
25         key = f"metrics_lr{lr:0.4f}_m{m:0.4f}.csv"
26         df = read_table_from_s3("simplebatch-bucket43879c71-mbqaltx441fu", key)
27         grid[i,j] = df["val_accuracy"].max()
28
29 # [8]
30 fig, ax = plt.subplots(figsize=(6,6))
31 ax.set_aspect('equal')
32
33 c = ax.pcolor(grid, edgecolors='w', linewidths=2)
34
35 for i in range(3):
36     for j in range(3):
37         text = ax.text(j+0.5, i+0.5, f"{grid[i, j]:0.1f}",
38                        ha="center", va="center", color="w")

```

Figure 72

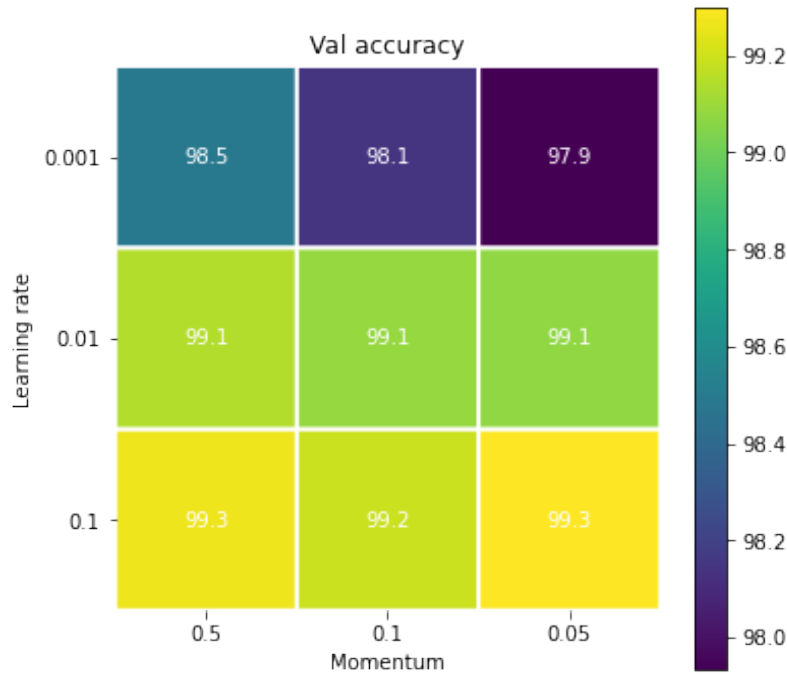


Figure 72. Validation accuracy

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.



Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

9.10. Validation

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.



Figure 73. ECR Docker image

다음은 AWS CLI를 사용하여 ECR 리포지토리의 이미지를 삭제하는 명령어입니다 (XXXX는 ECR 리포지토리 이름입니다).

```
$ aws ecr batch-delete-image --repository-name XXXX --image-ids imageTag=latest
```

이제 이미지를 삭제하고 다음 명령어를 실행합니다.

```
$ cdk destroy
```

[sec:batch_development_and_debug] === 배치 개발 및 디버그

이 섹션에서는 AWS Batch를 사용하여 GPU를 활용한 병렬 컴퓨팅을 설정하고 실행하는 방법을 알아보겠습니다.

이 섹션에서는 GPU를 지원하는 EC2 인스턴스를 사용하여 Jupyter Notebook을 실행하고, 이를 Docker로 패키징하여 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을 알아보겠습니다. Figure 74는 이 프로세스의 개요를 보여줍니다. Chapter 6에서는 GPU를 지원하는 EC2 인스턴스를 설정하는 방법을, Jupyter Notebook을 실행하는 방법을, Docker로 패키징하는 방법을, AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을, Chapter 9에서는 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을, Chapter 8에서는 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을 알아보겠습니다.

이 섹션에서는 MNIST 데이터를 사용하여 Jupyter Notebook을 실행하고, 이를 Docker로 패키징하여 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을 알아보겠습니다.

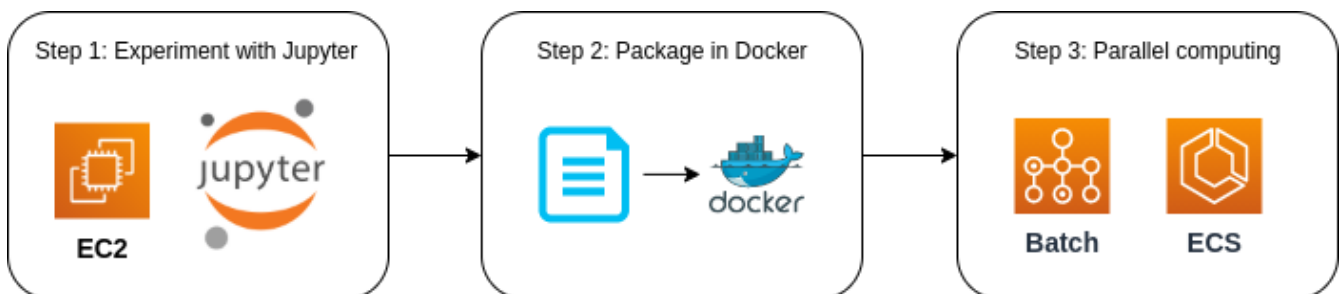


Figure 74. 배치 개발 및 디버그

9.11. 배치

이 섹션에서는 배치 개발 및 디버그를 위한 환경을 설정하는 방법을 알아보겠습니다.

この章では、GPU を利用した EC2 インスタンスで MNIST の認識を実行する方法を説明します (Chapter 6)。

この章では、Docker を利用して ECS クラスターでこのアプリケーションを実行する方法を説明します (Chapter 7)。
また、このアプリケーションを Lambda で実行する方法を説明します (Chapter 8)。

この章では、Chapter 9 で説明した AWS Batch を利用して、このアプリケーションを実行する方法を説明します。

この章では、このアプリケーションを実行するために SNS を利用する方法を説明します。

Chapter 10. Web

EC2, ECS, Fargate, Batch

10.1. — Twitter

Twitter, Facebook, YouTube

HTTP Twitter Figure 75

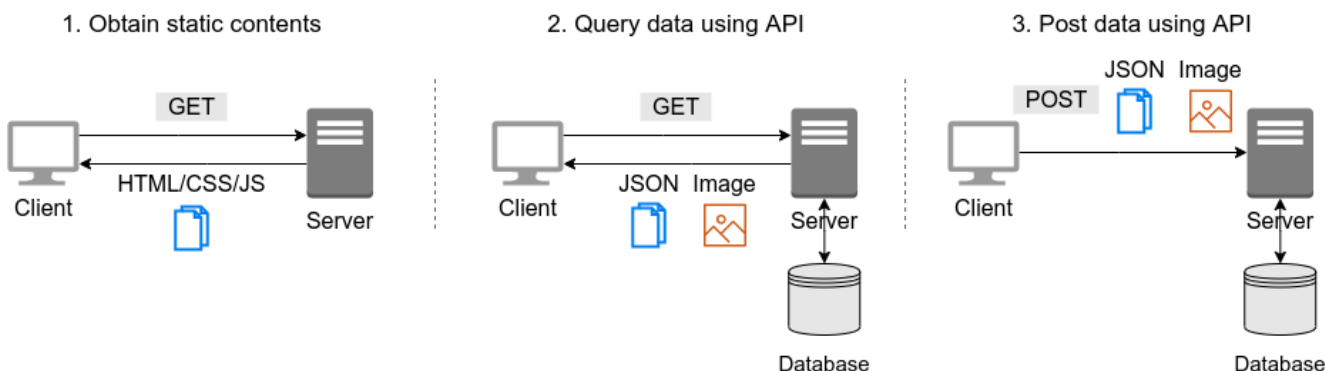


Figure 75. Web

HTTP (Hypertext Transfer Protocol) HTTPS (HTTPS (Hypertext Transfer Protocol Secure)) HTML (Hypertext Markup Language) CSS (Cascading Style Sheets) JavaScript (JS) Twitter API (Application Programming Interface) JSON (JavaScript Object Notation) HTML API

10.3. Twitter API

[Twitter API](#)
[Twitter API](#)
[Twitter Developer Documentation](#)
[Table 7](#)

Table 7. Twitter API

| メソッド | 説明 |
|-----------------------------|-------------------|
| GET statuses/home_timeline | 自分のタイムライン |
| GET statuses/show/:id | :id のツイートの詳細 |
| GET search | 検索結果 |
| POST statuses/update | ツイートを更新 |
| POST media/upload | メディアをアップロード |
| POST statuses/destroy/:id | :id のツイートを削除 |
| POST statuses/retweet/:id | :id のツイートをリツイート |
| POST statuses/unretweet/:id | :id のツイートをリツイート解除 |
| POST favorites/create | ツイートをブックマーク |
| POST favorites/destroy | ツイートをブックマーク解除 |

API Twitter

```

GET statuses/home_timeline
API
JSON
id, text, user, coordinates,
entities
id ID text user URL
coordinates
entities
(GET statuses/home_timeline)
GET statuses/show/:id :id

```

```

GET search API
API GET statuses/home_timeline
JSON

```

```

POST statuses/update
POST statuses/update
ID
POST media/upload
POST statuses/destroy/:id

```

```

POST statuses/retweet/:id
POST statuses/unretweet/:id
API
POST favorites/create
POST favorites/destroy

```

| Platform | API |
|----------|-----|
| Twitter | API |

Chapter 11. Serverless architecture

サーバーレスアーキテクチャ (Serverless architecture) とは、サーバーレスコンピューティング (Serverless computing) を利用して、アプリケーションを構築・実行するアーキテクチャである。AWS が2014年に発表した [Lambda](#) は、サーバーレスコンピューティングの代表的なサービスである。Google の [Cloud Functions](#)、Microsoft の [Azure Functions](#) も、サーバーレスコンピューティングのサービスである。

Serverless は、サーバーレスコンピューティングを指す用語である。サーバーレスコンピューティングは、"serverful" と対比して用いられる。

11.1. Serverful アーキテクチャ (対比)

サーバーレスアーキテクチャの対比として、Figure 77 に、サーバーレスアーキテクチャとサーバーフルアーキテクチャの対比を示す。API は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。AWS の EC2 は、サーバーフルアーキテクチャの代表的なサービスである。

サーバーレスアーキテクチャは、サーバーレスコンピューティングを利用する。Load balancing は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。Load balancer は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。

サーバーレスアーキテクチャは、サーバーレスコンピューティングを利用する。scale-out は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。scale-in は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。API は、サーバーレスアーキテクチャとサーバーフルアーキテクチャの両方で利用される。

サーバーレスアーキテクチャのAPIは、サーバーレスコンピューティングを利用する。AWS Lambda は、3000個のインスタンスを512MBのメモリで実行できる。AWS Lambda は、1000個のインスタンスを128MBのメモリで実行できる。サーバーレスアーキテクチャは、サーバーレスコンピューティングを利用する。(サーバーレスコンピューティングは、サーバーレスコンピューティングである。)

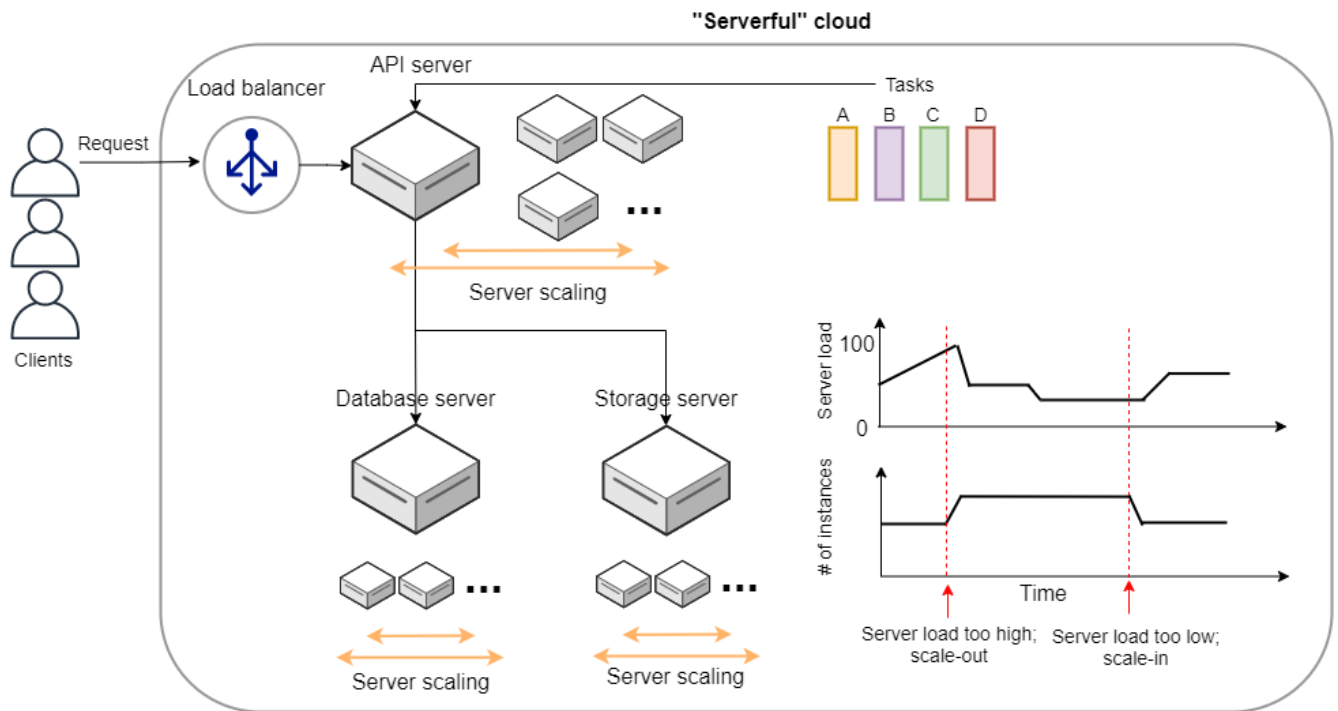


Figure 77. Serverful architecture

11.2. Serverless architecture

Section 11.1 Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest.

Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest. This model allows for a more flexible and cost-effective way to build and run applications.

Compared to serverful architecture, serverless architecture has several advantages. First, it is more flexible, as the user can scale the application up or down as needed. Second, it is more cost-effective, as the user only pays for the resources they use. Third, it is easier to manage, as the provider handles the infrastructure and scaling.

Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest. This model allows for a more flexible and cost-effective way to build and run applications. It is particularly well-suited for applications with variable workloads or unpredictable traffic.

Figure 78. Serverless architecture

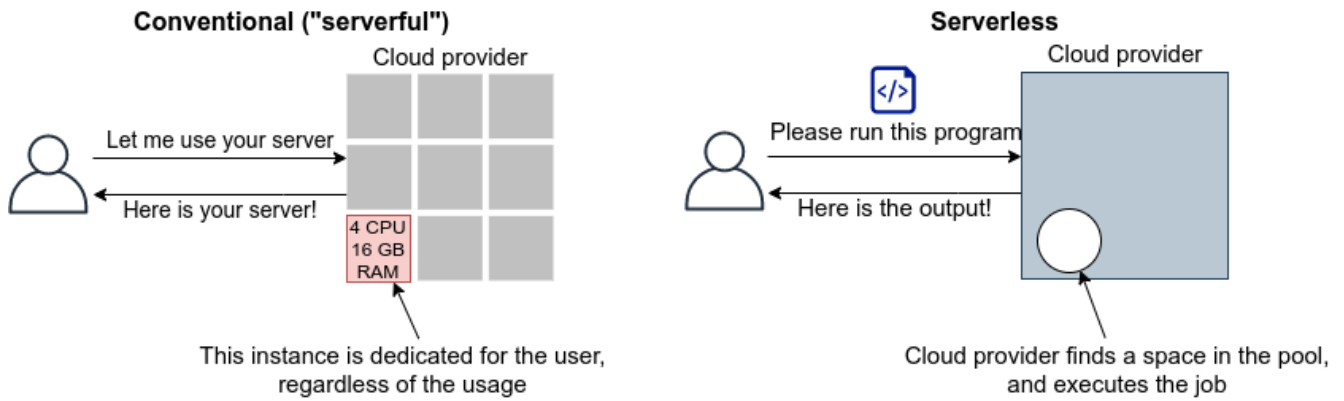


Figure 78. Serverless

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the scaling, availability, and maintenance of the underlying infrastructure. This model allows developers to build and deploy applications without the need to manage physical servers or virtual machines. The cloud provider handles the infrastructure, and the user only pays for the compute time used by their application.

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the scaling, availability, and maintenance of the underlying infrastructure. This model allows developers to build and deploy applications without the need to manage physical servers or virtual machines. The cloud provider handles the infrastructure, and the user only pays for the compute time used by their application.



Serverless architecture is a cloud service model in which the cloud provider dynamically manages the scaling, availability, and maintenance of the underlying infrastructure. This model allows developers to build and deploy applications without the need to manage physical servers or virtual machines. The cloud provider handles the infrastructure, and the user only pays for the compute time used by their application.

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the scaling, availability, and maintenance of the underlying infrastructure. This model allows developers to build and deploy applications without the need to manage physical servers or virtual machines. The cloud provider handles the infrastructure, and the user only pays for the compute time used by their application.

11.3. Serverless Architecture

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the scaling, availability, and maintenance of the underlying infrastructure. This model allows developers to build and deploy applications without the need to manage physical servers or virtual machines. The cloud provider handles the infrastructure, and the user only pays for the compute time used by their application. AWS provides several serverless services, including **Lambda**, **S3**, and **DynamoDB**. (Figure 79) shows the icons for these services. For more information, see [Chapter 12](#).



Figure 79. Lambda, S3, DynamoDB

11.3.1. Lambda

AWS provides several serverless services, including **Lambda**. Lambda is a serverless compute service that lets you run code without provisioning or managing servers. You can run code in response to events and automatically scale. Lambda supports several programming languages, including Python, Node.js, and Ruby. For more information, see [Figure 80](#).

Lambda 関数 (Function) を invoke すると Lambda 関数は
 Lambda invoke 関数 (関数呼び出し) を呼び出し
 関数を実行する

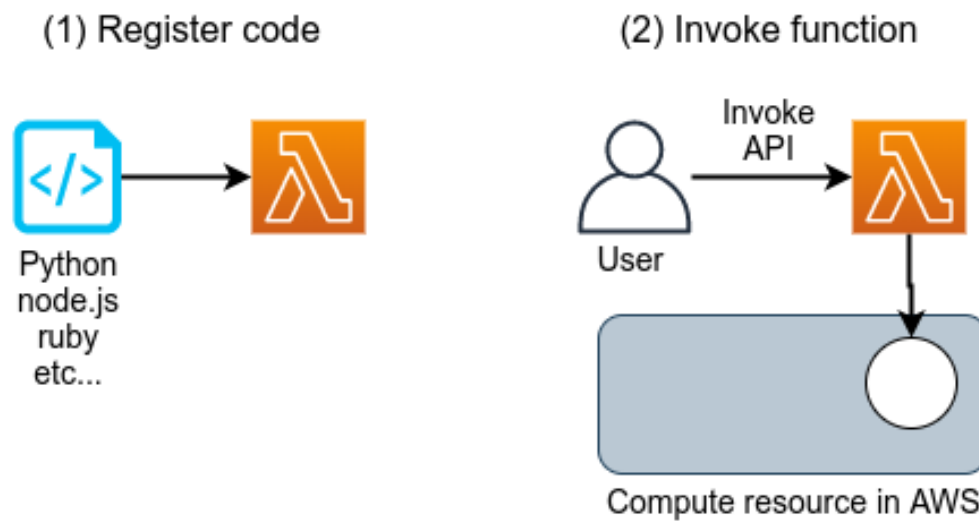


Figure 80. AWS Lambda

Lambda 関数 invoke すると AWS
 関数を実行する AWS 関数呼び出し関数
 Lambda 関数呼び出し関数
 FaaS (Function as a Service) 関数

Lambda 関数は 128MB から 10240MB のメモリ (RAM) を持つ CPU
 関数を実行する関数呼び出し関数 CPU 関数呼び出し関数 (RAM
 CPU 関数呼び出し関数 AWS 関数呼び出し関数) 関数 100 関数呼び出し関数 Table 8 Lambda
 関数呼び出し関数 (ap-north-east1 関数呼び出し関数)

Table 8. Lambda 関数

| Memory (MB) | Price per 100ms |
|-------------|-----------------|
| 128 | \$0.0000002083 |
| 512 | \$0.0000008333 |
| 1024 | \$0.0000016667 |
| 3008 | \$0.0000048958 |

関数呼び出し関数 関数呼び出し関数 \$0.2 関数 関数 128MB
 関数呼び出し関数 200 関数呼び出し関数 100 関数呼び出し関数 $0.0000002083 * 2 * 10^6 + 0.2 = \0.6 関数呼び出し関数
 関数呼び出し関数 200 関数呼び出し関数 100 関数呼び出し関数 \$0.6
 関数呼び出し関数 関数呼び出し関数 関数呼び出し関数 0 関数呼び出し関数
 関数呼び出し関数 関数呼び出し関数 関数呼び出し関数

Lambda 関数呼び出し関数 関数呼び出し関数
 関数呼び出し関数 関数呼び出し関数 関数呼び出し関数
 Lambda 関数呼び出し関数 関数呼び出し関数



11.3.2. Amazon S3

Amazon S3 is a simple storage service.

Amazon S3 (Simple Storage Service) is a scalable, durable, and secure storage service. It is managed by AWS and can be accessed via a variety of APIs, including the AWS CLI, the AWS SDKs, and the S3 console. S3 is designed to be highly available and durable, with data replicated across multiple Availability Zones. It is also designed to be secure, with data encrypted at rest and in transit. S3 is a pay-as-you-go service, meaning you only pay for the storage you use. (Figure 81)

Simple Storage Service (S3) is a scalable, durable, and secure storage service. It is managed by AWS and can be accessed via a variety of APIs, including the AWS CLI, the AWS SDKs, and the S3 console. S3 is designed to be highly available and durable, with data replicated across multiple Availability Zones. It is also designed to be secure, with data encrypted at rest and in transit. S3 is a pay-as-you-go service, meaning you only pay for the storage you use.

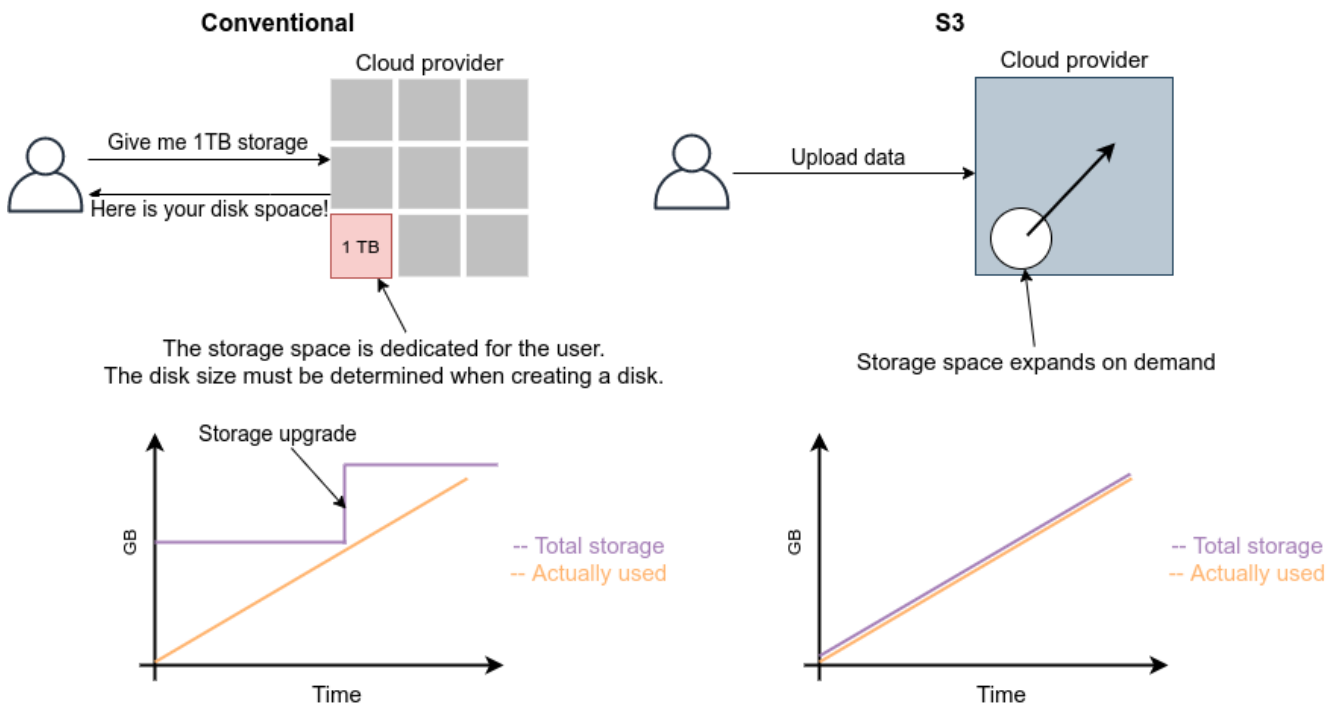


Figure 81. S3 storage model

S3 pricing is detailed in Table 9. For the `us-east-1` region, the pricing is as follows: Data storage (First 50TB) at \$0.023 per GB per month, PUT, COPY, POST, LIST requests (per 1,000 requests) at \$0.005, GET, SELECT, and all other requests (per 1,000 requests) at \$0.0004, and Data Transfer IN To Amazon S3 From Internet at \$0. For more details, see the "Amazon S3 pricing" page.

Table 9. S3 pricing

| Item | Price |
|--|--------------------------|
| Data storage (First 50TB) | \$0.023 per GB per month |
| PUT, COPY, POST, LIST requests (per 1,000 requests) | \$0.005 |
| GET, SELECT, and all other requests (per 1,000 requests) | \$0.0004 |
| Data Transfer IN To Amazon S3 From Internet | \$0 |

| 項目 | 料金 |
|--|---------------|
| Data Transfer OUT From Amazon S3 To Internet | \$0.09 per GB |

Amazon S3 のデータ転送料金は、**\$0.025 per GB** (データ転送先がインターネットの場合) で、**\$0.005 per GB** (データ転送先が Amazon S3 の他のリージョンの場合) で、**\$0.0004 per GB** (データ転送先が Amazon S3 の同じリージョンの場合) です。また、Amazon S3 からインターネットへデータを送信する際 (data-out) は、**\$0.09 per GB** の料金が適用されます。Amazon S3 にデータを送信する際 (data-in) は、**\$0.005 per GB** の料金が適用されます。また、Amazon S3 のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

11.3.3. 非関係データベース: DynamoDB

Amazon DynamoDB は、NoSQL データベースです。

Amazon Web Services (AWS) は、Amazon DynamoDB を提供しています。Amazon DynamoDB は、Amazon MySQL, PostgreSQL, MongoDB と比較して、高いパフォーマンスと可用性を提供します。また、Amazon DynamoDB は、Amazon S3 と連携して、データのバックアップと復元を簡単に実行できます。

Amazon DynamoDB は、CPU 使用率が高くなる可能性があります。Amazon DynamoDB の CPU 使用率を監視し、必要に応じて、Amazon S3 と連携して、データのバックアップと復元を実行してください。Amazon S3 のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

Amazon DynamoDB は、AWS のリージョンと一致する必要があります。Amazon DynamoDB は、Amazon S3 と連携して、データのバックアップと復元を実行できます。Amazon S3 のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

Amazon DynamoDB は、Amazon S3 と連携して、データのバックアップと復元を実行できます。Amazon S3 のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

Table 10. DynamoDB の料金

| 項目 | 料金 |
|---------------------|--|
| Write request units | \$1.25 per million write request units |
| Read request units | \$0.25 per million read request units |
| Data storage | \$0.25 per GB-month |

Amazon DynamoDB は、write request unit と read request unit を提供します。write request unit は、1 write request unit が 4kB のデータを 1 回書き込むことを意味します。read request unit は、1 read request unit が 1kB のデータを 1 回読み取ることを意味します。Amazon DynamoDB は、write request units が 100 単位で \$1.25、read request units が 100 単位で \$0.25 の料金を適用します。また、Amazon DynamoDB は、\$0.25 per GB の料金を適用します。Amazon DynamoDB は、Amazon S3 と連携して、データのバックアップと復元を実行できます。Amazon S3 のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

11.3.4. 非関係データベース: Amazon ElastiCache

Amazon ElastiCache は、Amazon Lambda, S3, DynamoDB と連携して、データのバックアップと復元を実行できます。Amazon ElastiCache のリージョンは、AWS のリージョンと一致する必要があります。AWS のリージョンは、AWS のリージョン一覧を参照してください。

- 

□□□□□□□□□□ NO □□□□□□□□

[illegible]

- Hellerstein et al., "Serverless Computing: One Step Forward, Two Steps Back" arXiv (2018)

Chapter 12. Hands-on #5: 関数型アーキテクチャ

関数型アーキテクチャの構築に必要となる AWS サービスとして、AWS Lambda、Amazon S3、Amazon DynamoDB を利用する。また、AWS Lambda のデプロイとテストを行う。

12.1. Lambda の構築

本書の Lambda の構築は、GitHub の [handson/serverless/lambda](https://github.com/hands-on/serverless/lambda) を参照する。

本書の Lambda の構築は、Figure 82 の STEP 1 のように AWS CDK を使って Python で Lambda の関数を定義し、STEP 2 のように Invoke API を使って Lambda をテストする。

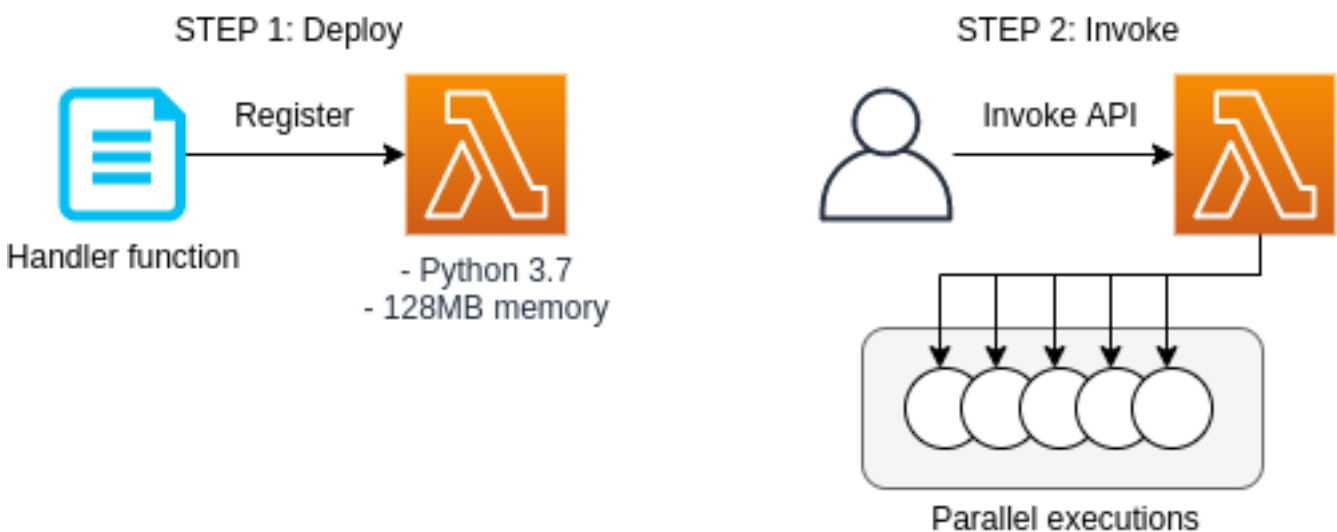


Figure 82. Lambda の構築



本書の Lambda の構築は、AWS Lambda の [ドキュメント](#) を参照する。

`app.py` の Lambda 関数のコードは以下の通り。

```
1 ①
2 FUNC = """
3 import time
4 from random import choice, randint
5 def handler(event, context):
6     time.sleep(randint(2,5))
7     sushi = ["salmon", "tuna", "squid"]
8     message = "Welcome to Cloud Sushi. Your order is " + choice(sushi)
9     print(message)
10    return message
11 """
12
13 class SimpleLambda(Stack):
14
15     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
16         super().__init__(scope, construct_id, **kwargs)
```

```

17
18     ②
19     handler = _lambda.Function(
20         self, 'LambdaHandler',
21         runtime=_lambda.Runtime.PYTHON_3_7,
22         code=_lambda.Code.from_inline(FUNC),
23         handler="index.handler",
24         memory_size=128,
25         timeout=cdk.Duration.seconds(10),
26         dead_letter_queue_enabled=True,
27     )

```

① 以下の Lambda 関数を実行すると、ランダムに生成された2-5桁の文字列が["salmon", "tuna", "squid"]の中からランダムに選択され、"Welcome to Cloud Sushi. Your order is XXXX" (XXXX はランダムな文字列)が返されます。

② 以下の Lambda 関数 <1> は、以下のパラメータで定義されています。

- `runtime=_lambda.Runtime.PYTHON_3_7`: Python3.7 を実行環境として指定します。Python3.7 の他に Node.js, Java, Ruby, Go も指定できます。
- `code=_lambda.Code.from_inline(FUNC)`: 関数本体を inline code で指定します。FUNC は、`FUNC=...` のように関数本体を指定します。
- `handler="index.handler"`: 関数本体のハンドラを指定します。handler は、`handler` のように指定します。
- `memory_size=128`: メモリサイズを 128MB に指定します。
- `timeout=cdk.Duration.seconds(10)`: タイムアウトを 10 秒に指定します。
- `dead_letter_queue_enabled=True`: デッドレターキューを有効にします。

以下は、Lambda 関数をデプロイするコマンドです。

12.1.1. デプロイ

以下は、Lambda 関数をデプロイするコマンドです。 (# はコメント) デプロイする前に、1, 2 のコマンドを実行する必要があります (Section 15.3)。

```

# 環境の準備
$ cd handson/serverless/lambda

# venv の作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# デプロイ
$ cdk deploy

```

以下は、Lambda 関数のデプロイ結果を示す Figure 83 のスクリーンショットです。SimpleLambda.FunctionName = XXXX と XXXX がランダムに生成された文字列です。

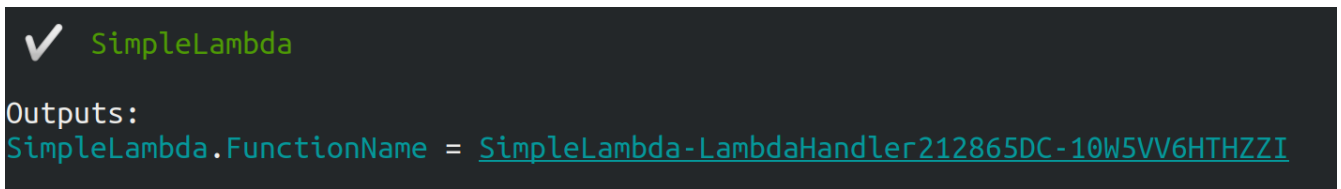


Figure 83. CDK の実行結果

AWS のコンソールで Lambda 関数を確認する。Figure 84 のように Lambda 関数が作成されている。

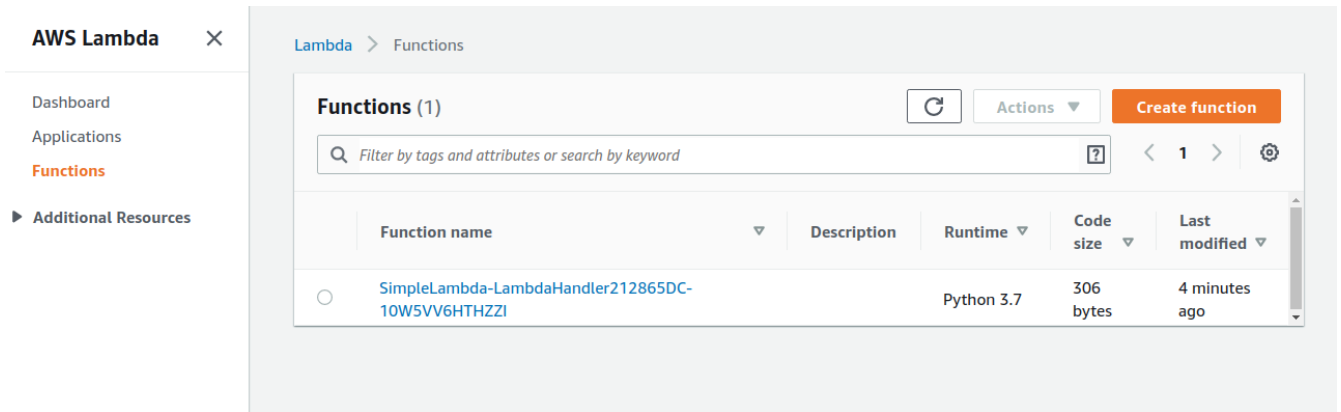


Figure 84. Lambda 関数の一覧

Figure 85 のように SimpleLambda 関数のコードを確認する。Python のコードが表示されている。

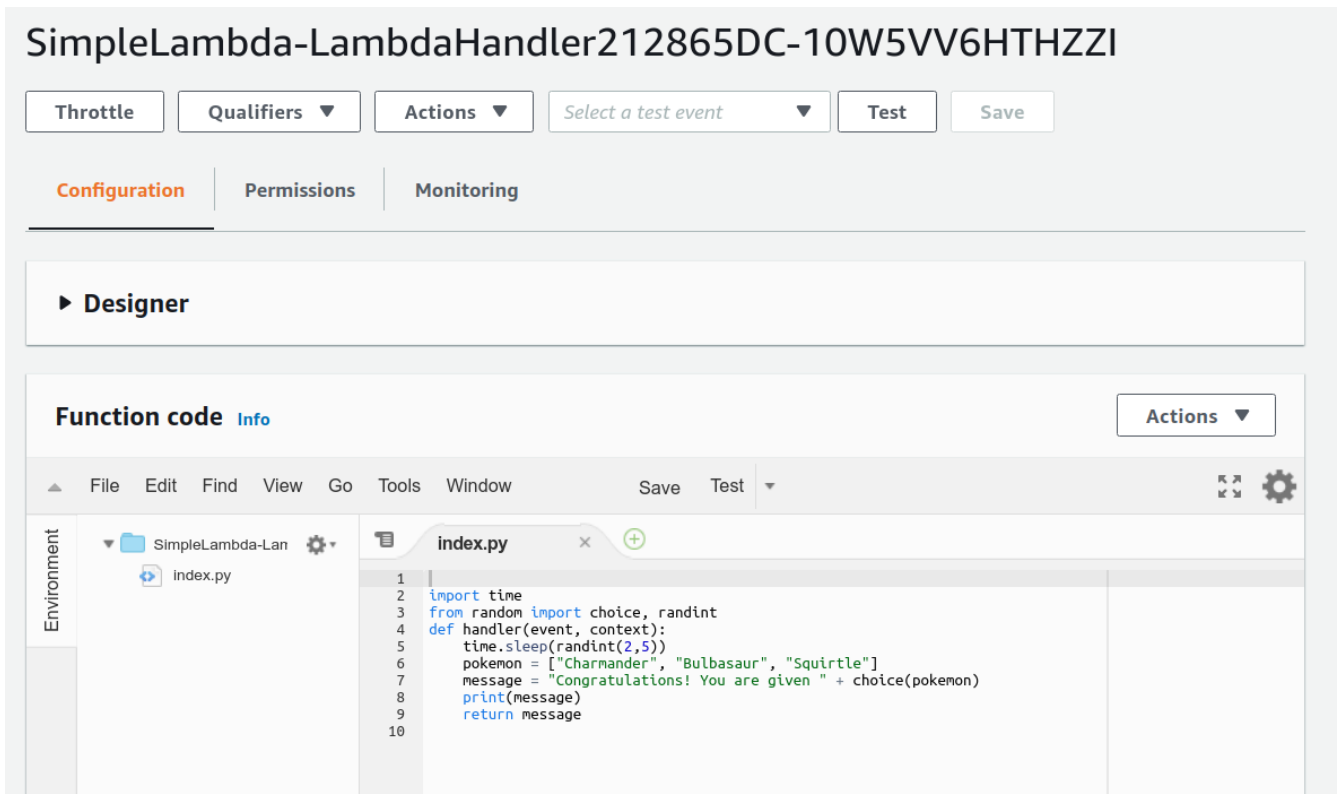


Figure 85. Lambda 関数のコード



Lambda 関数のコードを確認する。Figure 85 のように Python のコードが表示されている。CDK を使って Lambda 関数を作成する。

12.1.2. Lambda 実行

ここでは Lambda を `invoke` して AWS の API を呼び出すコードを実行する。 [handson/serverless/lambda/invoke_one.py](#) を実行する。

Lambda を実行するコードは `SimpleLambda.FunctionName = XXXX` として `XXXX` を指定する。

```
$ python invoke_one.py XXXX
```

実行すると `"Welcome to Cloud Sushi. Your order is salmon"` と表示される。

ここでは Lambda を 100 回実行するコード [handson/serverless/lambda/invoke_many.py](#) を実行する。

実行するコードは `XXXX` として `100` を指定する。

```
$ python invoke_many.py XXXX 100
```

実行すると

```
.....
Submitted 100 tasks to Lambda!
```

100 回実行した結果は [Figure 85](#) のように表示される。 [Figure 86](#) は "Monitoring" の画面である。

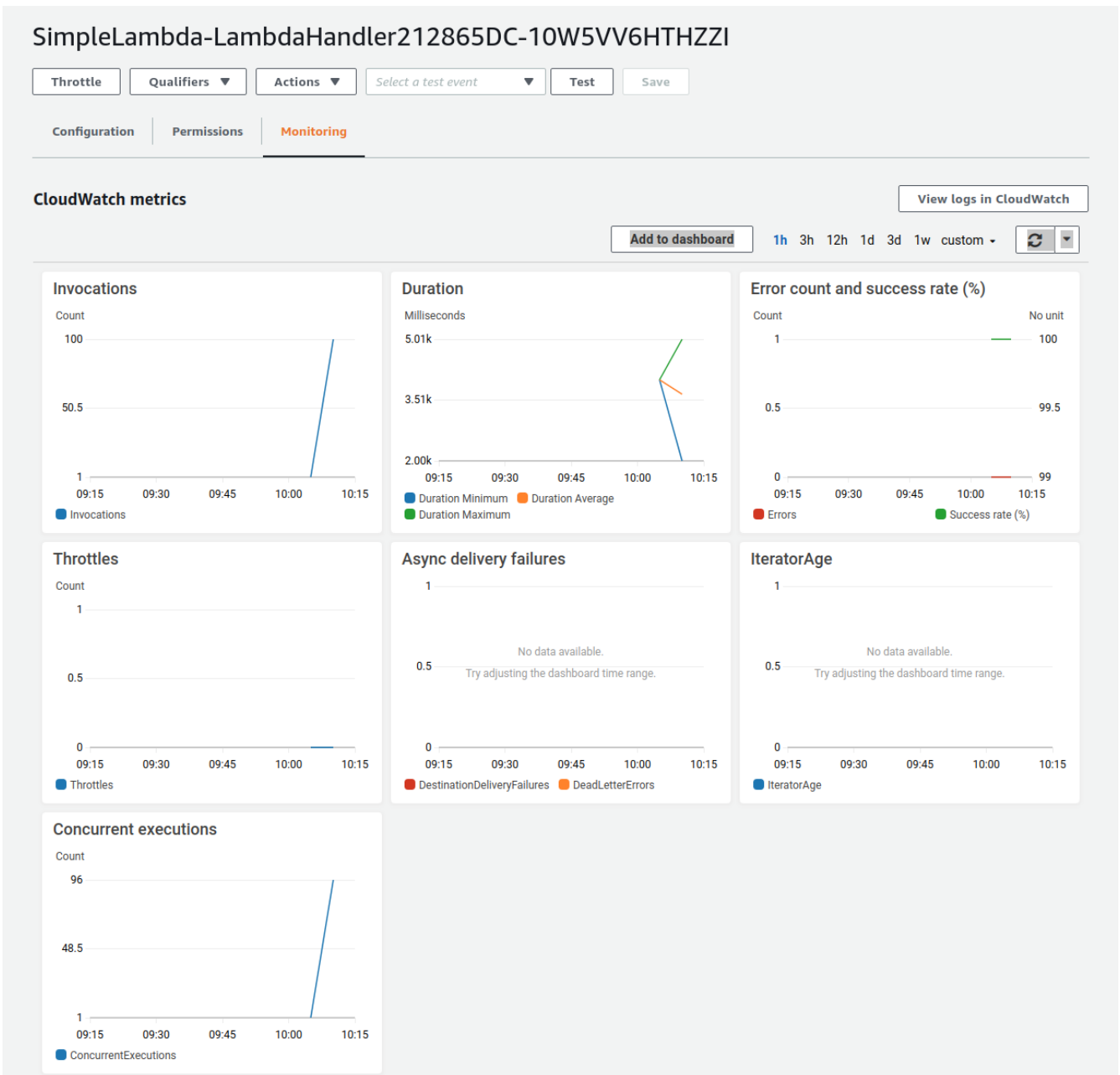


Figure 86. Lambda �� - ��



Figure 86 ��

Figure 86 �� "Invocations" �� 100 �� "Concurrent executions" �� 96 �� 96 �� 100 ��)

�� Lambda ��

serverful ��



1000 �� Lambda �� Lambda ��

12.1.3. 破壊デモ

デモ環境を破壊するためのコマンドを実行します。

```
$ cdk destroy
```

12.2. DynamoDB デモ

この章では DynamoDB をデモするために、AWS CDK を使用して DynamoDB を構築し、API を作成します。GitHub の </handson/serverless/dynamodb> フォルダを参照してください。

この章は Figure 87 の STEP 1 の AWS CDK を使用して DynamoDB を構築し、STEP 2 の API を作成する手順を示しています。



Figure 87. DynamoDB デモ



この章では AWS DynamoDB をデモするために、AWS CDK を使用して DynamoDB を構築し、API を作成します。

[handson/serverless/dynamodb/app.py](/handson/serverless/dynamodb/app.py) フォルダを参照してください。

```
1 class SimpleDynamoDb(Stack):
2     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
3         super().__init__(scope, construct_id, **kwargs)
4
5         table = ddb.Table(
6             self, "SimpleTable",
7             ①
8             partition_key=ddb.Attribute(
9                 name="item_id",
10                type=ddb.AttributeType.STRING
11            ),
12            ②
13            billing_mode=ddb.BillingMode.PAY_PER_REQUEST,
14            ③
15            removal_policy=cdk.RemovalPolicy.DESTROY
16        )
```

DynamoDB Partition key Partition key

- ① **partition_key**: DynamoDB Partition key Partition key () ID Partition key (: Sort Key "Core Components of Amazon DynamoDB") Partition key **item_id**
- ② **billing_mode**: `ddb.BillingMode.PAY_PER_REQUEST` **On-demand Capacity Mode** **PROVISIONED**
- ③ **removal_policy**: CloudFormation DynamoDB **DESTROY**

12.2.1.

(#) (Section 15.3)

```
#  
$ cd handson/serverless/dynamodb  
  
# venv  
$ python3 -m venv .env  
$ source .env/bin/activate  
$ pip install -r requirements.txt  
  
#  
$ cdk deploy
```

Figure 88 `SimpleDynamoDb.TableName = XXXX` XXXX

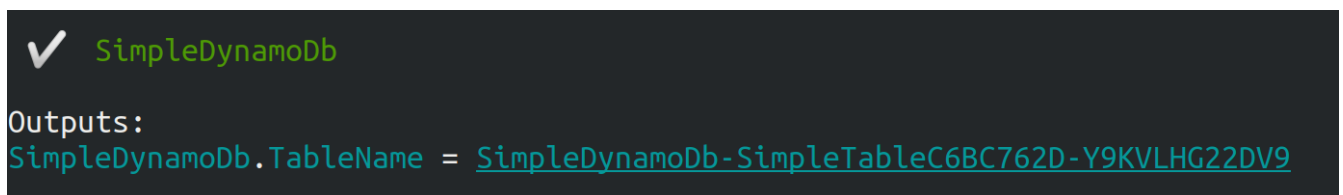


Figure 88. CDK

AWS DynamoDB "Tables" Figure 89

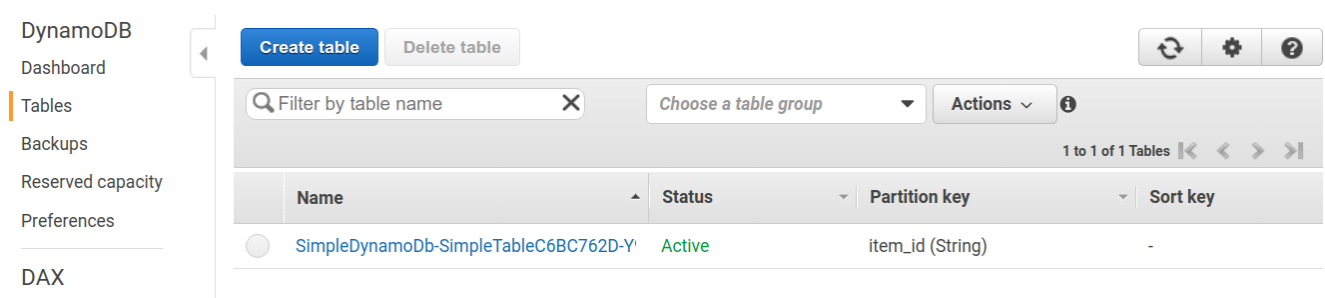


Figure 89. DynamoDB ()

Figure 90 shows the AWS Management Console interface for a DynamoDB table named SimpleDynamoDb-SimpleTableC6BC762D-Y9KVLHG22DV9. The 'Items' tab is selected, showing a search bar and a 'Create item' button. The table's primary key is 'item_id'.

Figure 90. DynamoDB console (Screenshot)

12.2.2. boto3

Section 12.2.1 describes how to use the boto3 library to interact with DynamoDB. Python's boto3 library is used to interact with DynamoDB.

The following code snippet, simple_write.py, demonstrates how to write an item to a DynamoDB table using boto3.

```
1 import boto3
2 from uuid import uuid4
3 ddb = boto3.resource('dynamodb')
4
5 def write_item(table_name):
6     table = ddb.Table(table_name)
7     table.put_item(
8         Item={
9             'item_id': str(uuid4()),
10            'first_name': 'John',
11            'last_name': 'Doe',
12            'age': 25,
13        }
14    )
```

The boto3 library is used to interact with DynamoDB. The write_item() function is defined to write an item to a DynamoDB table. The function takes the table name as an argument. The table is created using the boto3 resource object. The put_item() method is used to write an item to the table. The item is a dictionary with the following attributes: item_id, first_name, last_name, and age. The item_id attribute is a UUID4 value. The first_name attribute is 'John'. The last_name attribute is 'Doe'. The age attribute is 25. The item_id attribute is the Partition key.

The simple_write.py file is executed using the following command: python simple_write.py XXXX. The XXXX value is the name of the DynamoDB table (SimpleDynamoDb-SimpleTableC6BC762D-Y9KVLHG22DV9).

```
$ python simple_write.py XXXX
```


Figure 90 AWS DynamoDB console screenshot showing the 'Simple DynamoDb-SimpleTableC6BC762D-1K9Z83Q7426FD' table. The 'Items' tab is selected, and a single item is displayed with the following attributes: item_id (2b4874e5-aa5e-4e18-b303-72b61839da6f), age (25), first_name (John), and last_name (Doe).

Figure 91. DynamoDB console screenshot showing the 'Simple DynamoDb-SimpleTableC6BC762D-1K9Z83Q7426FD' table. The 'Items' tab is selected, and a single item is displayed with the following attributes: item_id (2b4874e5-aa5e-4e18-b303-72b61839da6f), age (25), first_name (John), and last_name (Doe).

boto3 Python library is used to interact with AWS services. The `simple_read.py` script is used to read data from the DynamoDB table.

```
1 import boto3
2 ddb = boto3.resource('dynamodb')
3
4 def scan_table(table_name):
5     table = ddb.Table(table_name)
6     items = table.scan().get("Items")
7     print(items)
```

The `table.scan().get("Items")` method is used to scan the table and retrieve all items.

The `XXXX` placeholder is used to specify the table name.

```
$ python simple_read.py XXXX
```

The output of the script is a list of items.

12.2.3. Batch Write

DynamoDB provides a batch write API to write multiple items in a single call.

The `batch_rw.py` script is used to write data to the DynamoDB table.

The `XXXX` placeholder is used to specify the table name.

```
$ python batch_rw.py XXXX write 1000
```

The output of the script is a confirmation message indicating that 1000 items were written to the table.

python batch_rw.py XXXX search_under_age 2

age

python batch_rw.py XXXX search_under_age 2

age

```
$ python batch_rw.py XXXX search_under_age 2
```

python batch_rw.py XXXX search_under_age 2

12.2.4. Python

DynamoDB Python

Python

```
$ cdk destroy
```

12.3. S3

S3 Python GitHub [hands-on/serverless/s3](https://github.com/hands-on/serverless/s3)

Figure 92 S3 Python STEP 1 AWS CDK S3 (Bucket) STEP 2

STEP 1: Deploy



S3

STEP 2: Manipulate data



Read / Write



Figure 92. S3



S3

app.py

```
1 class SimpleS3(Stack):
2     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
3         super().__init__(scope, construct_id, **kwargs)
4
5         # S3 bucket to store data
6         bucket = s3.Bucket(
7             self, "bucket",
8             removal_policy=cdk.RemovalPolicy.DESTROY,
9             auto_delete_objects=True,
```

`s3.Bucket()` クラスを使用して、AWS S3 バケットを作成します。バケットの名前は `bucket_name` 変数に指定します (i.e. AWS S3 バケットの名前)。バケットの作成は、CloudFormation テンプレートを使用して行われます。



CloudFormation テンプレートを使用して、S3 バケットを作成します。バケットの作成は、`cdk` コマンドを使用して行われます。バケットの作成は、`destroy` コマンドを使用して行われます。バケットの作成は、`removal_policy=cdk.RemovalPolicy.DESTROY`、`auto_delete_objects=True` を指定します。

12.3.1. テンプレート

テンプレートは、AWS CloudFormation テンプレートを使用して作成されます (# テンプレート) (Section 15.3)

```
# テンプレートを作成
$ cd handson/serverless/s3

# venv を作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# テンプレート
$ cdk deploy
```

Figure 93 テンプレートを作成する。テンプレートは `SimpleS3.BucketName = XXXX` を指定します (テンプレートは、AWS CloudFormation テンプレートを使用して作成されます)



SimpleS3

Outputs:

SimpleS3.BucketName = [simples3-bucket43879c71-j96v4b58l3jd](#)

Figure 93. テンプレート

12.3.2. テンプレート

テンプレートは、AWS CloudFormation テンプレートを使用して作成されます

テンプレートは、`tmp.txt` ファイルを使用して作成されます

```
$ echo "Hello world!" >> tmp.txt
```

テンプレートは、`simple_s3.py` を使用して `boto3` を使用して S3 バケットを作成します。テンプレートは、`simple_s3.py` を使用して `tmp.txt` ファイルを使用して作成されます。テンプレートは、`XXXX` を指定します。

```
$ python simple_s3.py XXXX upload tmp.txt
```

simple_s3.py

```
1 def upload_file(bucket_name, filename, key=None):
2     bucket = s3.Bucket(bucket_name)
3
4     if key is None:
5         key = os.path.basename(filename)
6
7     bucket.upload_file(filename, key)
```

bucket = s3.Bucket(bucket_name) Bucket() upload_file()

S3 Key (Path) Key Object storage --key simple_s3.py Key

```
$ python simple_s3.py XXXX upload tmp.txt --key a/b/tmp.txt
```

a/b/tmp.txt Key

AWS S3 S3 simples3-bucket (Figure 94)

Amazon S3 > simples3-bucket43879c71-j96v4b58l3jd

simples3-bucket43879c71-j96v4b58l3jd

Objects | Properties | Permissions | Metrics | Management | Access Points

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Copy S3 URI Copy URL Download Open Delete Actions Create folder

Upload

| ☐ | Name | Type | Last modified | Size | Storage class |
|--------------------------|---------|--------|------------------------------------|--------|---------------|
| ☐ | a/ | Folder | - | - | - |
| ☐ | tmp.txt | txt | July 7, 2021, 09:58:21 (UTC+09:00) | 13.0 B | Standard |

Figure 94. S3

tmp.txt a/b/tmp.txt S3 Key "/"



このコードを実行すると、指定したバケットからファイルがダウンロードされます。

このコードを実行すると、指定したバケットからファイルがダウンロードされます。

`simple_s3.py`

このコードを実行すると、指定したバケットからファイルがダウンロードされます。

XXXX

```
$ python simple_s3.py XXXX download tmp.txt
```

`simple_s3.py` のコードは以下の通りです。

```
1 def download_file(bucket_name, key, filename=None):
2     bucket = s3.Bucket(bucket_name)
3
4     if filename is None:
5         filename = os.path.basename(key)
6
7     bucket.download_file(key, filename)
```

S3 バケットからファイル `download_file()` を呼び出して、指定した Key のファイルをダウンロードします。

12.3.3. クリーンアップ

このコードを実行すると、S3 バケットからファイルがダウンロードされます。このコードを実行すると、指定したバケットからファイルがダウンロードされます。

```
$ cdk destroy
```

Chapter 13. Hands-on #6: Bashoutter

この章では、SNSアプリケーション「Bashoutter」を構築します。

このアプリケーションは、SNSサービス（**Bashoutter**）を構築し、AWS Lambda、DynamoDB、S3、SNS を利用します。Figure 95 は、このアプリケーションのスクリーンショットです。

Bashoutter

API Endpoint URL
https://ig5181x0bd.execute-api.ap-northeast-1.amazonaws.com/prod/

Post your Haiku!

五
クラウドで

七
SNSを

五
つくったよ

Your name
詠み人知らず

POST

People's Haikus

REFRESH

dec2bd2bfcdd144208201d652c703d9922020/07/0701:25

柿食えば
鐘が鳴るなり
法隆寺

♡ 3 正岡子規

521b6a8ed4c34e6b98f4fc59cf5564962020/07/0701:26

閑けさや
岩にしみ入る
蟬の声

♡ 6 松尾芭蕉

9cc9f01987bd4575a1741f0cb24a94ab2020/07/0701:26

古池や
蛙飛び込む
水の音

♡ 1 松尾芭蕉

fe181b3ac913439aa1471c6a35cc33822020/07/0701:28

菜の花や
月は東に
日は西に

♡ 1 与謝蕪村

6cdad2516ccb4a77b297c88690d9f7c12020/07/0701:29

クラウドで
SNSを
つくったよ

♡ 20 詠み人知らず

Figure 95. Hands-on #6 SNS アプリケーション "Bashoutter"

13.1. 準備

この章のコードは、GitHub の [handson/bashoutter](https://github.com/handson/bashoutter) リポジトリにあります。

この章のコードは、Section 4.1 のコードと類似しています。



この章のコードは、AWS のコードと類似しています。

- 簡単な API を構築する **API Gateway** (API) を構築する API の URI を構築する **Lambda** を構築する
- 簡単な API を構築する (API) を構築する **Lambda** を構築する
- データベース (データベース) を構築する **(DynamoDB)** を構築する
- **Lambda** を構築する **DynamoDB** を構築する
- 静的コンテンツ (静的コンテンツ) を構築する **S3** を構築する **S3** を構築する **HTML/CSS/JS** を構築する

簡単な API を構築する (handson/bashoutter/app.py)

```

1 class Bashoutter(Stack):
2
3     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
4         super().__init__(scope, construct_id, **kwargs)
5
6         ①
7         # dynamoDB table to store haiku
8         table = ddb.Table(
9             self, "Bashoutter-Table",
10             partition_key=ddb.Attribute(
11                 name="item_id",
12                 type=ddb.AttributeType.STRING
13             ),
14             billing_mode=ddb.BillingMode.PAY_PER_REQUEST,
15             removal_policy=cdk.RemovalPolicy.DESTROY
16         )
17
18         ②
19         bucket = s3.Bucket(
20             self, "Bashoutter-Bucket",
21             website_index_document="index.html",
22             public_read_access=True,
23             removal_policy=cdk.RemovalPolicy.DESTROY
24         )
25
26         common_params = {
27             "runtime": _lambda.Runtime.PYTHON_3_7,
28             "environment": {
29                 "TABLE_NAME": table.table_name
30             }
31         }
32
33         ③
34         # define Lambda functions
35         get_haiku_lambda = _lambda.Function(
36             self, "GetHaiku",
37             code=_lambda.Code.from_asset("api"),
38             handler="api.get_haiku",
39             memory_size=512,

```



```

40         **common_params,
41     )
42     post_haiku_lambda = _lambda.Function(
43         self, "PostHaiku",
44         code=_lambda.Code.from_asset("api"),
45         handler="api.post_haiku",
46         **common_params,
47     )
48     patch_haiku_lambda = _lambda.Function(
49         self, "PatchHaiku",
50         code=_lambda.Code.from_asset("api"),
51         handler="api.patch_haiku",
52         **common_params,
53     )
54     delete_haiku_lambda = _lambda.Function(
55         self, "DeleteHaiku",
56         code=_lambda.Code.from_asset("api"),
57         handler="api.delete_haiku",
58         **common_params,
59     )
60
61     ④
62     # grant permissions
63     table.grant_read_data(get_haiku_lambda)
64     table.grant_read_write_data(post_haiku_lambda)
65     table.grant_read_write_data(patch_haiku_lambda)
66     table.grant_read_write_data(delete_haiku_lambda)
67
68     ⑤
69     # define API Gateway
70     api = apigw.RestApi(
71         self, "BashoutterApi",
72         default_cors_preflight_options=apigw.CorsOptions(
73             allow_origins=apigw.Cors.ALL_ORIGINS,
74             allow_methods=apigw.Cors.ALL_METHODS,
75         )
76     )
77
78     haiku = api.root.add_resource("haiku")
79     haiku.add_method(
80         "GET",
81         apigw.LambdaIntegration(get_haiku_lambda)
82     )
83     haiku.add_method(
84         "POST",
85         apigw.LambdaIntegration(post_haiku_lambda)
86     )
87
88     haiku_item_id = haiku.add_resource("{item_id}")
89     haiku_item_id.add_method(
90         "PATCH",

```

```

91         apigw.LambdaIntegration(patch_haiku_lambda)
92     )
93     haiku_item_id.add_method(
94         "DELETE",
95         apigw.LambdaIntegration(delete_haiku_lambda)
96     )

```

- ① データベースを DynamoDB に変更する
- ② データを S3 にアップロードする
- ③ データ API を Lambda で実装する (Python3.7 を使用する) [handson/bashoutter/api/api.py](#) を参照
- ④ <3> Lambda の実行環境を設定する
- ⑤ API Gateway を API と Lambda を連携させる

[illegible]

13.2.3. Public access mode ☐ S3 ☐

S3 □□□□□□□□□□□□□□□□

```
1 bucket = s3.Bucket(
2     self, "Bashoutter-Bucket",
3     website_index_document="index.html",
4     public_read_access=True,
5     removal_policy=cdk.RemovalPolicy.DESTROY
6 )
```

`public_read_access=True` オプションを指定すると、S3 オブジェクトは **Public access mode**（パブリックアクセスモード）で公開されます。このモードでは、オブジェクトの URL を直接ブラウザから表示したり、`http://XXXX.s3-website-ap-northeast-1.amazonaws.com/index.html` のような URL でアクセスできます。

□□□□□□□□□□□□□□□□ HTML/CSS/JS □□□□□□□□□□□□□□□□□□□□□□□□□□□□



public access mode → S3 → [CloudFront](#)
CloudFront → **Content Delivery Network (CDN)** → HTTPS → CloudFront → ["What is Amazon CloudFront?"](#)

CloudFront

- <https://github.com/aws-samples/aws-cdk-examples/tree/master/typescript/static-site>



桶 S3 桶名 AWS 桶名 URL 桶名 桶名 example.com

13.2.4. API □□□□□□

API 関数 (handler) GET /haiku API Lambda

```
1 get_haiku_lambda = _lambda.Function(  
2     self, "GetHaiku",  
3     code=_lambda.Code.from_asset("api"),  
4     handler="api.get_haiku",  
5     memory_size=512,  
6     **common_params  
7 )
```

```
memory_size=512 512MB
code=_lambda.Code.from_asset("api") (api/) handler="api.get_haiku"
api.py get_haiku()
```

get_haiku() (handson/bashoutter/api/api.py)

```

1 ddb = boto3.resource("dynamodb")
2 table = ddb.Table(os.environ["TABLE_NAME"])
3
4 def get_haiku(event, context):
5     """
6     handler for GET /haiku
7     """
8     try:
9         response = table.scan()
10
11         status_code = 200
12         resp = response.get("Items")
13     except Exception as e:
14         status_code = 500
15         resp = {"description": f"Internal server error. {str(e)}"}
16     return {
17         "statusCode": status_code,
18         "headers": HEADERS,
19         "body": json.dumps(resp, cls=DecimalEncoder)
20     }

```

```
response = table.scan() # DynamoDB returns up to 1MB of data (200 items by default, up to 500 if you specify a larger limit)
```

API API



```
GET /haiku response = table.scan()
```

DynamoDB □ scan() □□□□□□□□ 1MB □□□□□□□□□□□□ □□□□□□□□□□□□ 1MB
□□□□□□□□□□□□□□□□ scan() □□□□□□□□□□□□ □□□ boto3 □□□□□□□□□□ □□□

13.2.5. AWS 認証 (IAM)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
1 table.grant_read_data(get_haiku_lambda)
2 table.grant_read_write_data(post_haiku_lambda)
3 table.grant_read_write_data(patch_haiku_lambda)
4 table.grant_read_write_data(delete_haiku_lambda)
```

○○○○○○○○○○○○○○○○○○○○○○○○○○○○ AWS ○○ IAM (Identity and Access Management) ○○○○○○○○○○○ IAM
○○

CDK 的 `dynamodb.Table` 的 `grant_read_write_data()` 方法可以授予 Lambda 函数对 DynamoDB 表的读写权限。IAM 角色是 CDK 的 `s3.Bucket` 的 `grant_read_write()` 方法授予的。Chapter 9 的 AWS Batch 部分也有类似的操作。



このIAM ポリシーは、Amazon S3 のバケットに格納されているすべてのオブジェクトに対して、`read` アクションを実行する権限を付与しています。

このポリシーは、IAM ユーザーに適用されています。このユーザーは、`grant_read_data()` を使用して、S3 バケットからデータを取得することができます。

13.2.6. API Gateway

[illegible]


```

20 )
21
22 ④
23 haiku_item_id = haiku.add_resource("{item_id}")
24 ⑤
25 haiku_item_id.add_method(
26     "PATCH",
27     apigw.LambdaIntegration(patch_haiku_lambda)
28 )
29 haiku_item_id.add_method(
30     "DELETE",
31     apigw.LambdaIntegration(delete_haiku_lambda)
32 )

```

- ① `api = apigw.RestApi()` API Gateway API Gateway
- ② `api.root.add_resource()` API Gateway `/haiku` API Gateway
- ③ `add_method()` API Gateway `GET, POST` API Gateway `/haiku` API Gateway
- ④ `haiku.add_resource("{item_id}")` API Gateway `/haiku/{item_id}` API Gateway
- ⑤ `add_method()` API Gateway `PATCH, DELETE` API Gateway `/haiku/{item_id}` API Gateway

API Gateway API Gateway API Gateway Lambda API Gateway



API Gateway API Gateway URL API Gateway `api.example.com` API Gateway AWS API Gateway URL API Gateway DNS API Gateway



API Gateway API Gateway `default_cors_preflight_options=` API Gateway `Cross Origin Resource Sharing (CORS)` API Gateway Web API

13.3. API Gateway

API Gateway API Gateway API Gateway (# API Gateway) API Gateway (Section 15.3)

```

# API Gateway
$ cd intro-aws/handson/bashoutter

# venv API Gateway
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# API Gateway
$ cdk deploy

```

API Gateway API Gateway Figure 98 API Gateway Bashoutter.BashoutterApiEndpoint =

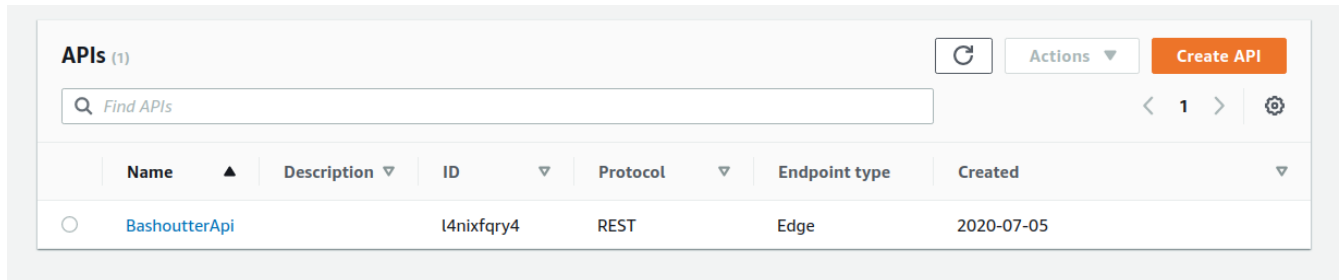
XXXX, Bashoutter.BucketUrl = YYYY

```
✓ Bashoutter

Outputs:
Bashoutter.BashoutterApiEndpointE9A7B94F = https://ig5181x0bd.execute-api.ap-northeast-1.amazonaws.com/pr
od/
Bashoutter.BucketUrl = bashoutter-bashoutterbucket6f28d9f3-zmi05o477p6r.s3-website-ap-northeast-1.amazona
ws.com
```

Figure 98. CDK

AWS API Gateway



| Name | Description | ID | Protocol | Endpoint type | Created |
|---------------|-------------|------------|----------|---------------|------------|
| BashoutterApi | | l4nixfqry4 | REST | Edge | 2020-07-05 |

Figure 99. API Gateway (1)

"BashoutterApi" API GET /haiku, POST /haiku

API Gateway "Method Request" Figure 100 Lambda Lambda

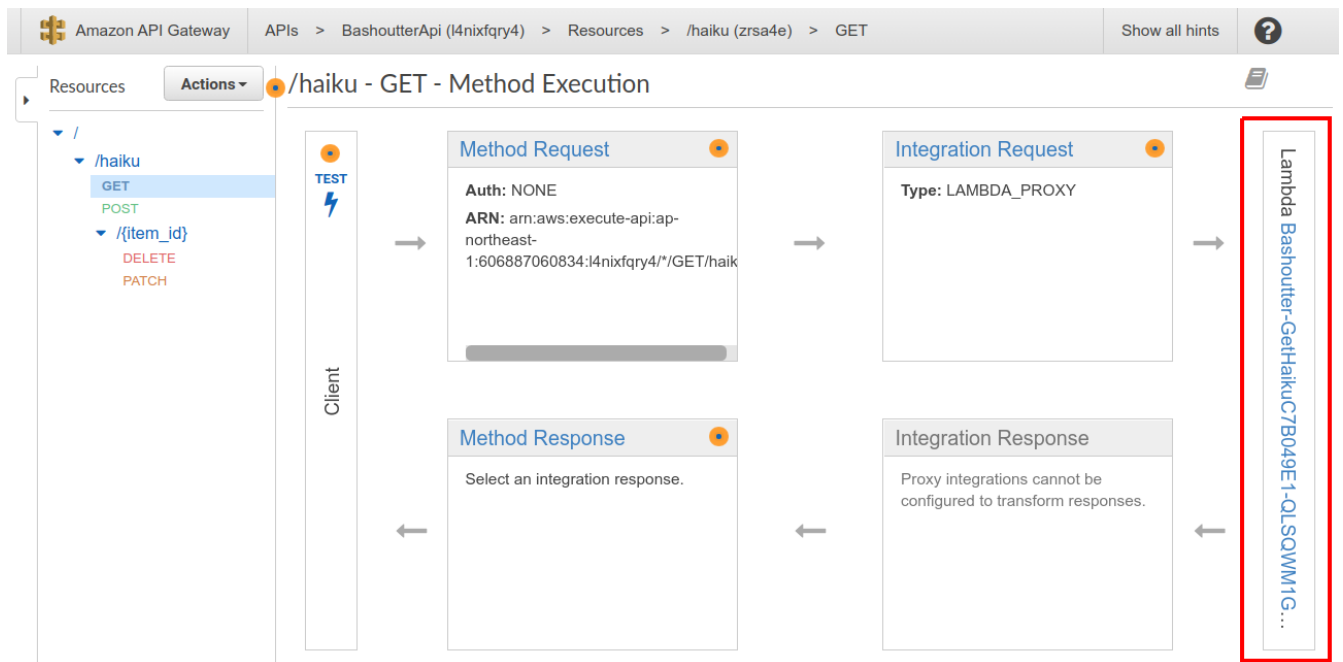


Figure 100. API Gateway (2)

S3 bashouter- (Figure 101)

API URL (Bashoutter.BashoutterApiEndpoint = XXXX XXXX
)

```
$ export ENDPOINT_URL=XXXX
```

GET /haiku API

```
$ http GET "${ENDPOINT_URL}/haiku"
```

([])

POST /haiku

```
$ http POST "${ENDPOINT_URL}/haiku" \  
username=" " \  
first=" " \  
second=" " \  
third=" "
```

```
HTTP/1.1 201 Created  
Connection: keep-alive  
Content-Length: 49  
Content-Type: application/json  
....  
{  
  "description": "Successfully added a new haiku"  
}
```

GET

```
$ http GET "${ENDPOINT_URL}/haiku"
```

```
HTTP/1.1 200 OK  
Connection: keep-alive  
Content-Length: 258  
Content-Type: application/json  
...  
[  
  {  
    "created_at": "2020-07-06T02:46:04+00:00",  
    "first": "  
    "item_id": "7e91c5e4d7ad47909e0ac14c8bbab05b",  
    "likes": 0.0,  
    "second": "  
  },  
]
```

```

        "third": "三",
        "username": "三三三"
    }
]

```

三三三三

三三 PATCH /haiku/{item_id} 三三三三三三三三三三三三三三三三 三三三三三三三三三三三三 item_id 三三三三三三 XXXX 三三三三三三三三三三三三

```

$ http PATCH "${ENDPOINT_URL}/haiku/XXXX"

```

{"description": "OK"} 三三三三三三三三三三三三 三三 GET 三三三三三三三三三三三三 (likes) 三三三三三三三三三三三三

```

$ http GET "${ENDPOINT_URL}/haiku"
...
[
  {
    ...
    "likes": 1.0,
    ...
  }
]

```

三三三 DELETE 三三三三三三三三三三三三三三三三三三 XXXX 三 item_id 三三三三三三三三三三三三三三三三三三

```

$ http DELETE "${ENDPOINT_URL}/haiku/XXXX"

```

三三 GET 三三三三三三三三三三三三三三三三三三 ([]) 三三三三三三三三三三三三三三三三三三

三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 API 三三三三三三三三三三三三三三三三三三

13.5. 三三 API 三三三三三三三三三三三三三三三三三三

三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 SNS 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 API 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三

[handson/bashoutter/client.py](#) 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 API 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三 POST /haiku 三三 API 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三

三三三三三三 API 三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三三

```

$ python client.py $ENDPOINT_URL post_many 300

```

API
1



```
$ python client.py $ENDPOINT_URL clear_database
```

13.6. Bashoutter GUI

API GUI (Figure 75)

CDK Public access mode S3 (HTML/CSS/JavaScript) AWS CLI S3

```
$ aws s3 cp --recursive ./gui/dist s3://<BUCKET_NAME>
```

Bashoutter (./gui/dist <BUCKET_NAME> AWS



GUI Bashoutter Vue.js Vuetify UI
Vue Single page application (SPA)
handson/bashoutter/gui

Bashoutter.BucketUrl= URL (Figure 98) Public access mode S3 URL

S3 URL Figure 103

API Endpoint URL

Post your Haiku!

五

七

五

Your name

POST

People's Haikus

REFRESH

Figure 103. "Bashoutter" の GUI

このアプリケーションは "API Endpoint URL" を入力するための **API Gateway** の **URL** を入力する (このアプリケーションは **API Gateway** の **URL** を入力するための **GUI** のみを提供する) のみを提供する。"REFRESH" ボタンをクリックすると、最新のハイクを fetch する。また、ハイクに対して "like" ボタンをクリックすると、ハイクの like 数が増える。

このアプリケーションは、"POST" ボタンをクリックすると、API Gateway の URL に POST リクエストを送信する。また、"REFRESH" ボタンをクリックすると、最新のハイクを fetch する。

13.7. 本アプリケーション

この **Bashoutter** アプリケーションは、SNS のようなアプリケーションを構築するための [Section 13.5](#) のアプリケーションを拡張したものである。このアプリケーションは、API Gateway を使用して、API を呼び出すことができる。

Bashoutter アプリケーションは、API Gateway を使用して、API を呼び出すことができる。

このアプリケーションは、API Gateway を使用して、API を呼び出すことができる。

```
$ cdk destroy
```



CDK を使用して S3 バケットを削除するには、**cdk destroy** を実行する。このコマンドを実行すると、S3 バケットが削除される。

このアプリケーションは、S3 バケットを削除するために、"Actions" → "Delete" をクリックする必要がある。

このアプリケーションは、S3 バケットを削除するために、<BUCKET NAME> を指定する必要がある。このアプリケーションは、"BashoutterBucketXXXX" というバケットを削除する必要がある。

```
$ aws s3 rm <BUCKET NAME> --recursive
```

13.8.

Chapter 12 AWS

Lambda, S3, DynamoDB Chapter 13

"Bashoutter"

Chapter 14. □□□

Chapter 15. Appendix: 環境構築

この章では、本書で使用する環境の構築方法について説明します。AWS CLI、AWS CDK、Python、Docker、WSL、Linux、Mac、Windows などの環境の構築方法について説明します。

環境構築
AWS
環境構築

- AWS 環境構築 (Section 15.1)
- AWS 環境構築 (Section 15.2)
- AWS CLI 環境構築 (Section 15.3)
- AWS CDK 環境構築 (Section 15.4)
- WSL 環境構築 (Section 15.5)
- Docker 環境構築 (Section 15.6)
- Python venv 環境構築 (Section 15.7)
- Docker image を Docker image (Section 15.8)

本書で使用する OS は Linux/Mac/Windows であり、Windows 環境では Windows Subsystem for Linux (WSL) を利用します (Section 15.5)。

本書で使用する Docker、AWS CLI/CDK、Python、Docker 環境の構築方法について説明します。

15.1. AWS 環境構築

本書で使用する AWS 環境の構築方法について説明します。AWS CLI、AWS CDK、Python、Docker、WSL、Linux、Mac、Windows などの環境の構築方法について説明します。

本書で使用する AWS 環境の構築方法について説明します。Create an AWS Account (Figure 104) を参照してください。

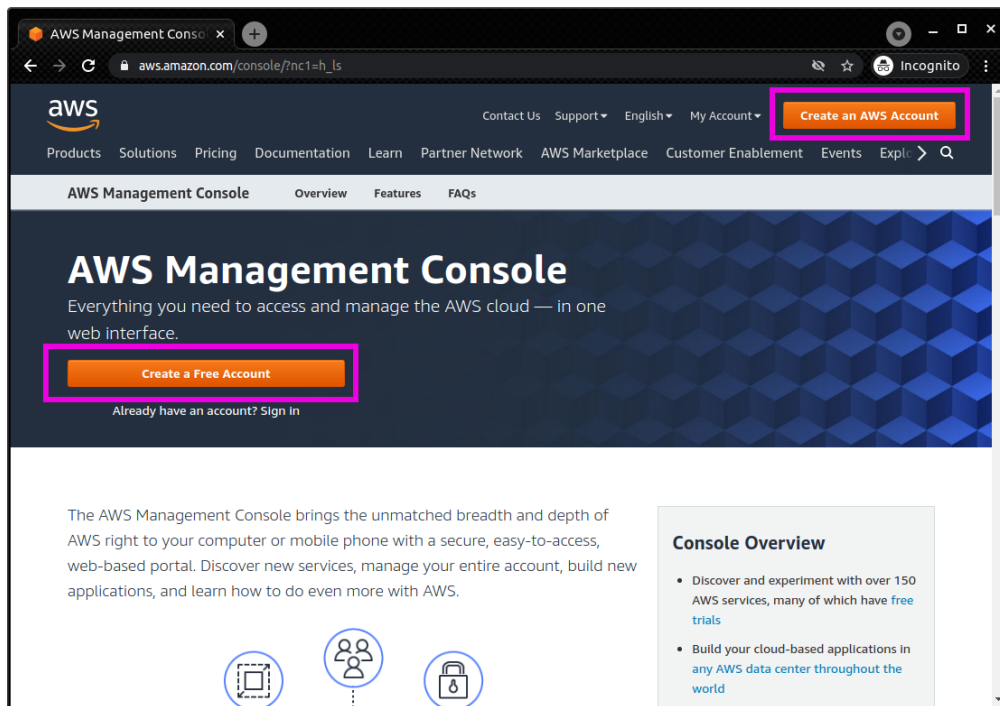


Figure 104. 스크린샷 (1): AWS 콘솔 화면

다음 단계를 위한 링크는 Figure 105에 표시되어 있습니다.

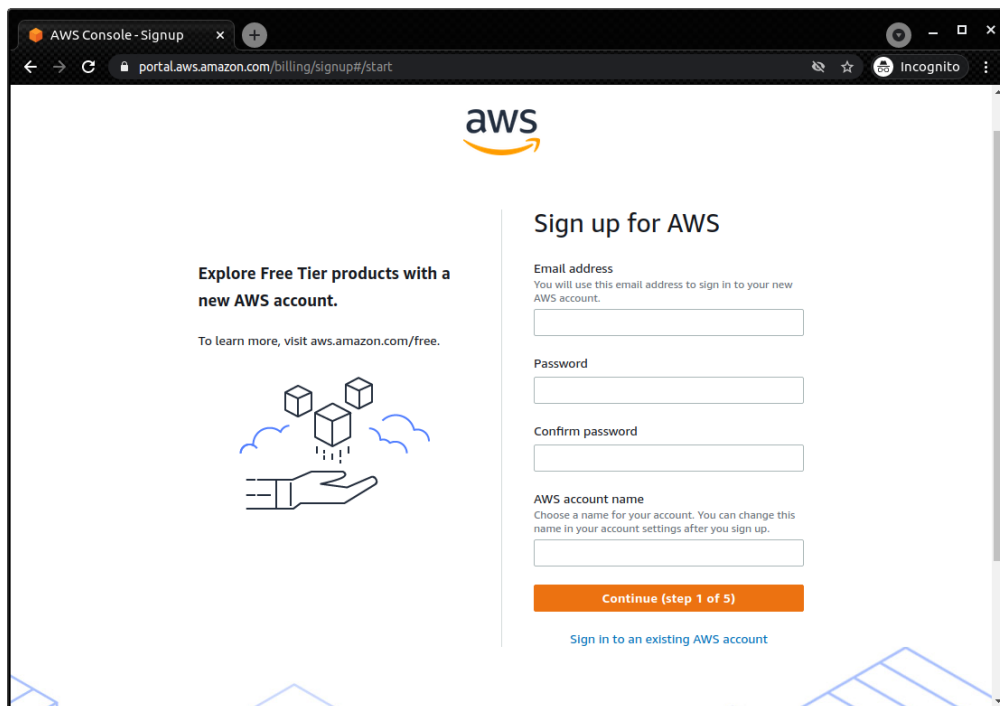


Figure 105. 스크린샷 (2): AWS 계정 생성 화면

다음 단계를 위한 링크는 Figure 106에 표시되어 있습니다.

Free Tier offers

All AWS accounts can explore 3 different types of free offers, depending on the product used.

- Always free**
Never expires
- 12 months free**
Start from initial sign-up date
- Trials**
Start from service activation date

Sign up for AWS

Contact Information

How do you plan to use AWS?

☐ Business - for your work, school, or organization

☐ Personal - for your own projects

Who should we contact about this account?

Full Name

Phone Number
Enter your country code and your phone number.

Country or Region

Address

Figure 106. ステップ (3): 連絡先情報

ステップ (3) の「連絡先情報」画面で、AWS アカウントの作成を進めます。この画面では、AWS アカウントの作成を進めるための必要事項を入力します。

Sign up for AWS

Billing Information

Credit or Debit card number



AWS accepts all major credit and debit cards. To learn more about payment options, review our [FAQ](#)

Expiration date
Month Year

Cardholder's name

Billing address
☒ Use my contact address
1919-1 Tancha, Onna-son
Kunigami-gun Okinawa 904-0412
JP

☐ Use a new address

Secure verification



 We will not charge for usage below AWS Free Tier limits. We temporarily hold \$1 USD/EUR as a pending transaction for 3-5 days to verify your identity.

Figure 107. ステップ (4): 支払い情報

ステップ (4) の「支払い情報」画面で、SMS 認証コードを入力します (Figure 108)。

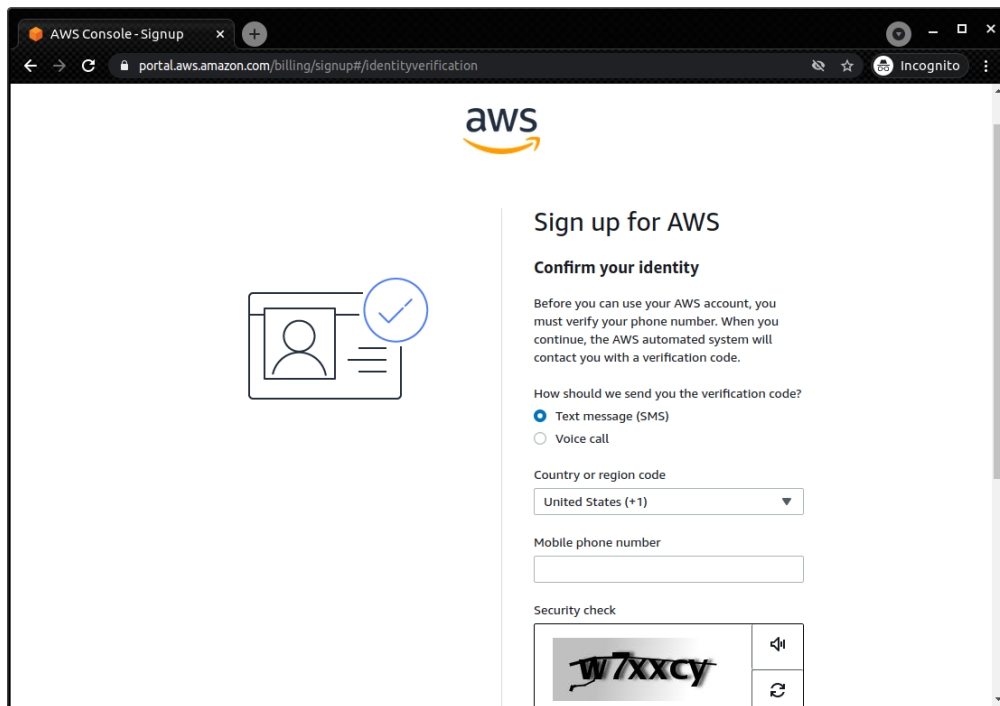


Figure 108. Step 5: Confirm your identity

After completing the identity verification step (Figure 109), you can choose the Basic support plan.

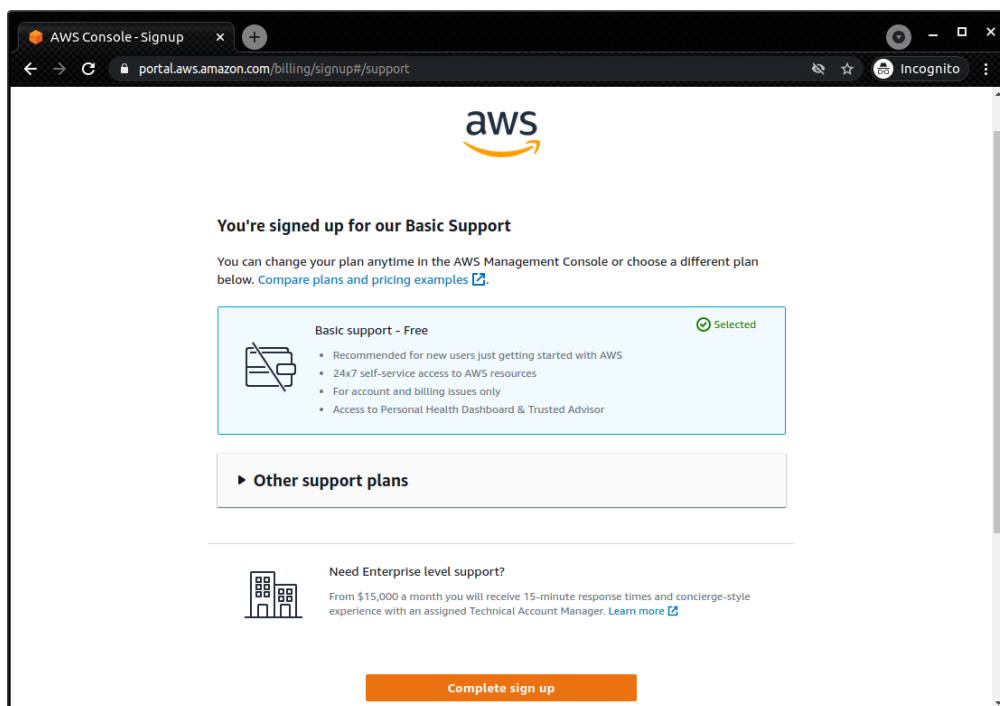


Figure 109. Step 6: Confirm your support plan

After completing the support plan selection step (Figure 110), you can complete the AWS account setup.

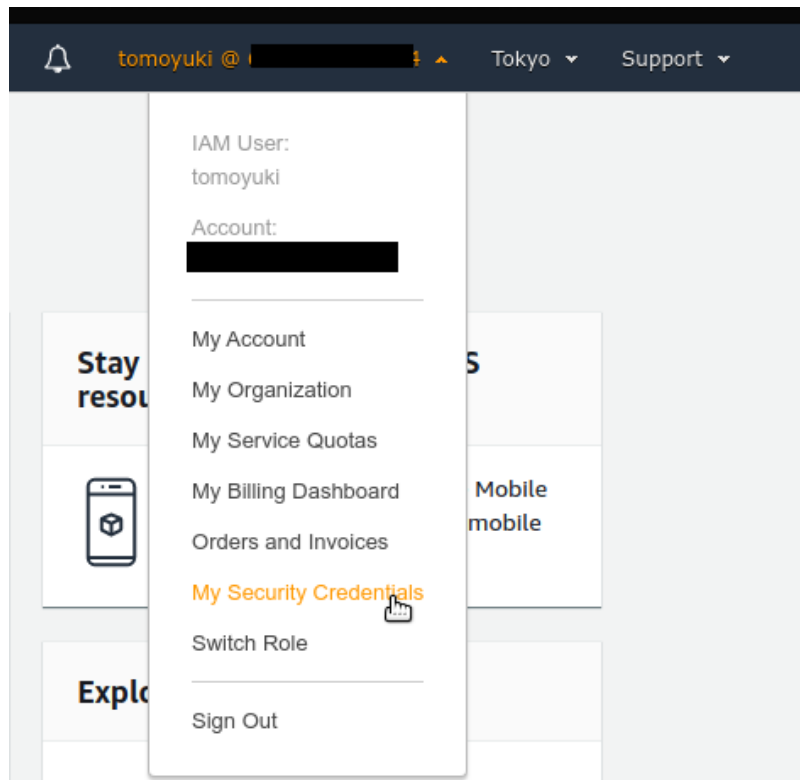


Figure 111. AWS 画面のユーザーメニュー

Access keys for CLI, SDK, & API access

Use access keys to make programmatic calls to AWS from the AWS Command Line Interface (AWS CLI), Tools for Windows PowerShell, the AWS SDKs, or direct AWS API calls. **If you lose or forget your secret key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.** [Learn more](#)

[Create access key](#)

Figure 112. AWS 画面のアクセスキー作成ボタン

AWS Educate Starter Account 環境構築の準備

- AWS Educate 環境構築の準備 **vocareum** 環境構築の準備 (Figure 113)
- **Account Details** 環境構築の準備 **AWS CLI: Show** 環境構築の準備
- **aws_access_key_id**, **aws_secret_access_key**, **aws_session_token** 環境構築の準備 (Figure 114) 環境構築の準備 **~/.aws/credentials** 環境構築の準備 (Section 15.3) 環境構築の準備 **aws_session_token** 環境構築の準備
- 環境構築の準備 **~/.aws/config** 環境構築の準備 環境構築の準備 **AWS Starter Account** 環境構築の準備 **us-east-1** 環境構築の準備



```
[default]
region = us-east-1
output = json
```

- 環境構築の準備 **default** 環境構築の準備 環境構築の準備 **default** 環境構築の準備 環境構築の準備 (環境構築の準備 Section 15.3)

Welcome to your AWS Educate Account

AWS Educate provides you with access to a wide variety of AWS Services for you to get your hands on and build on AWS! To get started, click on the AWS Console button to log in to your AWS console.

Please read the FAQ below to help you get started on your Starter Account.

- What are the list of services supported?
- What regions are supported with Starter Accounts or Classroom Accounts?

Your AWS Account Status

- Active**
full access ()
- \$100**
remaining credits (estimated)
- 1:47**
session time

Account Details AWS Console

AWS Access
Session started at: 2021-06-20T18:29:05-0700
Session to end at: 2021-06-20T21:29:05-0700
Remaining session time: 2h18m12s

AWS Starter account
Term: 364 days 23:13:23

AWS CLI:
Copy and paste the following into ~/.aws/credentials

```
[default]
aws_access_key_id=
aws_secret_access_key=
aws_session_token=
```

Figure 113. vocareum

AWS CLI:
Copy and paste the following into ~/.aws/credentials

```
[default]
aws_access_key_id=
aws_secret_access_key=
aws_session_token=
```

Figure 114. vocareum AWS

15.3. AWS CLI

(Linux) () ()

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ unzip awscliv2.zip
$ sudo ./aws/install
```

```
$ aws --version
```

()

```
$ aws configure
```

AWS Access Key ID, AWS Secret Access Key Section 15.2
Default region name (ap-northeast-1 =)
Default output format json

~~~~~ ~/.aws/credentials ~ ~/.aws/config~~~~~ cat  
~~~~~

```
$ cat ~/.aws/credentials
[default]
aws_access_key_id = XXXXXXXXXXXXXXXXXXXX
aws_secret_access_key = YYYYYYYYYYYYYYYYYY

$ cat ~/.aws/config
[profile default]
region = ap-northeast-1
output = json
```

~/.aws/credentials ~/.aws/config AWS CLI ~~~~~

~~~~~ [default] ~~~~~ default ~~~~~ [myprofile]  
~~~~~

AWS CLI ~~~~~

```
$ aws s3 ls --profile myprofile
```

~~~~~ --profile ~~~~~

~~~~~ --profile ~~~~~ AWS\_PROFILE ~~~~~

```
$ export AWS_PROFILE=myprofile
```

~~~~~

```
export AWS_ACCESS_KEY_ID=XXXXXX
export AWS_SECRET_ACCESS_KEY=YYYYYY
export AWS_DEFAULT_REGION=ap-northeast-1
```

~~~~~ ~/.aws/credentials ~~~~~ ( )



AWS Educate Starter Account us-east-1 ~~~~~ () AWS
Educate Starter Account ~~~~~ default region us-east-1 ~~~~~

15.4. AWS CDK ~~~~~

~~~~~ (Linux ) ~~~~~ ~~~~~  
~~~~~

Node.js ~~~~~

```
$ sudo npm install -g aws-cdk
```



aws-cdk	version	1.100.0	aws-cdk	version	1.100.0
aws-cdk	version	1.100.0	aws-cdk	version	1.100.0

```
$ npm install -g aws-cdk@1.100
```

aws-cdk version 1.100.0

```
$ cdk --version
```

aws-cdk version 1.100.0

```
$ cdk bootstrap
```



`cdk bootstrap` creates the AWS CLI profile and the `~/.aws/config` file. See [Section 15.3](#) for more details.



AWS CDK uses the AWS CLI profile created in [Section 15.3](#).

15.5. WSL

WSL (Windows Subsystem for Linux) is a technology that allows you to run a Linux distribution on Windows. It is a great way to get started with Linux on a Windows machine. WSL 2 is the recommended version for running Linux distributions.

WSL 2 is available on Windows 10 (Pro or Home) and Windows 11. It requires the Windows Subsystem for Linux (WSL) feature to be enabled. You can enable WSL 2 by running the following command in PowerShell:

For more information, see [WSL 2](#).

WSL 2 is available on Windows 10 (Pro or Home) and Windows 11. It requires the Windows Subsystem for Linux (WSL) feature to be enabled. You can enable WSL 2 by running the following command in PowerShell:

For more information, see [WSL 2](#).

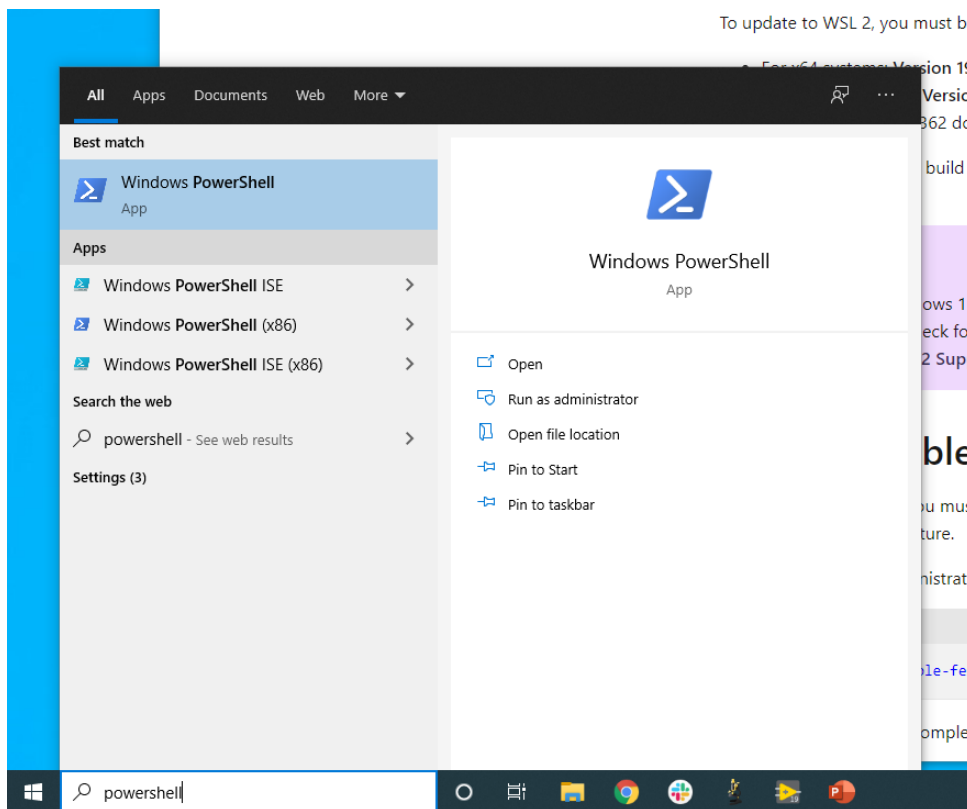


Figure 115. Windows PowerShell search

PowerShell search results

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

“The operation completed successfully.” WSL enable

Administrator PowerShell

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

“The operation completed successfully.”

Linux kernel update package https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi

PowerShell

```
wsl --set-default-version 2
```

Linux distribution Ubuntu 20.04

Microsoft store Ubuntu Ubuntu 20.04 LTS “Get”

Figure 116) Ubuntu 20.04

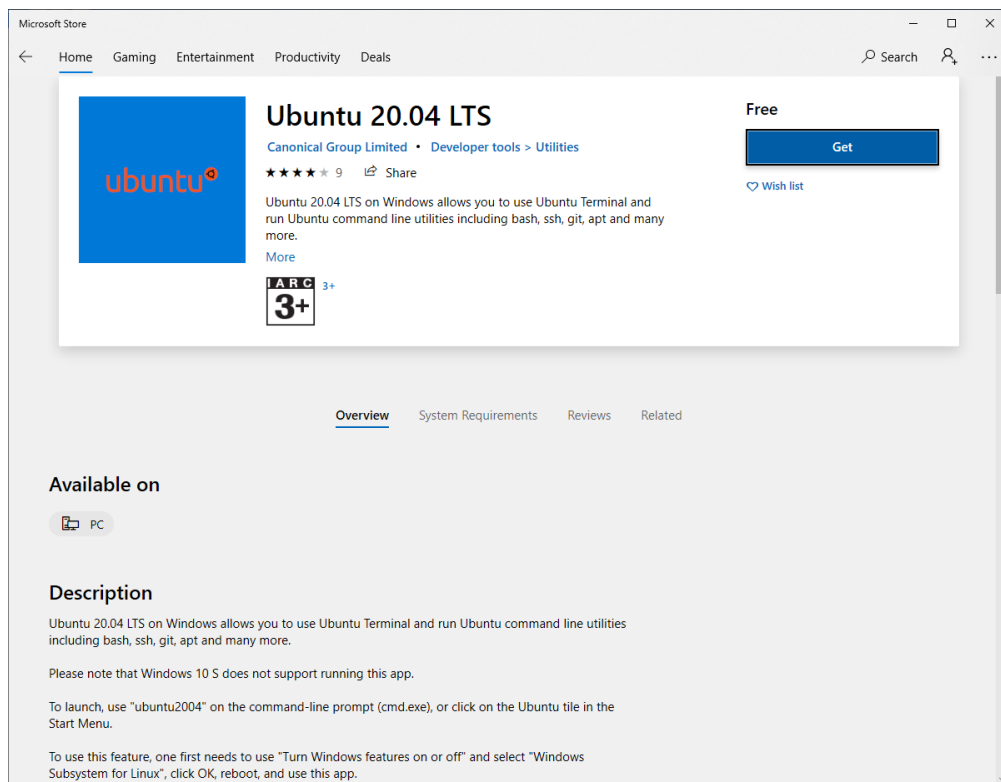


Figure 116. Microsoft store Ubuntu 20.04

Ubuntu 20.04

WSL2 Windows Ubuntu Ubuntu 20.04

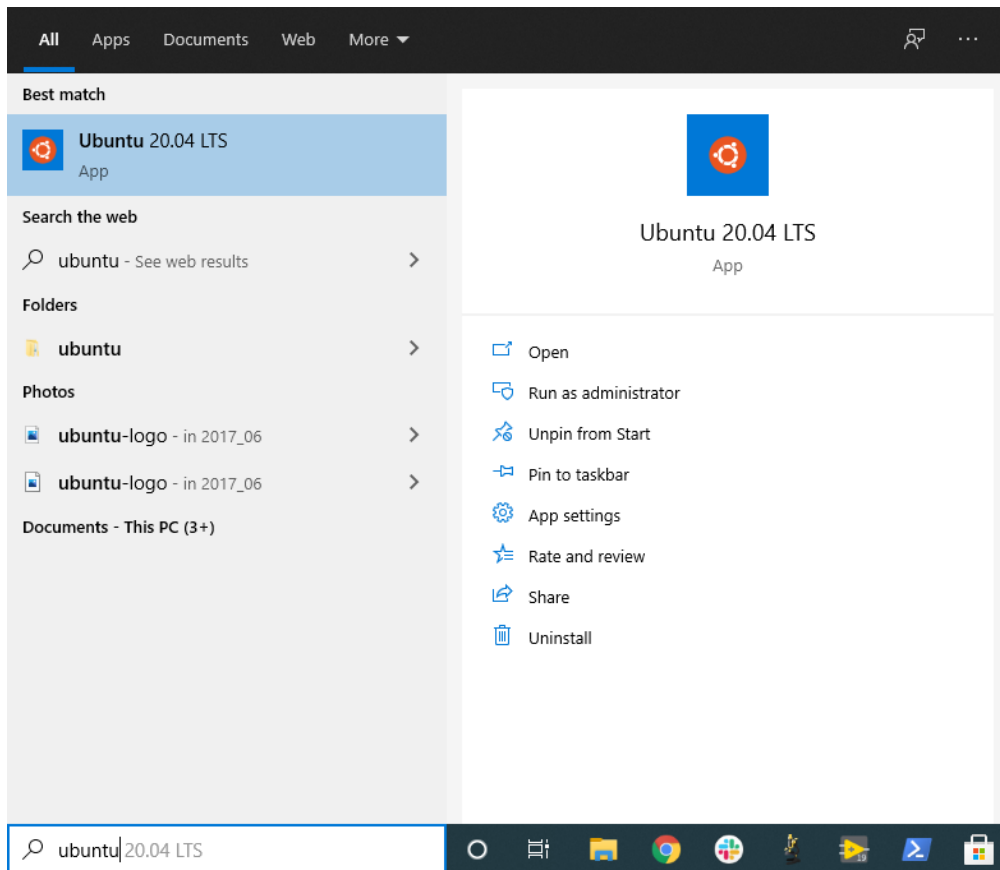


Figure 117. Ubuntu 20.04 アプリ

コマンドプロンプトで `ls, top` を実行すると WSL のコマンドプロンプトが表示されます。

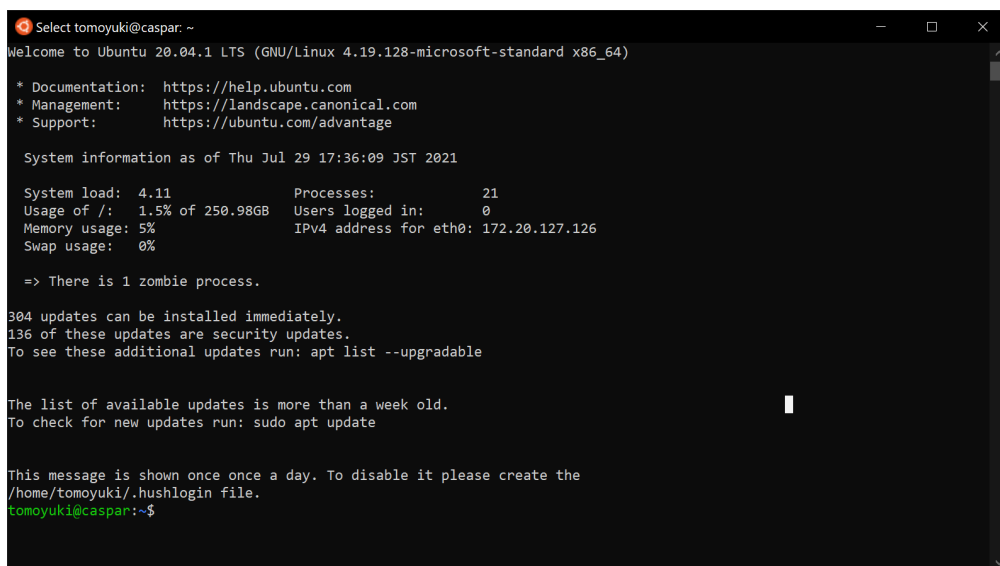


Figure 118. WSL コマンド

Windows Terminal を起動すると WSL のコマンドプロンプトが表示されます。

15.6. Docker のインストール

Docker のインストールは OS によって異なります。

Mac の場合は Docker Desktop をインストールします。Windows の場合は Docker Desktop をインストールします。

Applications をインストールする。 Docker Desktop をインストールする。

Windows Docker Desktop をインストールする。 WSL 2 をインストールする。 Docker Desktop をインストールする。 WSL 2 docker をインストールする。

Linux (Ubuntu) をインストールする。 Docker Desktop をインストールする。

Docker Desktop をインストールする。 Docker Desktop をインストールする。

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh
```

root として docker をインストールする。 sudo を使用する。 docker をインストールする。 (Post-installation steps for Linux) を参照する。

docker をインストールする。 docker をインストールする。

```
$ sudo groupadd docker
```

docker をインストールする。

```
$ sudo usermod -aG docker $USER
```

docker をインストールする。

docker をインストールする。

```
$ docker run hello-world
```

sudo を使用する。

15.7. Python venv をインストールする

numpy, scipy をインストールする。 Python をインストールする。

Python をインストールする。 venv, pyenv, conda をインストールする。

venv, Python, pyenv, conda をインストールする。

venv をインストールする。

```
$ python -m venv .env
```

このコマンドを実行すると、`.env/` ディレクトリが作成され、その中に Python の仮想環境が作成されます。

仮想環境を有効化します。

```
$ source .env/bin/activate
```

確認します。

仮想環境の有効化を確認します (`.env`)。仮想環境の有効化が完了すると、`(.env)` というプレフィックスが表示されます (Figure 119)。

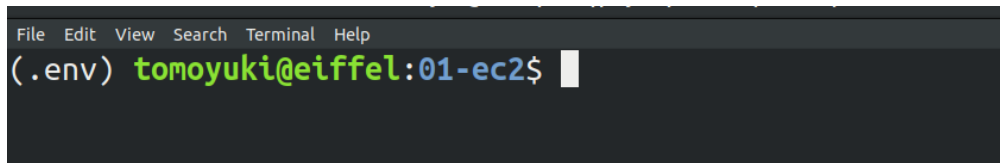


Figure 119. `venv` の有効化を確認する

仮想環境が有効化された状態で `pip` を使用して、`.env/` ディレクトリに依存関係をインストールします。

Python の `requirements.txt` ファイルを読み込んで、仮想環境に依存関係をインストールします。

```
$ pip install -r requirements.txt
```

仮想環境が有効化された状態で Python の仮想環境を確認します。



`venv` の有効化を確認します。仮想環境の有効化が完了すると、`.env` というプレフィックスが表示されます。

15.8. Docker image のインストール

Node.js、Python、AWS CDK などのアプリケーションを Docker image としてインストールします。



アプリケーションを Docker image としてインストールする。これは、アプリケーションを Docker image としてインストールする。

Docker をインストールし、Docker Hub から Docker image をプルします。GitHub の `docker/Dockerfile` を参照してください。

仮想環境の有効化を確認します。

```
$ docker run -it tomomano/labc:latest
```

仮想環境の有効化を確認します。Docker Hub から Docker image (pull) をプルします。

コンテナを実行する (インタラクティブモードで実行)

```
root@aws-handson:~$
```

手元の `ls` コマンドで `handson/` ディレクトリを確認し、`cd` で移動

```
$ cd handson
```

仮想環境をインストール

仮想環境をインストールする `virtualenv` (Section 4.4) をインストールする
仮想環境をインストールする `virtualenv` をインストール

AWS 認証情報を `Section 15.3` の `AWS_ACCESS_KEY_ID` と `AWS_SECRET_ACCESS_KEY` を
`~/.aws/credentials` に保存する
仮想環境を実行する

```
$ docker run -it -v ~/.aws:/root/.aws:ro tomomano/labc:latest
```

手元の `~/.aws` ディレクトリを `/root/.aws` にマウントし、`:ro` (read-only) で
仮想環境を実行する `read-only` モードで実行



`/root/` ディレクトリを `read-only` モードでマウントし、`SSH` を実行

Chapter 16. 謝辞

本書の作成に協力した方々に感謝します。

2021年 contributors

- 山田 太郎 - 校正

2020年 contributors

- 山田 太郎 - Docker 環境構築
- 山田 太郎 - 校正
- @shuuji3 - MR !15
- @takatama_jp - MR !14

本書は [Asciidoctor](#) で生成されています。

本書のソースコードは [GitHub](#) に公開されています。 [Issues](#) を開くと、[Pull request](#) を提出することができます。

Chapter 17. 閉鎖型

Tomoyuki Mano (Tomoyuki Mano)

本書は、(株)オライオンファーマセウティクス(株) 2021年10月現在、(PD) (株) オライオンファーマセウティクス (OIST) によって提供されています。本書は、OISTによって提供されています。

連絡先: tomoyukimano@gmail.com

GitHub: <https://github.com/tomomano>

Chapter 18. 著作権

本書は [CC BY-NC-ND 4.0](#) 著作権で保護されています

本書は、著作権者の許可なく、複製、改変、配布、または他の目的で利用することを禁じます。



```
<style>
  .imageblock > .title {
    text-align: inherit;
  }
</style>
```